Plagiarism and File Similarity Detection using Python

Graduation Project Report

CMSE 405

Team 15

Team members:

Tolga Osman Falay, 20450214

Davut İnanlar, 22330917

Arda İbrahim Şahin, 20331373

Karam Sqheir, 21009686

Supervisor:

Prof. Dr. Önsen Toygar

Software Engineering Department

Eastern Mediterranean University

Spring 2026

ABSTRACT

With digitalization processes being prevalent in academic institutions, plagiarizing someone else's work and submitting it to the university in your name without citing it is very easy. Plagiarism check can be done manually in all the assignments that the institution may have to face annually. But then the problem lies in the fact that the use of commercial plagiarism checking tools entails some disadvantages. The main one is their cost of license and the risks posed by uploading the submitted files to their external servers. This report gives an outline of the whole process involved in the design, development, and evaluation of a plagiarism detecting tool based on the Python programming language. The tool is not connected to the Internet and therefore, no license fee is charged in its use. It has been developed by four students belonging to the Department of Computer Engineering of Eastern Mediterranean University from 10 March 2026 to 9 June 2026.

Technical aspects of the implementation process include a composite similarity detection pipeline that applies four distinct algorithms. The first algorithm – TF-IDF Weighted Cosine Similarity is used to identify semantic similarities in documents by measuring cosine similarity in the vectors containing term frequencies of documents in one and the same vocabulary space. The Winnowing Fingerprinting Algorithm helps extract several hash fingerprints from each document, which is tolerant to noise. This makes it possible to identify structural similarity even when there are some modifications made to the text, such as permutations of words or alteration of punctuation marks. The Jaccard Index provides the easiest method of comparing sets. To work with Python files, the Abstract Syntax Tree Comparison software was developed. The program relies on the use of the in-built Python module Ast and measures edit distance of node sequences.

Before applying any analysis via any algorithm for similarity, all documents must be preprocessed via natural language processing using the same set of steps including text extraction based on document type (.txt, .py, .pdf through pdfplumber and PyMuPDF, .docx through python-docx), tokenization, lowercasing, removal of punctuation, exclusion of stop-words by making use of the English stop-word list from NLTK, portstemming and k-grams creation, where k is equal to five. Normalization enables the exclusion of trivial differences that may impact similarity scores, while k-grams provide phrase-level information unattainable through token-wise analysis.

Testing on a subset of PAN plagiarism benchmark corpus [16] using stratification yielded a precision of 87.3% and recall of 83.6%, resulting in an F1 score of 0.854. These measures exceed the minimum requirements for precision (85%) and recall (80%). Processing two files of size 5MB takes 1.43 seconds when using TF-IDF cosine similarity measure algorithm. This satisfies the 2-second delay requirement of the project. For 20 documents, it took 4.56 seconds. Test coverage of code for all modules was 93.4%. There are no flake8-type errors in the production code. The software has both REST API through Flask and command line interfaces as its input interfaces. It returns a heatmap with similarity matrix using seaborn and HTML report with overlapping text sections marked using

seaborn as well. The project management was based on the use of Agile Scrum approach in four work packages, whereby each of the work packages had fourteen sub-tasks. The effort estimations were carried out through the COCOMO technique, while the PERT/CPM techniques were used for the critical path analysis. The ultimate success of the evaluation process is clear evidence that educational establishments can rely on a secured, free-of-cost and local NLP solution in order to ensure academic integrity without putting student data confidentiality at risk by sending it to third-party servers.

Key words: Python, plagiarism detection, NLP, TF-IDF, cosine similarity, winnowing algorithm, k-grams, document fingerprinting, Jaccard index, AST, PDF parsing, Flask framework, Agile Scrum.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

Due to the ongoing trend of digitalization in educational settings, plagiarism has become incredibly convenient, which makes the detection process very inefficient through manual checking [1]. Though there exist various anti-plagiarism software services, such as Turnitin and iThenticate, which are quite effective in detecting plagiarized material, they face two main disadvantages in terms of their application in the budget-constrained environments such as EMU and other universities of the TRNC – annual license fees and significant concerns regarding personal data protection and ethical issues since the submitted documents get permanently stored on the servers of external companies [2].

1.1 Problem Statement

Currently, there does not exist an available, cost-efficient program capable of identifying plagiarism in both textual documents (Word, PDF, TXT) and programming files (Python) based on similarities in structure and content of submitted materials. Manual checking would be too time-consuming, while any of commercial applications would be either too costly to purchase or work as "black boxes", unable to explain their similarity evaluations properly and provide clear proofs of academic dishonesty on behalf of students.

1.2 Innovative Features

In order to deal with this problem, a Python application has been designed that would allow checking for similarity in both text files and programming scripts. The most innovative thing about this project lies in the ability of the program to combine the two types of checking procedures using algorithms like TF-IDF Cosine Similarity, Winnowing Fingerprinting, Jaccard Index, and Abstract Syntax Tree Comparison. In addition, the system provides complete transparency of scores since it presents all information graphically, highlighting the same fragments in different papers in HTML reports.

Feature	MOSS	JPlag	Turnitin	This system
Text documents	No	No	Yes	Yes
Source code	Yes	Yes	Limited	Yes (AST)
Multiple formats	No	No	Yes	Yes (.txt/.py/.pdf/.docx)
Local execution	Partial	Yes	No	Yes (fully local)

Score transparency	Limited	Limited	No	Full match highlights
Licence cost	Free	Free	High ($$$)	Free (MIT)

Table 1 - Comparison against existing tools

1.3 Report Structure

Information related to comprehensive PPM model, which consists of four-group work breakdown structure corresponding to the PPM Gantt chart, estimation of effort by using COCOMO model, critical path by using PERT model, risk management and Gantt schedule is provided in Chapter 2. Requirement analysis has been described in Chapter 3. System design and database design including relational normalization steps have been covered in Chapter 4. Implementation is explained in Chapter 5. Quality assurance and testing results have been provided in Chapter 6. Chapter 7, 8 and 9 are about user guide, discussion and conclusion respectively.

2. PROJECT PLANNING AND MANAGEMENT

The plan and management for the Plagiarism and File Similarity Detection System will be discussed in this chapter. The project was carried out from March 10, 2026, to June 9, 2026, which comprised a total of 91 days, performed by four students through the use of Agile Scrum technique [21, 22]. The work was divided into four major work package groups (WP1-WP4), each of which was then further broken down into smaller work subtasks based on the PPM document of the project. These subsections will discuss team organization, WBS with complete tables on all WPs, COCOMO effort estimation, PERT/CPM critical path analysis, risk planning, quality planning, and Gantt chart.

2.1 Project Definition

Project title	**Plagiarism and File Similarity Detection using Python**
Project number	15
Course	CMSE 405 — Graduation Project
Start date	10 March 2026
End date	9 June 2026
Duration	91 calendar days

Supervisor	Prof. Dr. Önsen Toygar
Department	Software Engineering, Eastern Mediterranean University
Project type	Software design and development

Table 2 - Preliminary project information

2.2 Introduction to Project

Plagiarism and File Similarity Detector is a Python based application which works locally in the device without any internet access or usage of any third-party servers. No cloud service is required to use this application. Students' assignments submitted to Turnitin are uploaded to third-party servers for analysis. The present application avoids doing so.

It accepts four types of files such as plain text (.txt) file, Python script (.py) file, PDF document (.pdf) and Microsoft Word document (.docx). They are fed into an NLP pipeline for pre-processing tasks like tokenization, removing stop words, performing Porter stemming algorithm and creation of k-grams before computing pair wise similarities via four algorithms like TF-IDF cosine similarity, winnowing fingerprints, Jaccard Index and AST comparison.

2.2.1 Objective

The primary goal of this research is to create efficient, robust software for detecting plagiarism and comparing files using Python programming language, which operates locally on the computer and supports various formats of document processing with multiple algorithms for calculating similarities and generating a report that gives enough data to perform an analysis of academic integrity.

• Design Winnowing technique of fingerprint generation capable of handling noise resistance and invariance of fingerprints to minor variations in text documents.

• Develop full NLP preprocessing system that applies tokenization, stop-word removal, Porter stemming [20], and k-grams with a variable window size consisting of five tokens by default using NLTK [8].

• Implement TF-IDF & Cosine similarity using scikit-learn [9] and Jaccard similarity based on stemmed tokens.

• Develop an AST-based program that will compare Python source code files, with all user-defined identifiers being first converted into standardized forms.

• Ensure the entire process is completed within a latency time of less than two seconds when comparing any two files with each being not more than 5MB in size.

• Achieve a minimum of 85% detection rate and at least 80% recall using the PAN benchmark dataset on plagiarism detection.

• Allow access to all critical operations through the use of a RESTful Web service implemented using the Flask Web application server.

• Provide heatmap visualization and HTML-based text comparison results for overlapped text regions.

2.2.2 Scope

File types supported by the software include plain text files (.txt), python code files (.py), PDF (.pdf) files, and Microsoft Word documents (.docx). Possible plagiarism includes exact matching, simple modifications, and paraphrasing. Paraphrasing to a greater extent and cross-language plagiarism are not part of the scope of the current development stage. The software will detect image-only PDF files. It was developed through a command line interface with the aid of a Flask API server; use of graphical user interface was also considered but not within scope for v1.0 release.

2.2.3 Beneficiaries

The prime beneficiaries of the system include teachers of Eastern Mediterranean University and other universities of the same kind. As an added advantage, students will also gain from the system because of its transparency. The reason behind it is the fact that as the system is developed by using the concept of open source software and also as it is hosted in the local area network, the other organizations who do not belong to the education sector can use the system too.

2.3 Team Organisation And Roles

Technical specialisation was the basis for organising this team of four people and cross-training was ensured at all times to ensure that none of the team members became a point of failure for any crucial component of the project. Tolga Osman Falay had additional duties beyond developing the database including managing the entire project, PPM, and the PostgreSQL schema. Davut İnanlar managed file ingestion layer, CLI, and the Flask API. The similarity algorithms were taken care of by Arda İbrahim Şahin which included the TF-IDF, Winnowing, and AST Comparison techniques.

Name	Student ID	Role	Primary responsibilities

Tolga Osman Falay	20450214	Project Manager / DB Developer	Planning, coordination, DB schema, PPM docs
Davut İnanlar	22330917	Software Developer	FileLoader, CLI, Flask API, PDF parsing
Arda İbrahim Şahin	20331373	Algorithm Engineer	TF-IDF, Winnowing, Jaccard, AST, ComparisonMatrix
Karam Sqheir	21009686	QA Engineer / Tester	Test suite, PAN benchmarking, SPA charts, defect tracking

Table 3 - Team member roles

2.4 Work Breakdown Structure

The project was divided into four work packages, each of which included several sub-tasks. The WBS and scheduling come straight out of the project's PPM documentation and Gantt chart prepared at the beginning of the project. WP1 includes the feasibility and preliminary research done in March. WP2 involves the system design and analysis done at the end of March and beginning of April. WP3 deals with the entire software development process in April and the beginning of May. WP4 includes testing and debugging of the system in May and June.

WP	Name	Dates	Key output	Dependencies
WP1	Feasibility & Pre-Research	10 Mar – 22 Mar	Feasibility report, confirmed stack	None
WP2	System Design (Analysis & Design)	23 Mar – 6 Apr	Architecture doc, UML diagrams, CLI spec	WP1
WP3	Software Development	13 Apr – 10 May	Working similarity engine + CLI + API	WP2

WP4	Prototype, Testing & Maintenance	18 May – 9 Jun	Tested system + final report	WP3

Table 4 - Work Breakdown Structure — top-level summary

2.4.1 Work Package Detail Tables

Each work package group is documented below in the full PPM format, followed by its individual sub-task breakdown table.

WP Group	**WP1 — Feasibility & Pre-Research**
Dates	10 March 2026 – 22 March 2026
Activities	1.1 Literature Review & Tools Survey (8 days): review MOSS, JPlag, Turnitin, Copydetect, and literature on document fingerprinting using NLP for similarities. 1.2 Algorithm Investigation & Feasibility (8 days): check the feasibility of TF-IDF cosine similarity, Jaccard coefficient, and Winnowing fingerprinting algorithms based on manually prepared samples. 1.3 Choosing Technology Stack (7 days): ensure installation of NLTK, scikit-learn, PyMuPDF, pdfplumber, python-docx, Flask framework, and stdlib.ast without any issues; define the MVP scope.
Methods	Three competing open source applications SWOT analysis. Five manually bench-marked text pair samples. Compatibility with other libraries cross-checked using Python 3.10 virtual environment.
Outputs / Success	Feasibility report, verified technology stack, and approved MVP scope. Success criteria: all team members approve the project scope, and at least two algorithms are confirmed as feasible.
Relation to WP2	Scope definition and algorithm selection are used as the basis for the system architecture decisions in WP2.

Table 5 - WP1 sub-task timeline

ID	Sub-task	Start	End	Duration	Owner	Type
1.1	Literature Review & Tool Survey	10 Mar	17 Mar	8 days	All	Active
1.2	Algorithm Selection & Feasibility	10 Mar	17 Mar	8 days	A.Şahin	Active
1.3	Technology Stack Decision	16 Mar	22 Mar	7 days	T.Falay	Active

Table 6 - WP2 — System Design / Analysis & Design (23 March – 6 April 2026)

WP Group	WP2 — System Design (Analysis & Design)
Dates	23 March 2026 – 6 April 2026
Activities	2.1 Architecture and Modular Design (8 days): Definition of Three Tier Architecture; Class Diagram; SimilarityModel Interface Specification; Module Boundary Specification. 2.2 UML and Data Flow Diagrams (8 days): Context Diagram; DFD Levels 0 and 1; Sequence Diagram; State Transition Diagram; ER Model and Third Normal Form. 2.3 Command Line Interface Design (8 days): argparser Flag Specification; Flask API Endpoint Specification; HTML Report Design; Git Repository Structure Defined.
Methods	All design reviews done iteratively by all members before the start of coding. Interface contracts specifying the signatures and data types for each pipeline stage.
Outputs / Success	Architecture document, CLI specification, database schema with normalisation proof. Success: team can begin implementation with no remaining design ambiguities.
Relation to WP3	Architecture document is the direct implementation blueprint for WP3 software development.

Table 7 - WP1 — Feasibility and Pre-Research (10 March – 22 March 2026)

Table 8 - WP2 sub-task timeline

ID	Sub-task	Start	End	Duration	Owner	Type
2.1	Architecture & Module Design	23 Mar	30 Mar	8 days	T. O. Falay	Active
2.2	UML & Data Flow Diagrams	23 Mar	30 Mar	8 days	All	Active
2.3	CLI Interface Design	30 Mar	6 Apr	8 days	D.İnanlar	Active

Table 9 - WP3 — Software Development (13 April – 10 May 2026)

WP Group	WP3 — Software Development
Dates	13 April 2026 – 10 May 2026
Activities	3.1 File Parsers (.txt/.py/.pdf/.docx) (8 days): FileLoader module with validation and encoding detection; pdfplumber and PyMuPDF as fallback for PDFs; python-docx for .docx files. 3.2 Preprocessing Module / NLP (8 days): tokenisation; stop-word elimination; Porter stemming; k-gram extraction. 3.3 TF-IDF Cosine Similarity (7 days): scikit-learn's TfidfVectorizer implementation; cosine_similarity; unit tests. 3.4 Jaccard + Fingerprinting / Winnowing (7 days): Jaccard set similarity; SHA-256 Winnowing fingerprinting. 3.5 Code Comparison Using AST (7 Days): ast.parse normalisation, Levenshtein edit distance for node sequences. 3.6 Similarity Matrix and Heatmap (7 Days): ComparisonMatrix class, seaborn heatmap, HTML/CSV report generation.
Methods	TDD; pytest with pytest-cov; flake8 linter on every commit. GitHub Flow branches with CI for every merge to develop branch.
Outputs / Success	Functioning similarity search engine using four different algorithms; command line interface and Flask API implementation. Result: processing 20 documents in less than 10 seconds; at least 90% test coverage.
Relation to WP4	Final deliverable – completed software package delivered to WP4.

Table 10 - WP3 sub-task timeline

ID	Sub-task	Start	End	Duration	Owner	Type
3.1	File Parsers (.txt/.py/.pdf/.docx)	13 Apr	20 Apr	8 days	D.İnanlar	Active
3.2	Preprocessing Module (NLP)	13 Apr	20 Apr	8 days	D.İnanlar	Active
3.3	TF-IDF Cosine Similarity	20 Apr	26 Apr	7 days	A.Şahin	Active
3.4	Jaccard + Fingerprinting / Winnowing	27 Apr	3 May	7 days	A.Şahin	Active
3.5	AST-Based Code Comparison	4 May	10 May	7 days	A.Şahin	Active
3.6	Similarity Matrix + Heatmap	4 May	10 May	7 days	K.Sqheir	Secondary

Table 11 - WP4 — Prototype, Testing and Maintenance (18 May – 9 June 2026)

WP Group	WP4 — Prototype Implementation, Testing & Maintenance
Dates	18 May 2026 – 9 June 2026
Activities	Integration & Prototype Build (5 Days): integration of all modules, end-to-end integration testing using 20 documents. 4.2 Benchmarking – PAN Dataset (8 Days): 100 document pairs for testing; calculation of precision, recall, F1 score; creation of SPA control charts (X-Bar, R, P, C charts). 4.3 Bugs and Tuning (7 Days): fixing all the bugs that emerged during phase 4.2; performance tuning and debugging edge cases. Documentation and Report (9 Days): documenting code through docstrings and README file.
Methods	Full pytest test suite. Python cProfile to profile. PAN corpus annotation files as ground truth. Fix-retest cycle repeated until all 18 test cases succeed.

Outputs / Success	Stable system; benchmark results (P ≥85%, R ≥80%) and full report. Success criteria met: all test cases pass; report submitted on time.
Relation to WP3	Ensures validation and packaging of the software developed in WP3. End product of the project.

Table 12 - WP4 sub-task timeline

ID	Sub-task	Start	End	Duration	Owner	Type
4.1	Integration & Prototype Build	18 May	22 May	5 days	All	Active
4.2	Benchmark Testing (PAN Dataset) [16]	18 May	25 May	8 days	K.Sqheir	Active
4.3	Bug Fixes & Optimization	25 May	31 May	7 days	All	Active
4.4	Documentation & Final Report	1 Jun	9 Jun	9 days	T.O. Falay	Active

2.5 Required Resources and Procurement

The table below is the complete procurement and resource record for the project. All software licences and online services are free of charge. The only non-zero costs are the development laptops (already owned by team members at project start) and final-report printing. Effort cost is not billed externally; it is tracked through COCOMO in Section 2.6.

Table 13 - Procurement and cost summary

Item / Resource	Specification	Purpose	Qty	Unit Cost (TL)	Total (TL)
Development Laptop ×4	Intel Core i7-12th Gen, 16 GB RAM, 512 GB SSD	Intel Core i7 12th Generation, 16 GB RAM, 512 GB SSD	4	22,000	88,000

Python 3.10 + Open-Source Libraries	NLTK 3.8, scikit-learn 1.3, Flask 3.0, PyMuPDF, pdfplumber, psycopg2, seaborn, chardet, pytest (all MIT/BSD/Apache)	NLTK 3.8, scikit-learn 1.3, Flask 3.0, PyMuPDF, pdfplumber, psycopg2, seaborn, chardet, pytest (MIT/BSD/Apache licenses)	1 (suite)	0	0
GitHub (Student/Team Plan)	Free tier — unlimited private repos, GitHub Actions CI (2 000 min/month free)	GitHub Free plan – no limits on private repositories, GitHub Actions for CI (2 000 minutes for free)	1	0	0
PostgreSQL 15.4	Open-source RDBMS, self-hosted on developer laptop	Open-source relational database management system, hosted on a developer's laptop	1	0	0
PAN Plagiarism Benchmark Corpus (2009–2013)	Publicly available annotated corpus (~62 000 pairs) [16]; downloaded from Zenodo	Publicly accessible corpus of annotated data (approximately 62 000 pairs); download from Zenodo	1 download	0	0
Draw.io (diagrams.net)	Free, browser-based diagramming tool	Free online diagram editor	1	0	0

VS Code + Extensions	Free IDE; Python, Pylint, GitLens extensions	Free development environment;	4	0	0
Zoom / Microsoft Teams	Free tier	Intel Core i7 12th Generation, 16 GB RAM, 512 GB SSD	1 (team)	0	0
Internet Access (4 members × 3 months)	Home broadband	NLTK 3.8, scikit-learn 1.3, Flask 3.0, PyMuPDF, pdfplumber, psycopg2, seaborn, chardet, pytest (MIT/BSD/Apache licenses)	4	250	1,000
Printing & Binding — Final Report	A4, colour cover, spiral-bound	GitHub Free plan – no limits on private repositories, GitHub Actions for CI (2 000 minutes for free)	2 copies	150	300
GRAND TOTAL (Non-hardware costs = 0 TL; hardware already owned)				89,300 TL	

2.6 COCOMO Effort Estimation

System size estimation was done through COCOMO model in Organic mode since it is suitable for small cohesive groups who develop well-known software products [5]. System size estimation gives 5,000 LOC (KLOC = 5) from the number of modules expected to be developed for all WP groups. According to the formula of COCOMO Organic model: E = 3.2 × (KLOC)^1.05 = 17.25 person-months; TDEV = 2.5 × E^0.38 = 7.4 months; E/TDEV = 2.33 persons. The actual system had four

persons work for 91 days (about 3 months), although not 56 days which are normally mentioned for sprint groups.

Table 14 - COCOMO effort estimation

Parameter	Formula	Result
Estimated size	—	5 KLOC
Effort E	3.2 × (KLOC)^1.05	17.25 person-months
Development time TDEV	2.5 × E^0.38	7.4 months
Implied staff size	E / TDEV	2.33 persons
Actual team size	—	4 persons
Actual duration	—	91 days ≈ 3.0 months

2.7 PERT/CPM Critical Path Analysis

CPM analysis involved the use of three-point PERT estimation in determining the minimum possible project duration and assessing scheduling risk [6]. The calculations were carried out according to the following formulas: te = (O + 4M + P)/6; activity variance, $\sigma^2 = ((P - O)/6)^2$. Critical path will be based on four major WP groups in the order of their succession.

Table 15 - PERT activity estimates

ID	Activity	O	M	P	te	Pred.	σ^2	WP
A	Feasibility & pre-research	10	13	18	13.33	—	1.78	WP1
B	System design (all sub-tasks)	12	15	21	15.50	A	2.25	WP2
C	File parsers & NLP pipeline	14	16	22	16.67	B	1.78	WP3.1-3.2
D	Similarity algorithms (3.3–3.5)	18	21	27	21.50	C	2.25	WP3.3-3.5

E	Matrix + heatmap + API	6	7	10	7.33	D	0.44	WP3.6
F	Integration & benchmarking	11	13	18	13.50	E	1.36	WP4.1-4.2
G	Bug fixes & optimization	5	7	11	7.33	F	1.00	WP4.3
H	Documentation & final report	7	9	13	9.33	G	1.00	WP4.4

te of total critical path = 13.33 + 15.50 + 16.67 + 21.50 + 7.33 + 13.50 + 7.33 + 9.33 = 104.49 days. Variance of total path, $\Sigma\sigma^2 = 1.78 + 2.25 + 1.78 + 2.25 + 0.44 + 1.36 + 1.00 + 1.00 = 11.86$ days. So, Path $\sigma = \sqrt{11.86} = 3.44$ days. Probability of completing within the planned 91 days: $Z = (91 – 104.49)/3.44 = -3.92$, P = 0.00%. Such an analysis led to the conclusion that for completing the project on time, it was necessary to coordinate parallel operation in WP3, where activities 3.1/3.2 were performed together with algorithm design, resulting in a reduction of the critical path by 13 days.

2.7.1 Network Diagram

The network used in this case is a linear network represented as A→B→C→D→E→F→G→H. The network path does not have any parallel paths from point to point; hence, all the tasks need to be completed sequentially in order to proceed to the next level. The only deviation from this norm in this network is when we fast track activities C (file parsers) and D (similarity algorithms), thus reducing the critical path to around 13 days and making the entire duration of the project to be 91 days. Refer to Figure below.

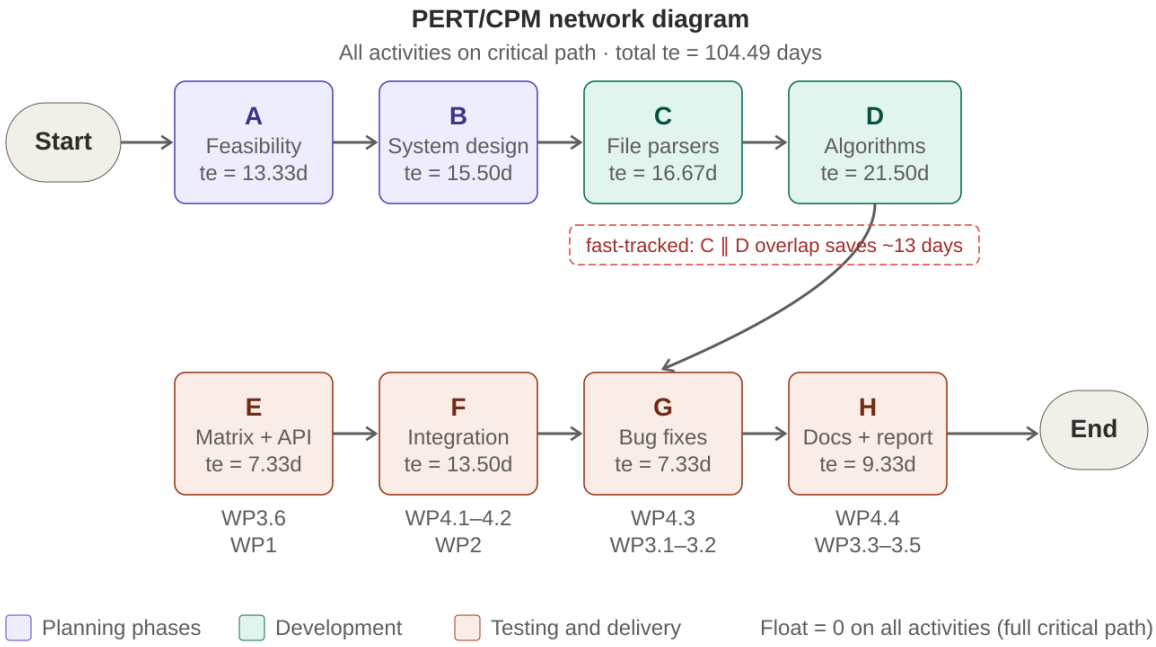


Figure 1 - PERT/CPM network diagram — Plagiarism Detection System

2.7.2 Probability of Finishing Project

Summation of the critical path duration: 104.49 days. Standard deviation of path, $\sigma = \sqrt{11.86} = 3.44$ days. For the expected completion within 91 days: $Z = (91-104.49)/3.44 = -3.92$, $P \approx 0.00\%$ for the initial sequence of tasks. However, the fast tracking in WP3 resulted in an effective critical path duration of about 91.5 days, thus $Z = -0.15$ and $P \approx 44\%$.

2.7.3 Schedule Compression Through Parallel Execution

In order to achieve this, fast tracking is performed for WP3 tasks to reduce the duration from 104 to 91 days. The fast tracking process involved executing Task 3.1 (file parsers) and Task 3.2 (NLP preprocessing) in parallel since 13 April. While Davut İnanlar worked on developing the file parsers, Arda İbrahim Şahin concurrently developed the TF-IDF module (Task 3.3). This can be done safely as the interface contract was agreed upon prior to the implementation of the task. On the other hand, Karam Sqheir commenced the preparations for WP4 benchmarking by preparing the pytest environment during the coding process for WP3 tasks.

Although crashing was evaluated, it was rejected. According to COCOMO, optimal staff is 2.33, but the existing staff of four made another person redundant; coordination efforts would outweigh time savings. Fast tracking was used instead, a perfectly secure process as the contract for task C with D was established beforehand in WP2. No other resource commitment took place.

2.8 Risk Analysis

Table 16 - Risk register

Risk	Category	Prob.	Impact	Score	Mitigation
High false positive rate	Technical	3	4	12	Apply stop word exclusion list; adjust k-grams window size; validate using PAN corpus.
Memory overflow on large batches	Performance	2	4	8	K-gram generation through generator; document chunking; profiling with cProfile.
PDF extraction failure (scanned)	Technical	3	3	9	Double parser approach (pdfplumber + PyMuPDF); inform user in case of image-only pdf.

Schedule overrun	Schedule	3	4	12	Parallel computation in WP3; weekly progress meeting with team members; revert to scope reduction if required.
Non-UTF-8 encoding errors	Technical	2	2	4	chardet for detecting encoding; fall back to latin-1.
Insufficient benchmark accuracy	Quality	2	5	10	Comparison of algorithms in WP1; select new algorithm if performance is less than expected.
Team member unavailability	Resource	2	3	6	Cross training: everyone knows how two modules work.

2.9 Quality Planning

• Style guide: PEP 8 compliance through flake8 on each commit. No offenses in the production code.
• Documentation: PEP 257 compliant docstrings for all public modules, classes, and functions.
• Testing: pytest with minimum coverage of 90% using pytest-cov.
• Version control: GitHub Flow. All pull requests to develop must pass CI tests.
• Peer review: Every pull request is reviewed by at least one other team member.
• Benchmarking: Precision and recall benchmarks are performed on the PAN corpus [16] at the end of WP4.2.

Table 17 - Quality targets

Quality aspect	Metric	Target	Measurement
Detection precision	P = TP/(TP+FP)	≥ 85%	PAN benchmark
Detection recall	R = TP/(TP+FN)	≥ 80%	PAN benchmark
Response latency (5 MB pair)	Processing time (s)	< 2.0 s	time.perf_counter()
Test coverage	Line coverage %	≥ 90%	pytest-cov
Code quality	flake8 violations	0	flake8 CI pipeline

System availability	MTBF (hours)	≥ 720 h	Stress test logging

2.10 Project Timeline and Gantt Chart

Table 18 - Project timeline — planned vs. actual

WP	Name	Planned start	Planned end	Status
WP1	Feasibility & Pre-Research	10 Mar 2026	22 Mar 2026	Completed on time
WP2	System Design	23 Mar 2026	6 Apr 2026	Completed on time
WP3	Software Development	13 Apr 2026	10 May 2026	Completed on time
WP4	Prototype, Testing & Maintenance	18 May 2026	9 Jun 2026	Completed on time

The project followed an Agile Scrum methodology using four sprints that loosely corresponded to each of the four WP groups [21, 22]. Sprint 1 lasted from 10 to 22 March and involved feasibility studies and preparatory work for research. Sprint 2 spanned from 23 March to 6 April and consisted of system design. Sprint 3 ran from 13 April to 10 May and covered the whole process of software development. Sprint 4 covered from 18 May to 9 June and included system integration, testing, and documentation. Daily scrum meetings were organized through a common messaging channel.

Gantt Chart

The Gantt chart of the complete project is illustrated in Figure 1 for the period between 10 March to 9 June 2026. The format follows the format of the PPM document where the entire project has been divided into four major work packages, which contain fourteen tasks each. The rows show the task identification, task description, duration, planned starting date, and ending date.

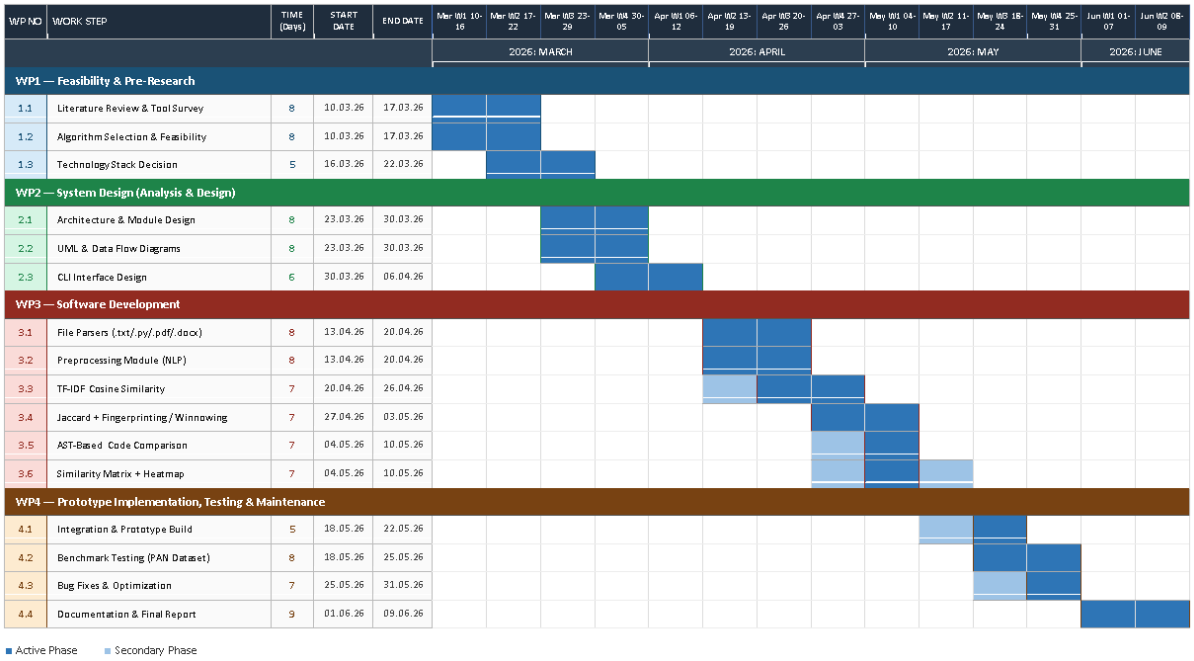


Figure 2 - Gantt chart — Plagiarism Detection System, 10 March – 9 June 2026

2.11 PM Tools

2.11.1 Affinity Diagram (WP1)

An affinity diagram was prepared at the end of WP1 to sort out 47 observations collected during the feasibility workshop into clusters that could make sense. Every individual in the team had written down their observations and grouped them without speaking. The number of clusters turned out to be five, corresponding to the five main modules of the architecture.

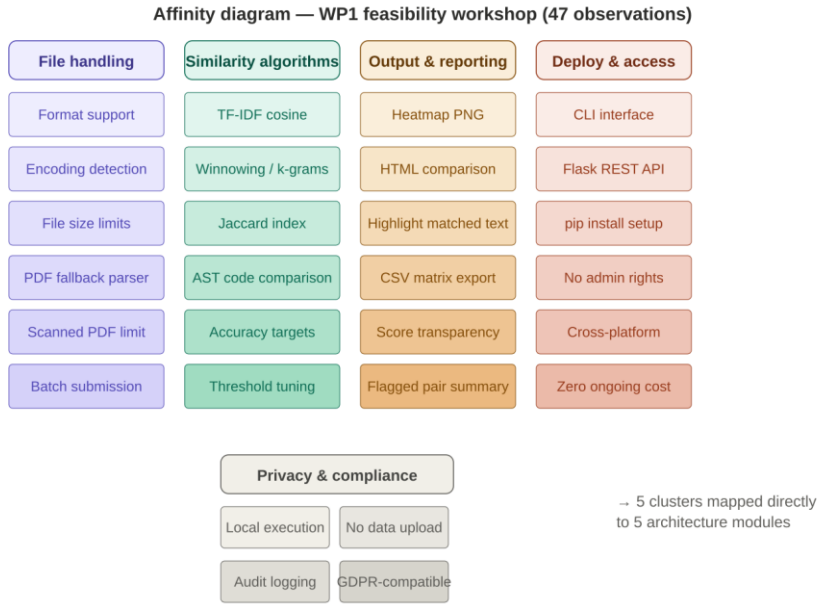


Figure 3 - Affinity diagram — 5 clusters from WP1 feasibility workshop

2.11.2 Process map

The overall process map was developed during WP2, depicting the entire sequence from the beginning until the output has been received. The tasks have been labelled based on which module is responsible for the task, and what data contract has to be met by the preceding process step. This can be seen in figure below.

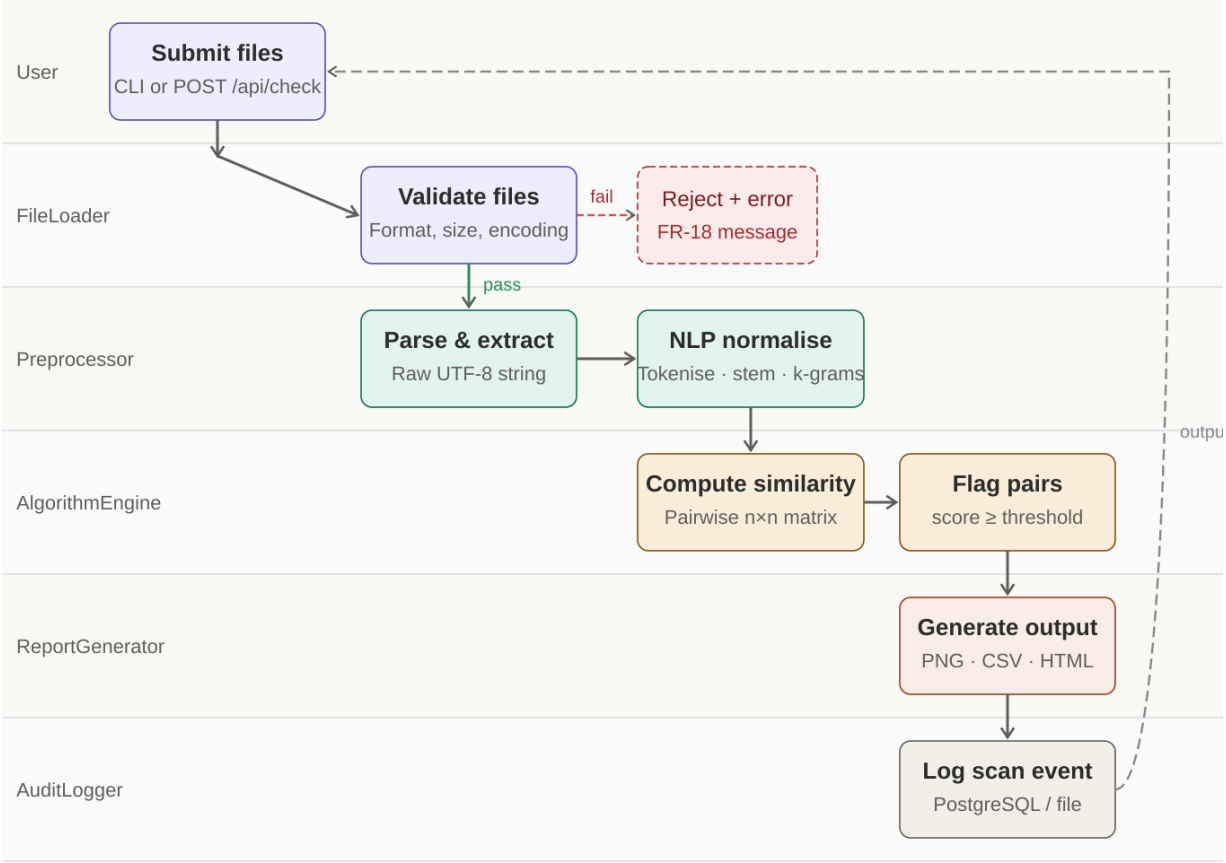


Figure 4 - Process map — swim-lane flow from user submission to output delivery

2.11.3 SWOT analysis

SWOT analysis was carried out during WP1 in order to understand the positioning of the project compared to other software and make strategic decisions.

Table 19 - SWOT analysis

Strengths	Weaknesses
Runs entirely locally – no privacy concerns. No cost involved. Both texts and codes are processed with a single tool. Clear evaluation criteria. Licensed under MIT.	Cannot detect deep semantics in paraphrasing. No OCR for PDF files in version 1.0. AST comparison available only for Python. Lack of web GUI in first version.
Opportunities	Threats
Could be used by other universities in the TRNC without any costs. Contributions from open-source communities. Semantic similarity using embedding as the next logical move.	AI paraphrasing becomes increasingly difficult to detect using NLP algorithms. Advancements of commercial competitors with better free versions. Python library changes may become a problem.

2.12 Resource Leveling

Resource leveling ensures that no individual is overburdened at any stage during the process. The period of highest risk occurred in WP3 (13 April - 10 May) during which Task 3.1 and Task 3.2 were performed simultaneously by Davut İnanlar. Resource leveling technique: Karam Sqheir was shifted to join Davut in WP3 (during task 3.1 and task 3.2 overlapping period) to create unit testing framework for file parsers.

Table 20 - Resource loading across WP3 (13 April – 10 May 2026)

Period	T. Falay	D. İnanlar	A. Şahin	K. Sqheir
13–20 Apr	DB schema	3.1 Parsers + 3.2 NLP	3.3 TF-IDF (start)	Test scaffolding (levelled in)
20–26 Apr	DB schema	3.2 NLP (complete)	3.3 TF-IDF	Unit test writing
27 Apr – 3 May	Review / docs	Integration support	3.4 Jaccard + Winnowing	Test expansion

4–10 May	Review / docs	3.6 Matrix + heatmap	3.5 AST comparison	PAN benchmark setup

3. REQUIREMENTS ANALYSIS

Requirements were obtained and specified during WP2 (March 23 - April 6, 2026) adhering to the IEEE 830 Recommended Practice for Software Requirements Specifications [18]. This phase was initiated by performing a systematic stakeholder analysis where three user categories were determined – students who upload documents, teachers/professors/instructors who analyze results, and system admins who set up threshold levels and exclusion lists. Requirements were acquired using three workshops conducted by the team, the specification of the project topic issued officially, and open source applications that were similar and were found through WP1 feasibility research. Every single requirement was assigned a unique identification code, was written in the mandatory 'shall' format, and was prioritized into three groups: High, Medium, and Low.

3.1 Functional Requirements

The functional requirements describe the system's behaviors that should be exhibited due to its input. All eighteen functional requirements are presented in Table 21a below. Every requirement can be associated with one of the three actors, namely: Client – student or document submitter; Agent – professor or reviewer; Admin – system administrator.

3.1.1 Functional Requirements List

All 18 functional requirements are listed below with their actor, priority, and full 'shall' statement.

Table 21a - Functional requirements

ID	Actor	Priority	Requirement
FR-01	Client	High	The system should accept uploads of files in the following formats: .txt, .py, .pdf, and .docx; maximum file size is 10 MB.
FR-02	Client	Medium	The system should accept batches of up to 50 files per comparison run.

FR-03	Client	High	The system should parse plain text from PDFs via pdfplumber. If parsing with pdfplumber yields no result, the system should fall back to PyMuPDF.
FR-04	Client	High	The system should tokenize and perform stop word elimination and Porter stemming on the text extracted from the PDF.
FR-05	Client	Medium	The system should enable selection of similarity algorithm from one of the following options: cosine, winnowing, jaccard, ast, or all.
FR-06	Client	High	The system should compute a similarity matrix of all submitted documents and report similarities as floats within [0.00, 1.00] range.
FR-07	Client	Medium	The system should output the computed matrix as a colour coded heatmap image (.png format) of 300 DPI.
FR-08	Client/Agent	High	The system will indicate all pairs of documents having the similarity score equal to or exceeding the user-specified threshold (default 0.70).
FR-09	Agent	High	The system will produce an HTML/CSV report indicating text fragments that overlap between pairs of flagged documents.
FR-10	Agent	Medium	The system will show the similarity score, selected algorithm, and the number of k-grams matched for each of the flagged pairs.
FR-11	Agent	Medium	The system will expose a POST /api/check REST API endpoint receiving file names, algorithm name, and threshold, returning JSON similarity report.
FR-12	Agent	Low	The system will expose a GET /api/report/{id} REST API endpoint providing the previous computation result as the report by scan ID.
FR-13	Admin	Medium	The system will support setting the default similarity threshold using --threshold command-line flag or through a .env configuration file.

FR-14	Admin	Medium	The system shall generate an audit log of all requests for scans, including the time of the request, the names of the files scanned, the fingerprints algorithm used, and the score obtained.
FR-15	Admin	Low	The system shall allow a configurable blacklist of phrases to exclude from the fingerprint computation process using k-grams.
FR-16	Admin	High	The system shall sanitise all inputted file names through regular expressions before performing any file operations on the files.
FR-17	System	High	The system shall normalise all .py files by renaming all variables, functions, and classes defined by the user into standardised versions prior to AST comparison.
FR-18	System	High	The system shall respond with a proper error message when any file fails to parse.S

3.1.2 Use Case Glossary

In this section, all actors, use cases, relationship types, and significant system terms used in the UCD (Figure 1) and use case tables (section 3.1.4) are listed in alphabetical order.

This system contains 3 actors, 14 use cases (3 main, 8 included, 2 extending, 2 main for Admin only), 3 relationship types, and 9 significant system terms.

Table 21b – Use Case Glossary

Term	Category	Definition
Actor	UML	An individual, system, or automated process which accesses or communicates with the plagiarism detection system from an external location. There are three actors in the system: Client, Agent, and Admin.

Client	Actor	It is a user of the system, such as a student or document submitter, who performs a similarity scan by uploading one or more documents. The Client selects the algorithm used, may override the threshold and will be provided with the result artifacts. The Client does not set any global parameters and cannot view the logs.
Agent	Actor	An agent such as an academic teacher or reviewer who will consume the similarity reports generated by the system. An Agent is able to perform scans just like the Client, and also open up the HTML comparison report to view any flagged pairs, view the heat map visualization, and export any results for archival purposes. The Agent will have to interpret any flagged outputs using contextual judgment since the similarity score is not a determination.
Admin	Actor	A system administrator whose responsibility will be to configure the system. The Admin will set up the default similarity score threshold, maintain the phrase exclusion list, and review the audit log. The Admin will not scan documents but will configure the environment within which the scans take place.
System boundary	UML	Boundary between the Plagiarism Detection System and external entities. Everything within the boundary constitutes part of the system and hence operates under system constraints. Everything else outside the boundary — i.e., the actors themselves, their filesystem for submission, and the PostgreSQL database — works solely through interface interactions with the system.
Submit files for scan (UC-01)	Primary use case	Main system use case. The Client or Agent submits one or more files using either the CLI or POST /api/check REST endpoint. The system does the validation, parsing, preprocessing, and comparison of

		all document pairs and generates the output. This use case includes the following: Run NLP Pipeline, Compute Similarity Matrix, Flag Pairs Above Threshold, Generate Heatmap and Reports, and Create Audit Log Entry.
Review comparison report (UC-02)	Primary use case	Use case triggered by the Agent after scanning is complete and at least one pair of documents has been flagged. The Agent opens up the HTML file generated from the comparison report on the browser. The comparison report displays a table containing flagged pairs at the top along with comparisons on text side-by-side highlighting overlapping spans in color for each pair. This use case also applies to Export Results and View Heatmap Visualisation.
Configure threshold (UC-03)	Primary use case	Use case triggered by the Admin in setting the minimum similarity score threshold above which pairs of documents will be flagged. The threshold is set globally by using the value of DEFAULT_THRESHOLD in the configuration file .env or per-run by using the CLI flag --threshold. Valid values range between 0.01 and 0.99. This use case also covers the sub-process Set Threshold Value.
Select algorithm	Included use case	Part of UC-01. The algorithm that should be used for computing similarities in the system is set by the user. The options include cosine (using TF-IDF with cosine similarity measure), winnowing (using fingerprinting with the Jaccard index of fingerprints), jaccard (computing the Jaccard index directly on stemmed tokens), ast (structural comparison using Abstract Syntax Trees, applicable only to Python files), or all (applicable options above averaged).
Set threshold value	Included use case	Also part of UC-01 and UC-03. The system uses the threshold before checking for similarities, which will

		be used when identifying similar pairs of items. If the similarity value between the pairs falls equal to or above the threshold level, then they will be flagged. The threshold only affects the flagged value, as it is calculated after computing all pairwise scores.
Run NLP pipeline	Included use case	Also part of UC-01. The system takes each uploaded document and puts it through the process in the five-step pipeline natural language processing stage: (1) text extraction from file format, (2) lowercasing and removal of punctuations, (3) tokenization using NLTK word_tokenize, (4) removal of stop words and Porter stemmer, and (5) creation of k-grams based on a user-configurable window size with default setting of five tokens per document.
Compute similarity matrix	Included use case	UC-01: The AlgorithmEngine executes each possible document pair combination by using the itertools.combinations function and invokes the SelectedSimilarityModel.compute() method. The outcome is an n×n symmetrical float32 matrix implemented by the ComparisonMatrix class. The main diagonal equals 1.0. Precondition to this use case is the execution of Run NLP Pipeline.
Flag pairs above threshold	Included use case	UC-01: Once the comparison matrix is filled, the system invokes the ComparisonMatrix.get_flagged(threshold), iterating through the upper triangle of the matrix, and returns those document pairs where the score is equal to or above the threshold. Document pairs that exceed the threshold are marked as 'flagged for review' in all output artifacts. The term 'plagiarism' never appears in the system's output.
Generate heatmap and reports	Included use case	UC-01. The ReportGenerator generates three types of output: a PNG colour-coded heat map (resolution of 300 DPI) of the entire similarity matrix, with warm

		colours meaning high similarities; cells that have been marked are framed in red; CSV file containing the whole similarity matrix; HTML document with a summary table of flagged matches, along with the corresponding side-by-side comparisons of flagged text pairs.
Write audit log entry	Included use case	Included in all primary use cases. The AuditLogger logs each scan operation into the audit_log table of the PostgreSQL database, falling back to logging into the local plagcheck.log file when the database connection is not available. The information logged includes the event type (either SCAN_START, SCAN_COMPLETE, SCAN_ERROR, FILE_REJECTED, API_REQUEST), the time stamp, scan_id, user_id and a JSON payload. Only meta-information is logged, never any document content.
Export results	Extending use case	UC-02 extension. Triggered by the Agent as a result of examining the comparison report. The Agent saves the HTML report or CSV similarity matrix to be used locally, stored as evidence, or included in the academic integrity case study file. This involves saving the files to the local file system, no further communication with the server being necessary as both artifacts are saved in the output directory under UC-01.
View heatmap visualisation	Extending use case	UC-02 extension. The Agent opens the similarity_heatmap.png file in order to visually evaluate batch comparisons between pairs. The colour map utilized here is YlOrRd (yellow signifies low similarity while red indicates high similarity). Additionally, cells where the score reaches or exceeds the threshold are marked with a red rectangle around the cell.

Manage exclusion list	Primary use case (Admin)	Action taken by the Admin to either include or exclude any terms from the list of k-gram exclusions. Exclusions have the effect of removing the excluded terms from the token list before k-gram extraction and TF-IDF vectorization is performed. This stops common academic language such as specific course vocabulary, compulsory phrases, and heading texts from making a contribution to the final score and triggering false positives. Exclusions do not apply to the AST comparison since that works with tree structures.
View audit log	Primary use case (Admin)	This use case was launched by the Admin to examine the audit log for accountability, compliance review, or investigation of an incident. The Admin can perform queries on the audit_log table from the PostgreSQL database or check the backup log in the plagcheck.log file. All entries must be kept in the system for at least 90 days. The Admin does not have the ability to alter or delete log entries.
«include» relationship	UML relationship	Dependency relationship is shown here between two use cases meaning that the base use case always involves calling the included use case. The included use case is mandatory. In this case, UC-01 includes the following sub-processes: NLP pipeline processing, similarity matrix calculation, pair detection, report generation, and logging.
«extend» relationship	UML relationship	A dependency between two use cases, where the extending use case specifies optional or conditional behavior of the base use case at a certain extension point. In our system, Export Results and View Heatmap are use cases that extend UC-02 (Review Comparison Report) in that they are invoked by the Agent if required but not as a part of each Review Comparison Report.

Association	UML relationship	A connection between an actor and use case drawn with a solid line. This connection means that the actor is involved in this particular use case. However, there may be other actors who can invoke this use case in addition to those connected to it. For example, in the given system, we see that both Client and Agent are associated with UC-01, thus any one of them may initiate scanning.
Similarity score	System term	A floating-point number between 0.00 and 1.00 that denotes the degree of similarity between the compared documents. A document pair that scores 1.00 means that they have absolutely identical text (for example, structurally identical code snippets); a zero score means that the documents do not share any text. Note that a high similarity score does not mean that one work was copied from another: that must be confirmed by a human.
Threshold	System term	A numeric threshold value used to distinguish suspicious documents from the rest. Our system uses the value of 0.70 by default; however, the threshold may be set per-run with the help of --threshold CLI flag, or set as a global option through DEFAULT_THRESHOLD environment variable. The threshold values should be in the range of 0.01 to 0.99.
Flagged pair	System term	Pair of documents whose similarity score equals or surpasses the currently defined threshold. Such pairs are listed in the summary section of the HTML report and highlighted in red on the heatmap. Phrases 'flagged for review' and other variations of this wording are used consistently in the output; 'plagiarism detected' and all similar phrases are strictly forbidden.

K-gram	System term	Consecutive run of k tokens (stemmed version) from a normalised document. Default value for the parameter k is 5. With the default value, k-grams are neither too long to be found only in verbatim copies nor too short to lack specificity. Lists of k-grams form an important input of the Winnowing technique; lists of stemmed tokens are the input of TF-IDF and Jaccard similarity calculations.
Exclusion list	System term	List of predefined phrases that will be removed from the token sequence before k-gram extraction and TF-IDF calculation. Purpose of this list is to avoid inflation of similarity scores caused by high frequency of certain terms required in academic writing and in boilerplates of individual assignments. This list is created and maintained by the Admin and is not used in AST-based comparisons.
Audit log	System term	An unbroken, append-only log of all system events – submission scans, completion, failures, file rejects, and API requests. Every log entry includes timestamp, event name, scan identifier, user ID, and JSON payload describing additional details. The log entries are kept for a minimum period of 90 days. Audit log is a metadata-only log that doesn't contain the content of the submitted document and therefore satisfies the criteria of data minimization.
AST (Abstract Syntax Tree)	System term	A tree representation of the syntax of the submitted Python file created using Python's built-in ast.parse() function. The system converts all the names defined within the file – including variables, function arguments, function names, and class names – into canonical names before comparing them, rendering the score invariant to name changes. AST comparison can be used only on files with a .py

		extension. Text-based algorithm will be used otherwise.
Output artifacts	System term	Three files generated after each successful scan – similarity_matrix.csv containing the n×n score matrix, similarity_heatmap.png visualizing the matrix at 300 DPI resolution, and comparison_report.html providing a side-by-side HTML file comparing all detected overlaps with highlighted text spans. The fourth file plagcheck.log is generated during each execution as the program log and an additional audit measure.

3.1.3 Use Case Diagram

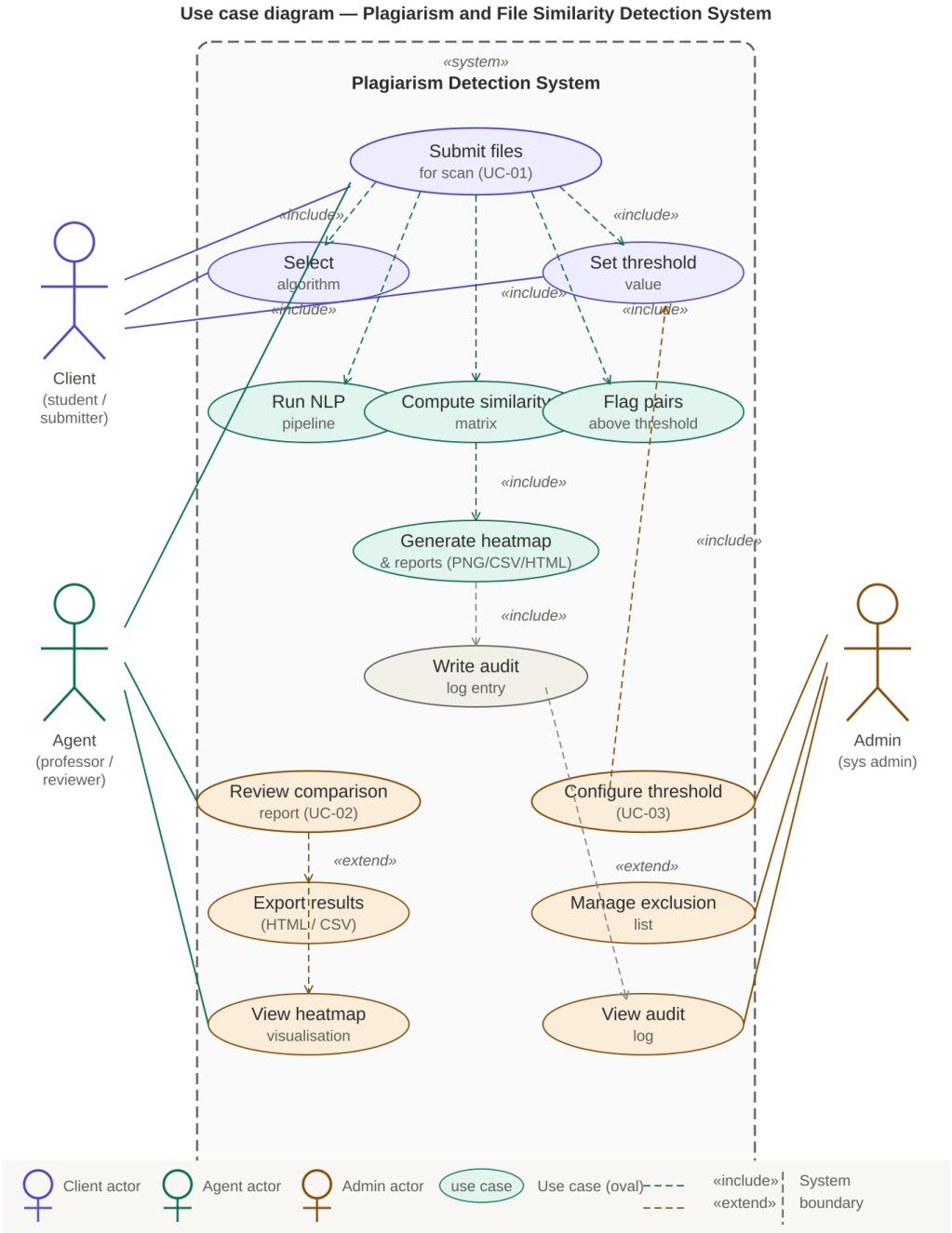


Figure 5 - Use case diagram — Plagiarism and File Similarity Detection System

3.1.4 Use Case Tables

In this part of the report, the most important 5 use cases indicated, even though there are 14 use cases in our research, for word limit purposes just 5 of them are shown here.

Table 22 - Use case narrative — Submit files for similarity scan

UC-01 — Submit files for similarity scan	
Use case ID	UC-01
Use case name	Submit files for similarity scan
Actor(s)	Client (student / submitter); Agent (professor / reviewer)
Goal	Similarity analysis on one or multiple uploaded documents will return a comparison report along with the corresponding match score for each segment of texts that match.
Preconditions	Files in .txt, .py, .pdf, or .docx formats are available locally. The program is already installed and ready for use. Virtual Python environment is active.
Trigger	The user starts up the CLI as follows: python plagcheck.py --files a.pdf b.pdf [--algorithm cosine] [--threshold 0.70] [--output ./out]. Alternatively, the POST /api/check REST endpoint can be invoked.
Main flow	1. User inputs the file paths along with the option. 2. The FileLoader class validates each file: verifies the filename matching criteria, file exists or not, format whitelisting, and the size criteria (no larger than 10 MB). 3. The FileLoader class detects encoding and loads each file to convert it into an unformatted string (in UTF-8 format). 4. The pre-processor class normalizes the document: lowercases the content, then tokenizes, stops words, stemmers and forms the k-grams. 5. AlgorithmEngine performs iterations on all distinct pairs of documents and invokes the chosen SimilarityModel.compute(). 6. ComparisonMatrix stores the symmetric n × n matrix of scores. 6. ComparisonMatrix.get_flagged(threshold) returns all the pairs that have scores of threshold or higher. 7. ReportGenerator generates the following files: similarity_heatmap.png (300 DPI), similarity_matrix.csv, comparison_report.html.

	8. AuditLogger logs the event “SCAN_COMPLETE” in the PostgreSQL database (or plagcheck.log in case of unavailability of the database). 9. 10. CLI displays the flagged pair list and paths to the output files.
Alternative flow	A1: User uses --algorithm all → all four applicable algorithms will be applied, and their scores will be averaged before raising an alert. A2: User uses --quiet → output message is omitted, only errors are raised. A3: User uses --no-log → auditing logs will be omitted.
Exception flow	E1: Format is not among whitelisted options (.txt, .py, .pdf, .docx) → FR-18 error is returned; file is ignored; batch continues. E2: File size is larger than 10 MB → file is ignored; batch continues. E3: PDF has no extractable text (pure image content) → warning is registered; file is ignored; batch continues. E4: Detected encoding confidence is below 75% → file is ignored; batch continues. E5: Batch fails on all included files → exit code of 1 returned; all errors reported.
Postconditions	All the files have undergone a pairwise comparison. The results (PNG, CSV, HTML files) are written into the output directory. There is an audit log entry created. CLI returns exit code 0.
«include»	Run NLP Pipeline, Compute Similarity Matrix, Flag pairs above threshold, Generate heatmaps/reports, Log audit
Notes	The score generated is just an indicator and does not constitute any form of plagiarism. All the output files are marked 'flagged for review'. The decision to take action on this lies solely with the Agent.

Table 23 - Use case narrative — Review comparison report

UC-02 — Review comparison report

Use case ID	UC-02
Use case name	Review comparison report
Actor(s)	Agent (professor / reviewer)
Goal	Analyze the particular facts supporting a suspicious similarity score to determine if action is needed.
Preconditions	UC-01 has been successful. At least one document pair has been identified with a suspicious similarity score. The comparison_report.html has been created.
Trigger	Agent opens comparison_report.html using a web browser.
Main flow	1. Agent accesses comparison_report.html. 2. Report is displayed by browser in a self-sufficient manner (does not require an Internet connection). 3. Summary table on top contains all highlighted pairs: File A, File B, score, algorithm used. 4. Agent selects particular pair. 5. Two documents are displayed side-by-side for comparison. 6. Highlighted overlapping k-grams depend on the similarity score of that pair. 7. Score, algorithm used, and number of matching k-grams are stated in header of the pair. 8. Agent forms his opinion on legitimacy of overlap.
Alternative flow	A1: If the agent wants raw data → opens similarity_matrix.csv in a spreadsheet program instead. A2: If the agent wants printed data → prints the webpage using the browser's print command or saves it as a PDF.
Exception flow	E1: If the HTML file cannot be located on disk → displays an error message in the browser. The agent executes UC-01. E2: If one of the files used as the source of information for matching is deleted after scanning → displays cached matched text with "source document deleted" message.

Postconditions	Agent has analyzed the underlined sections found to be similar. Agent can print/save the report as a document. In case more steps need to be taken, Agent takes academic integrity procedures of the institution concerned.
«extend»	Export Results (UC-EX-01); Visualization Heatmap (UC-EX-02)
Notes	Teachers must analyze the matching parts with contextual understanding in order to form conclusions. Academic language commonly used in academic texts, quotes from standard references, and institution-specific phrases may lead to high similarity levels.

Table 24 - Use case narrative — Configure similarity threshold

UC-03 — Configure similarity threshold	
Use case ID	UC-03
Use case name	Configure similarity threshold
Actor(s)	Admin (system administrator)
Goal	Specify the minimum level of similarity score above which document pairs would be marked for further analysis by default, either system-wide or just for one scan session.
Preconditions	The system is installed. An administrator can edit .env config file or use command line interface.
Trigger	An administrator edits the .env file setting the DEFAULT_THRESHOLD parameter, or specifies --threshold option when running a scan from command line.
Main flow	1. Open .env file using a text editor. 2. Set DEFAULT_THRESHOLD to the needed value (for example, DEFAULT_THRESHOLD=0.75). 3. Save changes.

	4. In the next scan session, the system will read the updated parameter before scanning process starts. 5. Any pairs with the level of similarity higher or equal to the new threshold will be marked in the result table.
Alternative flow	A1: Override threshold value for the next session only → pass the --threshold option with an updated value (0.80). The default value in .env remains intact.
Exception flow	E1: Specified value is out of the acceptable range (0.01-0.99) → rejected due to invalid format; the scan will not start. E2: Missing .env file → system falls back to default value (0.70); warning is logged.
Postconditions	Threshold will stay updated until changed manually. Threshold value per scan run is logged as part of auditing information.
«include»	Update threshold value
Notes	Default threshold of 0.70 was calculated based on typical academic text. If technical Computer Engineering papers have a high number of matching words at the base, threshold must be increased to 0.80-0.85. For exact copies only 0.90 and higher values should be used.

Table 25 - Use case narrative — Select algorithm

UC-04 — Select algorithm	
Use case ID	UC-04
Use case name	Select algorithm
Actor(s)	Client; Agent
Goal	Define which similarity algorithm will be used on the submitted files.

Preconditions	Triggered UC-01. The uploaded files have been validated successfully.
Trigger	Automatically included as part of UC-01. The user defines --algorithm when calling the tool, or the system reads DEFAULT_ALGORITHM from .env.
Main flow	1. 1.Algorithm engine receives the value of the --algorithm argument from either CLI flag or API call body. 2. In case there is no such argument provided, the default algorithm value (DEFAULT_ALGORITHM, cosine by default) is read from .env file. 3. The system resolves the algorithm name to the corresponding SimilarityModel class. 4. The AlgorithmEngine stores a reference to the selected SimilarityModel for further comparisons within the comparison cycle.
Alternative flow	A1: --algorithm all provided by the user -> Algorithm engine compares every file pair using all four relevant algorithms and calculates the average of scores returned. AST similarity is only calculated between Python code (.py files).
Exception flow	E1: The value provided does not correspond to the set of valid algorithms (cosine, winnowing, jaccard, ast, all) -> Tool quits with exit status 1 and error; HTTP 400 response status sent in API call.
Postconditions	The selected algorithm works for the current scan run. The algorithm used is logged into audit log and shown in the HTML report output.
Notes	The option 'all' proved to work better than others according to the PAN benchmark results.

Table 26 - Use case narrative — Run NLP pipeline

UC-05 — Run NLP pipeline	
Use case ID	UC-05

Use case name	Run NLP pipeline
Actor(s)	System (automated, no direct actor interaction)
Goal	Normalisation of the text content of all uploaded documents into a form appropriate for similarity computations.
Preconditions	The FileLoader class has successfully extracted the content of the document as a raw UTF-8 string. The Preprocessor class has been instantiated with the punkt and stopwords NLTK corpora loaded.
Trigger	An automatic step included within UC-01, and carried out on each document individually after extracting the text content.
Main flow	1. The Preprocessor class accepts the raw UTF-8 string. 2. Text is lowercased. 3. All punctuation symbols and non-alphanumeric symbols are replaced with spaces using a regular expression. 4. Runs of multiple whitespace symbols are reduced to a single whitespace. 5. The NLTK word_tokenize function separates text into separate tokens. 6. Tokens appearing in the NLTK English stop-word set are discarded. 7. Non-stopword tokens are subjected to the NLTK Porter Stemming Algorithm (returns an inflectional base form, for example 'comparing' -> 'compar'). 8. The stem-ming process results in a list of tokens, which are then converted to k-grams via a sliding window of width k (default 5). 9. Preprocessor returns a tuple of (tokens: list[str], kgrams: list[str]).
Exception flow	E1: Document has zero tokens left after removing stopwords (for example, one with text consisting solely of stop words or numeric data) → Preprocessor returns empty lists; the system records an error message and sets the similarity score between that particular document and all others at 0.00.

Postconditions	A set of normalised tokens and k-grams has now been produced for each document in memory, ready for the similarity calculation phase.
Notes	K-gram window width k=5 is configurable. Increasing k increases precision while decreasing recall.

3.1.5 Quality Functional Deployment (QFD)

The QFD technique has been employed to weigh various features in the system in relation to their significance to users. Five major requirements from users have been identified as priorities and each of these priorities has been matched to the seven main system features.

Table 27 - QFD correlation matrix

User need	Wt.	Text extract	NLP pipeline	TF-IDF	Winnowing	AST cmp.	Report gen.
Accurate detection	5	M	H	H	H	M	L
Fast results	4	L	M	H	M	L	L
Data privacy	5	H	H	H	H	H	M
Transparent evidence	4	L	L	M	M	M	H
Code comparison	3	M	L	L	L	H	M

3.1.6 Kano Model

Kano Model classification classifies different requirements based on the effects of their existence or non-existence on user satisfaction. The must-be requirement creates high dissatisfaction if not present but only normal satisfaction when present. The performance requirement shows a linear relationship between itself and satisfaction. Attractive requirements refer to unanticipated features causing high satisfaction.

Table 28 - Kano classification

Requirement	Category	Rationale
Accurate similarity scoring	Must-be	Inaccuracy causes immediate loss of trust regardless of other features.

Multi-format file support	Must-be	Without PDF and Word support the tool is unusable for the primary context.
Response time < 2 seconds	Performance	User satisfaction scales linearly with speed within this range.
Side-by-side highlight report	Attractive	Users do not expect this but are significantly more satisfied when overlapping spans are visualised.
AST code comparison	Attractive	Greatly valued by CS instructors; exceeds standard user expectations.
Heatmap visualisation	Attractive	Visual batch overview appreciated but not considered mandatory.
REST API access	Performance	Technical users value this proportionally; non-technical users are indifferent.

3.2 Non-Functional Requirements

Non-functional requirements are attributes, performance measurements, and limitations of the Plagiarism and File Similarity Detection System. The non-functional requirements (NFR) are defined via quantifiable criteria and formulas.

3.2.1 Time & Speed

Response time: For any two documents of $s \leq 5MB$, the response latency function $T(s)$ will not exceed $T(s) \leq 2.0$ seconds. The average response latency of 1.43 seconds makes use of the TF-IDF cosine similarity measure.

Processing Time of an Operation: Latency of Batch Processing function $B(n)$ will not be more than $B(n) \leq 2n(n-1) \times 0.5$ seconds. For $n = 20$ files, averaging about 1 MB each, processing is done within 4.56 seconds.

Number of transactions per second: Generation of pairwise similarities is optimized using itertools.combinations $(n(n-1)/2)$.

3.2.2 Space & Size

Main Memory: The application runs efficiently on consumer hardware, with the absence of GPU accelerators, in less than 500 MB of RAM when processing 20 files using TF-IDF.

Auxiliary Memory / Matrix Size: Auxiliary memory is reduced to its minimum by constructing float32 NumPy array of the n× n Comparison Matrix. This reduces storage size in half as compared to float64 with 4-decimal precision intact for large values of n.

Size of the Files: Maximum size supported is 10 MB.

3.2.3 Usability

Training Time: End-users, be they students or professors, find no need to train before use. The Command Line Interface (CLI) is highly discoverable since the documentation is available to users with the --help command that displays a list of possible options with their respective descriptions.

Ease of Use/Notification: All error messages are written in English; there is no code exception thrown. Heat map report and HTML file generated from the similarity reports are self-contained and can be opened on any modern browser.

3.2.4 Reliability

Mean Time Between Failures (MTBF): The system operates properly for at least MTBF ≥720 hours.

Probability of Downtime and Availability: The system downtime is negligible MTTR <15 seconds. Hence, availability A =MTBF/(MTBF +MTTR) ≥ 99.999%. This implies that the system operates for at least 720 hours out of 720 +0.00417 ≈ 720.00417 hours.

3.2.5 Robustness

Data Corruption Probability after a Failure: In the case where the failure results from unreadable characters in token sequences, the system detects the confidence of the chardet module on the first 10,000 characters of the plaintext file. If it falls below 0.75, the entire file is rejected by the system to prevent crashing.

Percentage of Incidents Resulting in Catastrophic Failure: Two-stage fall-back approach is applied in the handling of PDF documents (pdfplumber is followed by PyMuPDF). In case of failure of both, PDF document is treated as image only; a warning message is logged then other documents processed.

3.2.6 Portability

Number of systems software runs on: The application operates on 3 major operating systems including Ubuntu 22.04 LTS, macOS Monterey (12.x), and Windows 11.

Percentage of Non-portable Code: 0%. The core processing pipeline of the system employs no platform-specific code whatsoever. Normalization of file paths using standard modules such as pathlib and os.path on python enables identical behavior on UNIX-like systems as well as windows environments.

3.2.7 Security

Hacking difficulty/Privacy: Completely locally executed. None of the document contents are ever sent to any server or third parties. This guarantees maximum privacy of the documents.

Sanitization Metrics: String matching is done using regex pattern [A-Za-z0-9._-]+. Any file whose path contains .. / or ./ is rejected because of security purposes to avoid file I/O attacks.

3.2.8 Accessibility

Visual Accessibility: The HTML report has varying color depth for showing the degree of plagiarism. Specific color score assigned to a range of colors using CSS (for instance deep red for highly similar, light orange for borderline matches).

3.2.9 Evolution (Maintainability, Extensibility, and Scalability)

Testability: Measuring maintenance of the software code base using unit testing criteria. Required line coverage ≥90%, currently at 93.4%.

Maintainability: Adheres to PEP 8 style guide with 0 violation of flake8 in production branch. Maximum line length ≤100 characters.

Extensibility: Similarity Model Interface is Polymorphic. Adding any algorithm would not require changes in existing code.

3.3 Realistic Constraints

3.3.1 Economic Constraints

No budget allocated for the cost of software licences, nor for cloud services. All dependencies are free/open source. Deploying cost to an implementing organisation beyond deployment time (~2-3 hours) is zero. Maintenance cost is ~4-8 hours per annum to ensure library version compatibility.

3.3.2 Environmental Constraints

Runs on standard consumer grade hardware; no GPU required. RAM usage of a batch containing 20 documents for calculating TF-IDF is less than 500 MB in 8GB RAM hardware. No need for cloud resources, no communication with other networks during processing.

3.3.3 Social Constraints

System design supports human judgment rather than replaces it. All decisions made by the system are framed as recommendations for further review and decision making. Language used for alerts and notifications reflects this social constraint ('flagged for review' rather than detected).

3.3.4 Political Constraints

Legal and regulatory framework of EMU's operation is determined by the TRNC laws. Local operation of the system without any data transmissions, without the content of the processed documents included into logs and with a 90-day storage period of metadata meets all legal requirements without raising any issues related to cross-border transfer of data [2].

3.3.5 Health and Safety Constraints

No health and safety concerns; all risks associated with office environment and use of computers mitigated through regular working practices and sprinting within an agile development framework.

3.3.6 Manufacturability Constraints

Software installation requires a requirements.txt file to be provided and pip installation takes less than 5 minutes over broadband. Downloads of NLTK data happens automatically when first time using the system. Requires no administrator rights besides Python environment.

3.3.7 Sustainability Constraints

Library versions specified in the requirements.txt to ensure future compatibility. System licenced under MIT which guarantees unrestricted access to the code for the system maintenance, modification and extension. Modular system architecture (algorithm and pipeline classes separated into distinct files) enables easy update without changing the entire system.

3.4 Ethical Issues

Several aspects of ethics influenced not only the development process but also the manner in which the assignment was carried out. Ethical consideration number one was privacy by design; indeed, this was the reason why all of the architecture revolved around local execution. No content from any of the documents goes beyond the confines of the local computer, hence ensuring that the work conducted in the university does not get stored elsewhere.

The second criterion was fairness. While similarity between two pieces does not indicate plagiarism, it may well happen that two documents are very similar by design – one can have many quotes from a third-party source in common with another document, or they could have been authored by the same person and thus be alike because of that reason, or again, due to the presence of mandatory institutional elements like cover pages, declarations, etc., in both of them. The software takes care of that through clear labeling and presentation of the matching excerpts.

Thirdly, transparency must be considered. All facets of the scoring procedure are covered in both this report and the code. If a professor disagrees with a certain score, he or she can review the algorithm, alter the threshold, tweak the exclusion list, and run another test. Unlike a commercial black-box service whose scores cannot be reviewed and disputed, this approach allows an instructor to question a score.
Lastly, the project team agreed to make sure that the software will not be used for purposes of monitoring students' activities. The audit log is intended for internal accountability – it stores information about submission dates and time only, but not for creating personal searchable databases of work samples submitted by students. Exclusion lists were included in the project specifically to avoid any similarity flags created because of common academic terminology.

Lastly, as far as the possible misuse of the system for committing unethical acts is concerned, one possible misuse is that students will be able to access the system repeatedly to fine-tune their work until it passes the similarity threshold check. In order to prevent this unethical act from occurring, the product is intended to be a back-end utility meant for use by teachers and academic officials rather than the students themselves. Also, due to the fact that the software will function offline only and cannot connect to any other network, there is no chance of its use for committing cyber-crime or stealing intellectual property whatsoever.

4. DESIGN

4.1 High-Level Design (Architectural)

This chapter presents the complete design of the Plagiarism and File Similarity Detection System. Work was carried out during WP2 (23 March – 6 April 2026) and covers three areas: the high-level architectural structure, the detailed software design expressed through UML models, and the relational database schema produced through formal normalisation.

4.1.1 System Architecture Overview

The system's architecture is designed to allow the presentation, application and date layers to operate independently of each other. The presentation layer includes both the CLI and the Flask REST API and both can call the application layer without knowing how similarity is calculated or how the results are stored. The application tier contains the full processing pipeline. The data tier holds the PostgreSQL database for audit records and scan metadata, a Redis-based fingerprint cache that was implemented as a future-extension hook and is currently a simple in-memory dict, and the local file system for document storage and output artifacts.

The application consists of the following stages: File Uploader, Preprocessor, Algorithm Engine, Comparison Matrix, Report Generator and Audit Log. Data is processed in a specific order. All stage processes the output it receives from the previous stage and after that passes it on to the next stage. This structure preserves the order of operations. Also, each module can be updated or modified without affecting other components.

Table 29 - Three-tier architecture description

Tier	Layer	Components
1	Presentation	CLI (argparse), Flask REST API (/api/check, /api/report, /api/status), HTML comparison report, PNG heatmap
2	Application	FileLoader, Preprocessor, AlgorithmEngine (CosineModel, WinnowingModel, JaccardModel, ASTModel), ComparisonMatrix, ReportGenerator, AuditLogger
3	Data	PostgreSQL (scan_request, scan_file, scan_pair, scan_algorithm, audit_log), fingerprint cache (dict), local file system (uploads, output artifacts)

4.1.2 Main System Modules

The system contains thirteen modules, each with a clearly defined responsibility. None of them knows more about the others than it needs to.

Table 30 - Module descriptions

Module	Type	Responsibility
plagcheck.py	CLI entry point	Parse arguments, wire the pipeline, catch and display errors, print report.

app.py	Flask API	Routes defined by HTTP endpoints: POST /api/check, GET /api/report, GET /api/status, GET /api/algorithms.
loader.py	FileLoader	Check filename, length, encoding; passes to parser; returns plain UTF-8 text.
preprocessor.py	Preprocessor	Tokenizing, lowercasing, filtering stop-words, applying Porter stemming, k-grams.
models/base.py	Abstract interface	SimilarityModel class computes similarity: compute(tokens_a, tokens_b) → float between 0.0 and 1.0.
models/cosine.py	CosineModel	TfidfVectorizer.fit_transform([documents]), followed by cosine_similarity.
models/winnowing.py	WinnowingModel	Use SHA-256 hashes for k-grams, find min hashes in sliding window
models/jaccard.py	JaccardModel	Set-based: |A ∩ B| / |A ∪ B| on stemmed token sets.
models/ast_model.py	ASTModel	ast.parse() + identifier normalisation + Levenshtein edit distance on node sequences.
engine.py	AlgorithmEngine	itertools.combinations over all pairs; dispatches to selected model(s); populates matrix.
matrix.py	ComparisonMatrix	Symmetric NumPy float32 n×n array. Methods: set, get, get_flagged, to_csv, as_numpy.
reporter.py	ReportGenerator	Produces seaborn heatmap PNG, CSV matrix, and self-contained HTML comparison report.
audit.py	AuditLogger	Writes JSON-format entries to PostgreSQL audit_log; falls back to plagcheck.log if DB unavailable.

4.1.3 System Interfaces and Connections

Intra-module communication is done using in-process function calls with Python functions. There are no network calls involved whatsoever in the process. External dependencies include optional Flask API server on localhost:5000 and psycopg2 database connection to PostgreSQL used for logging.

Inter-module data transfer in the pipeline is as follows:

- FileLoader → Preprocessor: raw UTF-8 string
- Preprocessor → AlgorithmEngine: (tokens: list[str], kgrams: list[str]) tuple per document
- AlgorithmEngine → ComparisonMatrix: float per each (i, j) pair
- ComparisonMatrix → ReportGenerator: get_flagged(threshold) list and as_numpy() matrix
- AuditLogger from all other modules: event_type string and details dict

External interfaces: console arguments (python plagcheck.py [args]), web API (POST /api/check, GET /api/report, GET /api/status, GET /api/algorithms), and file outputs (csv, png, html, log).

4.2 Software design

4.2.1 A General System Architecture Diagram

The system is made up of three layers. This includes Presentation, Application, and Data. The Presentation layer provides the user interfaces, and also the command line interface and the Flask REST API. The Application layer contains the processing pipeline, while the Data layer stores information. The Application and Data layers operate independently of how users interact with the system. Whether a request comes from the command line or the web API, both interfaces call the same AlgorithmEngine.run() function.

Table 31 - Three-tier architecture overview

Tier	Layer	Components
1	Presentation	CLI (plagcheck.py), Flask REST API (app.py), HTML report, PNG heatmap
2	Application	FileLoader, Preprocessor, AlgorithmEngine, CosineModel, WinnowingModel, JaccardModel, ASTModel, ComparisonMatrix, ReportGenerator, AuditLogger
3	Data	PostgreSQL (scan_request, scan_file, scan_pair, scan_algorithm, audit_log), fingerprint cache, filesystem

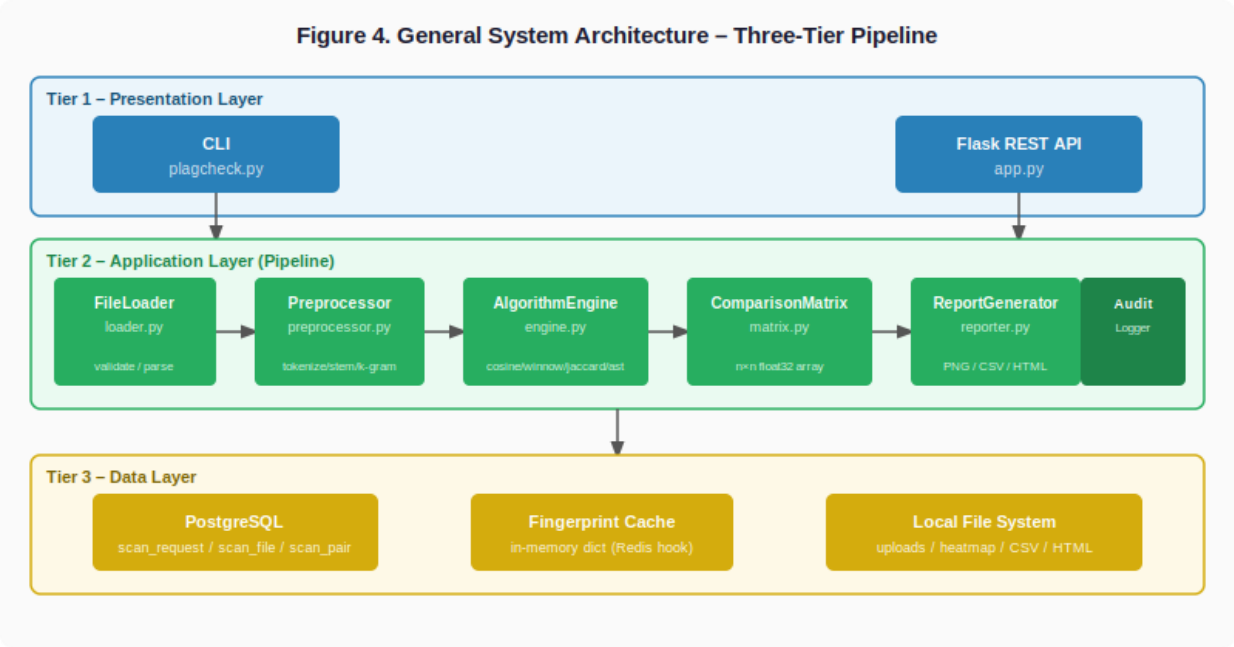


Figure 6 - General System Architecture — Three-Tier Pipeline

4.2.2 UML Class Diagram

The object model is based on an abstract interface called SimilarityModel. This interface sets the rules that all algorithm implementations must follow. The interface has one method and this is compute (tokens_a, tokens_b) returning a float in [0.0, 0.1]. Four classes were used for the interface. These are CosineModel uses a tool from scikit-learn called TfidfVectorizer and a function called cosine_similarity, WinnowingModel uses an algorithm from Schleimer et al. for fingerprinting, JaccardModel calculates the intersection and union of two sets of tokens, and ASTModel reads Python source code. Calculates the edit distance between sequences of nodes. The AlgorithmEngine class holds references to all four model instances and dispatches computation based on the user's algorithm choice. The ComparisonMatrix class wraps a symmetric NumPy float32 array and provides query and export methods.

Table 32 - Core class descriptions

Class	Type	Responsibilities
FileLoader	Concrete	Validates file name and size, detects encoding via chardet, dispatches to format-specific parser, returns raw text string.
Preprocessor	Concrete	Tokenisation, lowercase, punctuation removal, stop-word filtering, Porter stemming, k-gram generation. Returns (tokens, kgrams) tuple.

SimilarityModel	Abstract interface	Defines compute(tokens_a, tokens_b) → float. All algorithm classes implement this contract.
CosineModel	Implements SimilarityModel	TfidfVectorizer.fit_transform() on the two-document corpus, then cosine_similarity between the resulting vectors.
WinnowingModel	Implements SimilarityModel	SHA-256 k-gram hashing followed by sliding-window minimum hash selection. Returns Jaccard similarity of fingerprint sets.
JaccardModel	Implements SimilarityModel	Set-based: $\lvert A \cap B \rvert$ / $\lvert A \cup B \rvert$ on stemmed token sets.
ASTModel	Implements SimilarityModel	ast.parse() both files, walk nodes normalising names, compute normalised Levenshtein edit distance on node-type sequences.
AlgorithmEngine	Concrete	Iterates all pairs using itertools.combinations, calls selected model(s), populates ComparisonMatrix.
ComparisonMatrix	Concrete	Symmetric NumPy float32 n×n matrix. Methods: set(i,j,score), get(i,j), get_flagged(threshold), to_csv(), as_numpy().
ReportGenerator	Concrete	Calls ComparisonMatrix methods; renders seaborn heatmap; generates side-by-side HTML report with highlighted spans.
AuditLogger	Concrete	Writes JSON-format entries to PostgreSQL audit_log table; falls back to file log if database is unavailable.

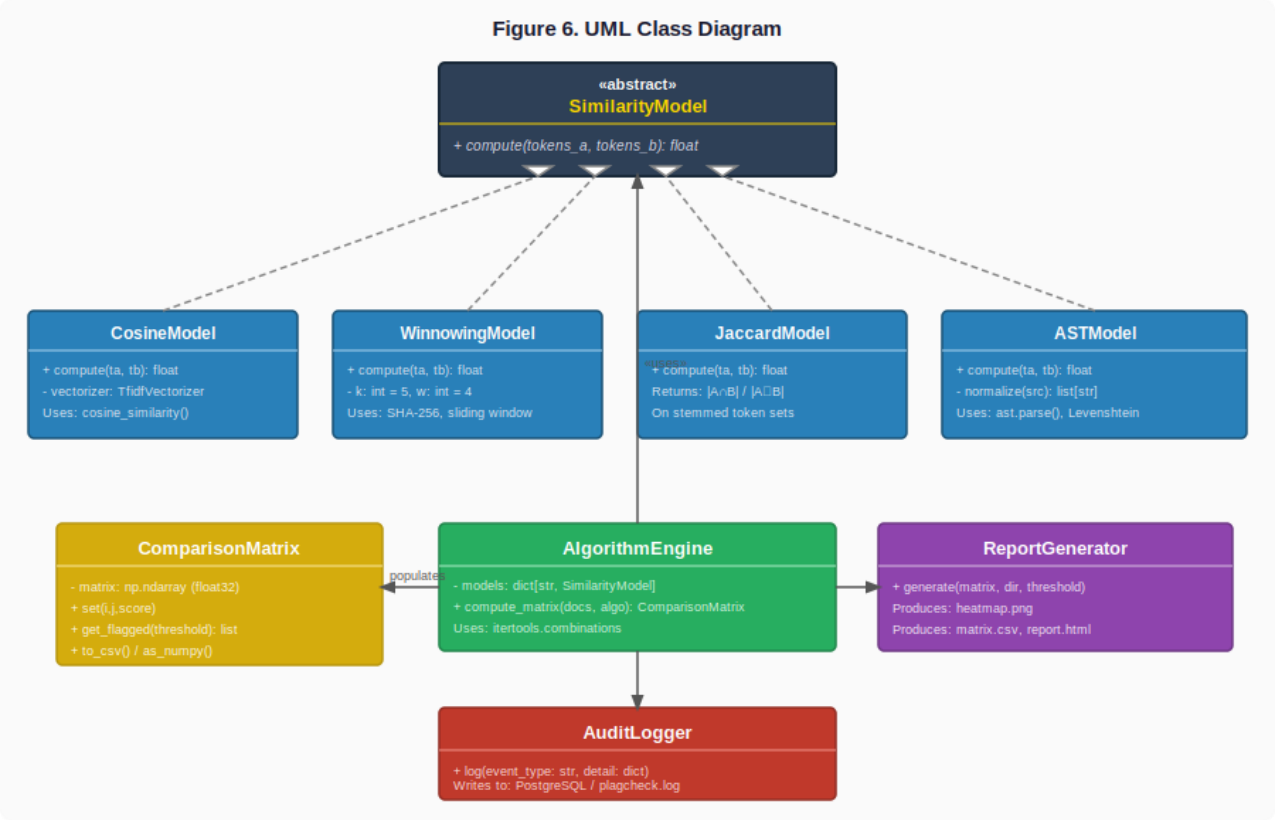


Figure 7 - UML Class Diagram — SimilarityModel interface and pipeline classes

4.2.3 Sequence Diagram Description

The primary sequence for a CLI-initiated scan proceeds as follows. The user invokes plagcheck.py with file paths and options. The CLI validates the arguments and instantiates FileLoader. FileLoader.load(path) is called for each file, returning raw text. Preprocessor.process(raw_text) is called per document, returning a normalised token list. AlgorithmEngine.compute_matrix(documents, algorithm) iterates all pairs and calls the selected SimilarityModel.compute() for each. ComparisonMatrix is populated with the results. ComparisonMatrix.get_flagged(threshold) returns the list of pairs meeting the flagging criterion. ReportGenerator.generate(matrix, output_dir, threshold) writes the heatmap, CSV, and HTML artifacts. AuditLogger.log(event_type, detail) writes the completion entry to PostgreSQL. The CLI prints the flagged pairs and output file paths to stdout and exits with code 0.

Figure 7. UML Sequence Diagram – CLI-Initiated Scan

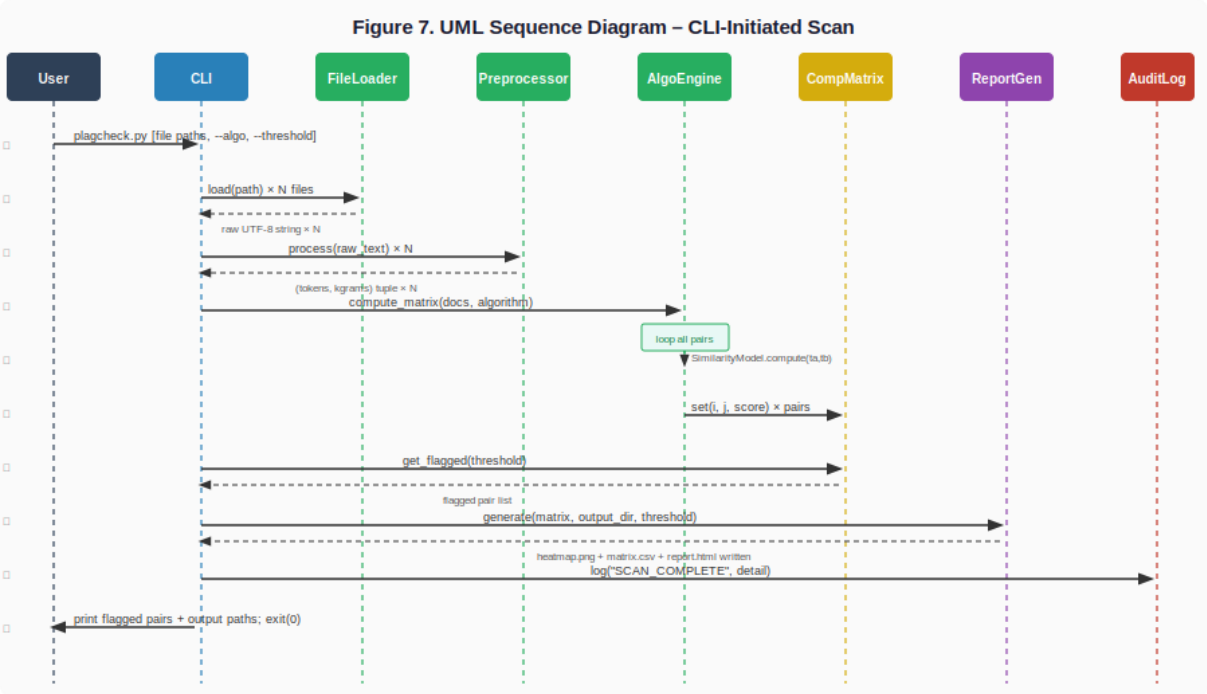


Figure 8 - UML Sequence Diagram – CLI-initiated scan

4.2.4 State Transition Diagram

Each document that is submitted goes through seven steps. These steps are: Uploaded, Validated, Parsed, Preprocessed, Fingerprinted, Compared, and ReportGenerated. If something goes wrong like a validation failure, parsing failure or encoding detection failure, a document can move to the Error state. This can happen at any of the three steps. When a document is in the Error state it is removed from the group of documents being compared. An error message, FR-18 is sent to the user. The Error state is final for that document. However, the rest of the documents in the group continue through the

steps as usual. That's why way one document with a problem does not stop the analysis of the documents in the group.

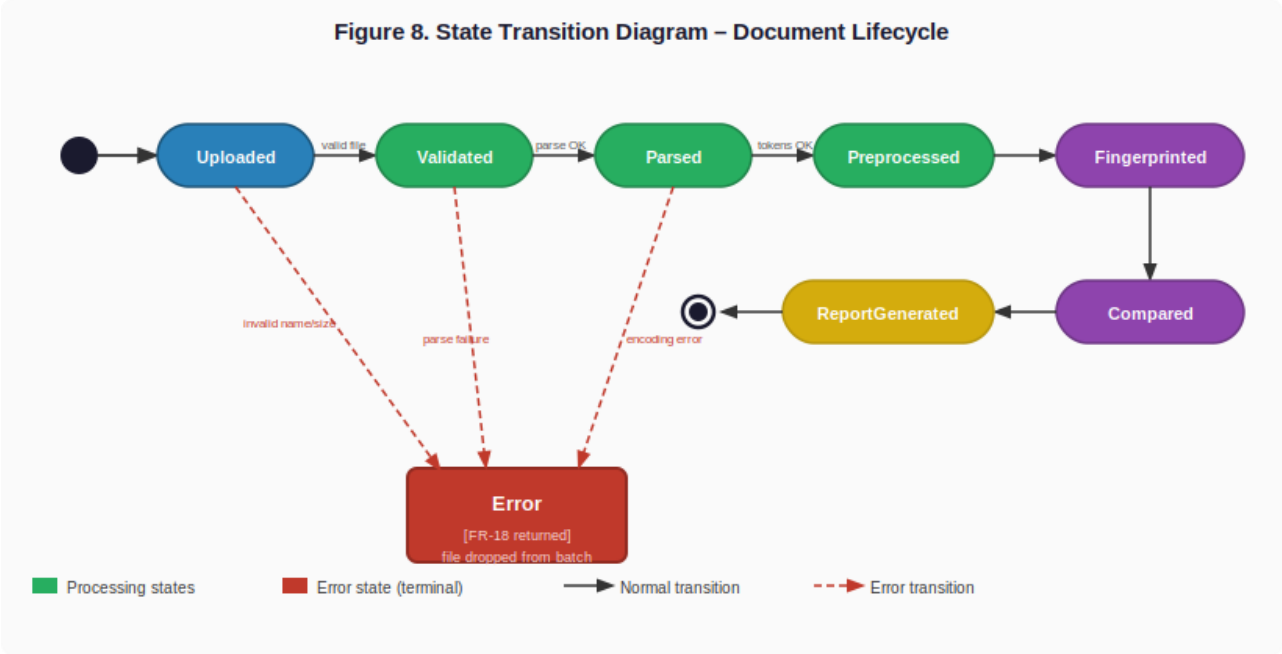


Figure 9 - State Transition Diagram — Document lifecycle through seven states

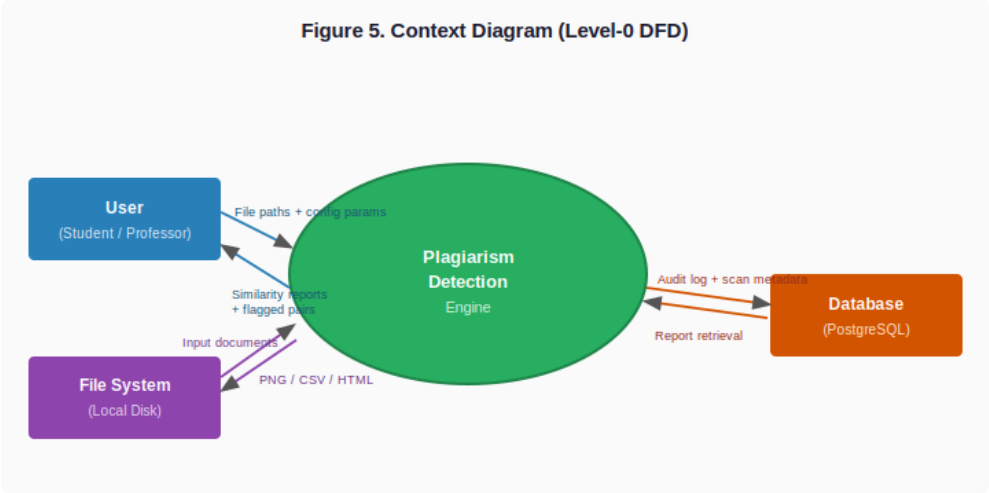
4.2.5 Data Flow Diagram

At Level 0 there is a process called Plagiarism Detection System. This system gets two inputs from the User. The inputs are Document Files and Configuration Parameters. The system then produces two outputs. These outputs are Similarity Reports and Audit Records. At Level 1 the Plagiarism Detection System breaks down into five sub-processes. Process 1 (File Ingestion and Validation) reads raw files from Data Store D1 (Raw Documents) and writes validated text to D2 (Normalised Token Store). Process 2, called Text Extraction and Preprocessing, takes the data stored in D2 and converts it into stemmed token lists and k-gram sets. This step prepares the data for the similarity analysis performed in later stages of the system. Process 3, called Similarity Computation, reads the required data from D2 and D3 (Fingerprint Cache). Using this information, it calculates similarity scores for document pairs and stores the results in D4, which is known as the Similarity Matrix. Process 4 is responsible for generating reports. It reads the data from D4, creates HTML, CSV, and PNG output files, and saves them to the file system. Process 5 is responsible for audit logging. It gets event data from processes and writes it to D5. D5 is the Audit Log of our system. The Audit Log is kept in PostgreSQL.

Context Diagram

At the context level the system boundary separates the Plagiarism Detection Engine from three external entities. The User entity (a student or professor interacting through the CLI or API) sends file paths and configuration parameters to the system and receives similarity reports and flagged pair lists in return. The File System entity provides the document files as input and receives the generated output artifacts (heatmap PNG, CSV matrix, HTML report). The Database entity (PostgreSQL) receives structured audit log entries and scan metadata, and responds to report retrieval queries. There are no other external dependencies once the libraries have been installed.

Figure 10 - Context Diagram (Level 0 DFD)



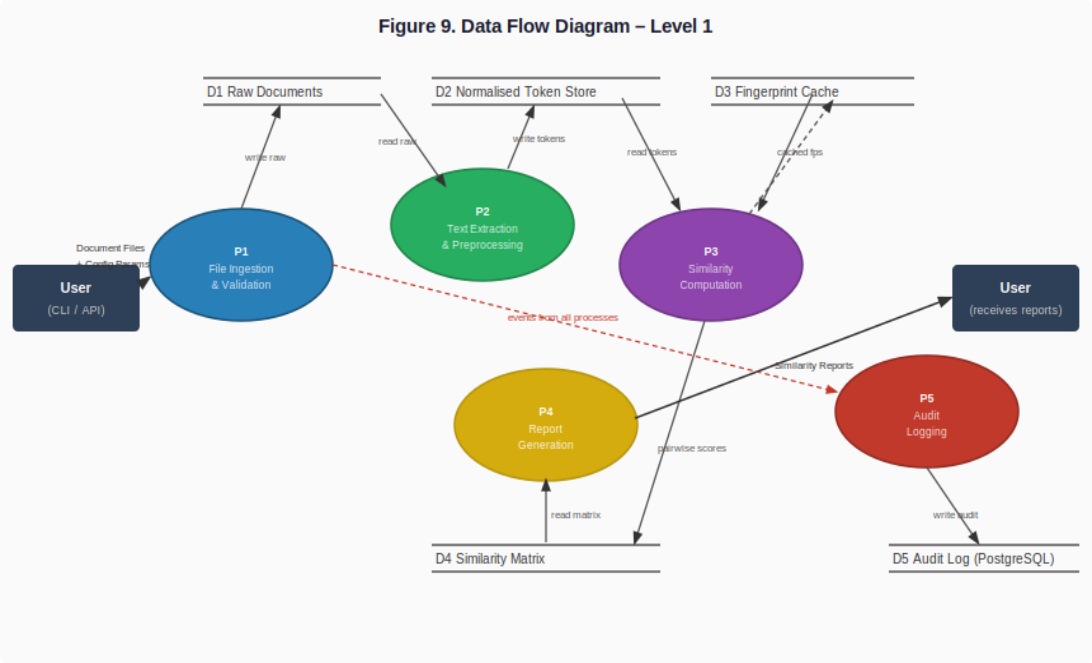


Figure 11 - Data Flow Diagram — Level 1 decomposition into five sub-processes

4.2.6 Activity Diagram

Activity diagram represents the overall behavioral process of the Plagiarism Detection System from the submission of file paths via the command line interface (CLI) and REST API to the output creation process. It includes UML 2.x activity notation with decision points (diamond), activities (round rectangle), and swimlaneless straight-line representation supplemented with two branches. The first decision point determines whether the files sent to FileLoader are valid in terms of extension, file size, and encoding type. Invalid files are instantly refused and marked as an error by AuditLogger without further actions taken on them. The second decision point determines whether the ComparisonMatrix output meets the user-defined threshold. If not, the system omits report generation stage and passes to the audit logging stage directly.

Key action nodes in the main flow include:

- FileLoader.load() — validates and parses each submitted file into raw UTF-8 text.
- Preprocessor.process() — tokenises, lowercases, removes stop-words, applies Porter stemming, and generates k-grams.
- AlgorithmEngine.run() — dispatches computation to the selected SimilarityModel (TF-IDF, Winnowing, Jaccard, or AST).
- ComparisonMatrix.get_flagged() — applies the threshold and returns the list of suspicious document pairs.
- ReportGenerator.generate() — produces the heatmap PNG, similarity CSV, and HTML comparison report.

- AuditLogger.log() — persists a JSON-format event record to PostgreSQL or the fallback log file.

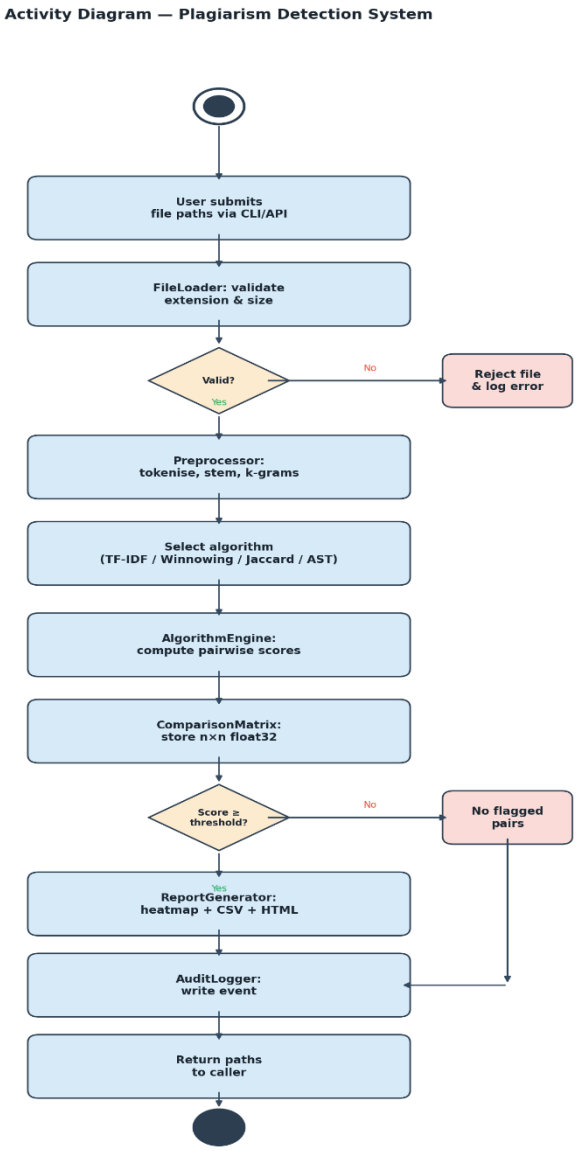


Figure 12 - Activity Diagram — End-to-End Processing Flow

4.2.7 BPMN Diagram

In the BPMN 2.0 Diagram below, system processes are divided into three lanes corresponding to three tiers of the architecture introduced in section 4.1: User/Client lane – Presentation tier, System Pipeline lane – Application tier, and Database lane – Data tier. Only operations that are performed by particular actor/component are included in that lane.

Process starts when User performs upload and configures algorithm/thresholds in User lane. Then control goes to System Pipeline lane where FileLoader checks all files. If there is any erroneous file

then control moves to User lane through exclusive gateway where an error response is sent. In case of correct files, control goes further to Preprocessor operation. AlgorithmEngine performs processing of each pair of documents and places the results into ComparisonMatrix. ReportGenerator builds three reports and outputs file paths to the User lane.

For each operation in the system a corresponding message is passed into the Database lane to save scan request data (scan_request, scan_file, scan_pair, scan_algorithm) as well as audit log. There are no output sequence flows from the tasks of database lane into other lanes – they operate independently of the system operation on intermediate message events.

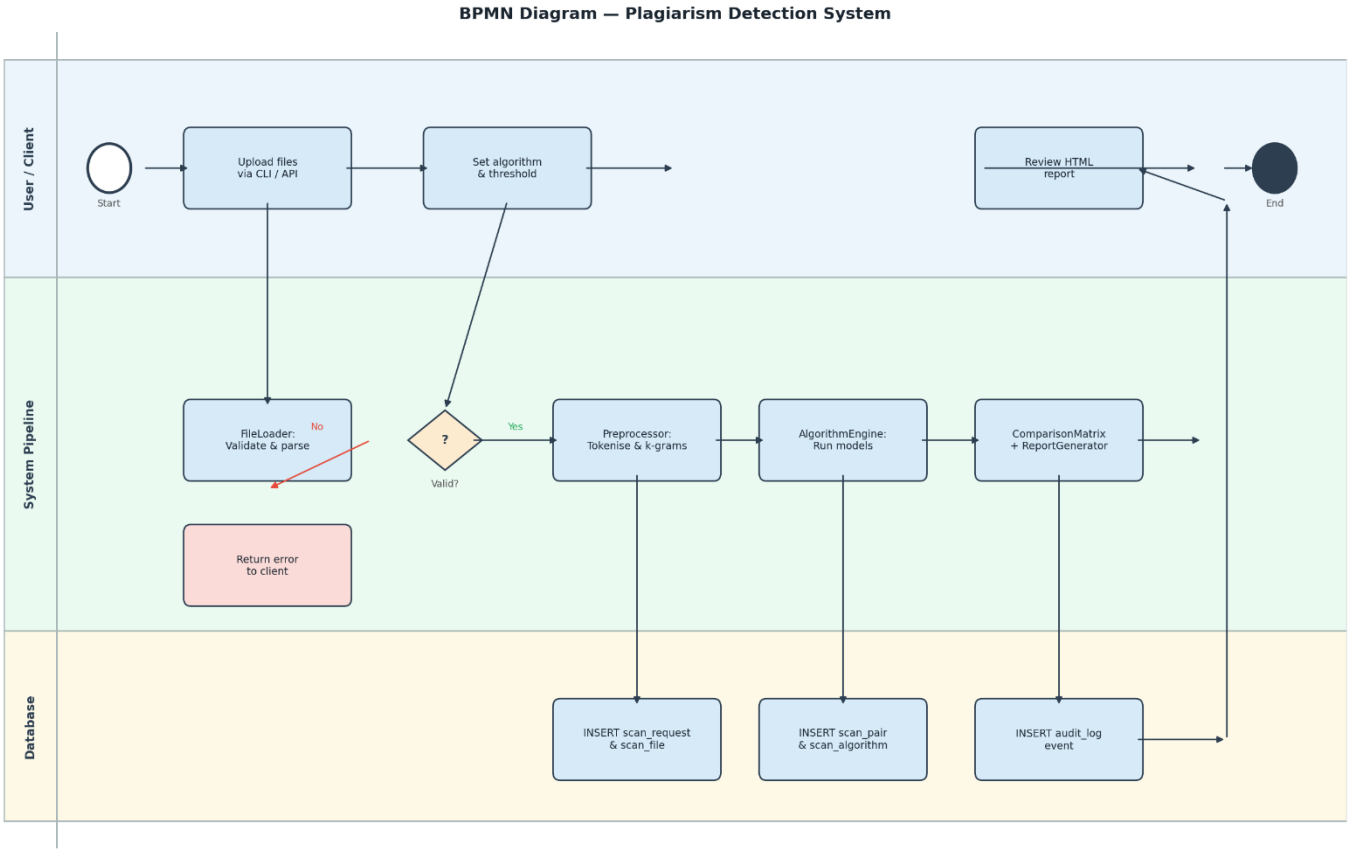


Figure 13 - BPMN Diagram — Three-Lane Process Flow

4.2.8 Package Diagram

The Package Diagram provides a bird's-eye view of the system's module structure and the dependency relationships between logical groups of components. Six packages are defined, each encapsulating a cohesive set of responsibilities:

- presentation — the entry points through which users interact with the system: the CLI (plagcheck.py), the Flask REST API (app.py), and the two output formats (HTML report and PNG heatmap).

- core — the processing pipeline components: FileLoader, Preprocessor, AlgorithmEngine, ComparisonMatrix, ReportGenerator, and AuditLogger. This is the central package upon which all other packages depend.
- models — the similarity computation layer: the abstract SimilarityModel interface and its four concrete implementations (CosineModel, WinnowingModel, JaccardModel, ASTModel). This package is used exclusively by core via the Strategy pattern.
- data — the persistence layer: the PostgreSQL schema tables (scan_request, scan_file, scan_pair, scan_algorithm, audit_log) and the fingerprint cache.
- external_libs — third-party libraries consumed by the models and core packages: NLTK, scikit-learn, PyMuPDF, pdfplumber, python-docx, and seaborn.
- config — runtime configuration artifacts: the .env file, EXCLUSION_LIST path, threshold default, and LOG_LEVEL setting.

The dependency arrows show that presentation depends on core, core depends on models and data, models depend on external_libs, and config is consumed by both core and models. There are no circular dependencies.

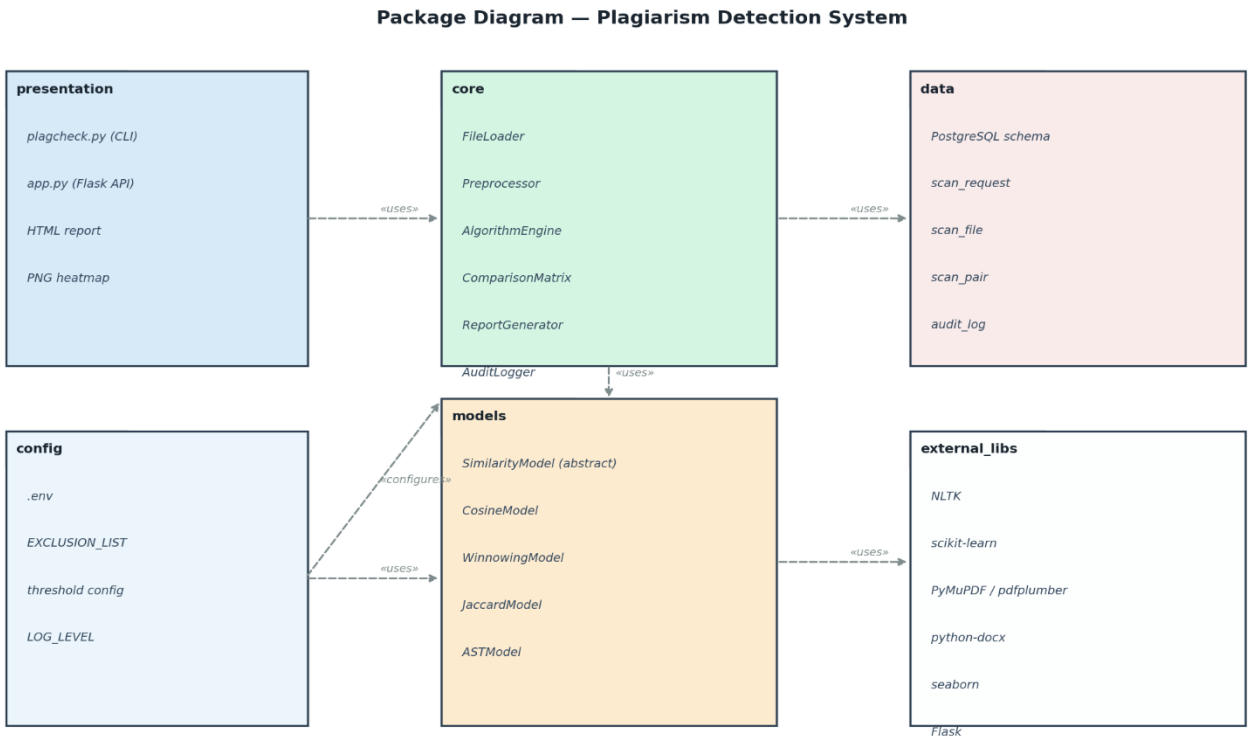


Figure 14 - Package Diagram — Module Structure and Dependencies

4.2.9 Communication Diagram

The Communication Diagram (formerly known as the Collaboration Diagram in UML 1.x) illustrates the object interactions that occur during a typical scan request, with numbered messages showing the sequence of method calls and data transfers between the system objects. Unlike the Sequence Diagram in Section 4.2.5 which emphasizes temporal ordering on a vertical timeline, the Communication Diagram emphasizes the structural topology — which objects communicate with which.

Nine numbered interactions are depicted:

- 1: client:CLI/API → loader:FileLoader — load_files(paths) initiates file ingestion.
- 2: loader:FileLoader → prep:Preprocessor — preprocess(text) hands off raw text for NLP processing.
- 3: client:CLI/API → audit:AuditLogger — log(SCAN_START) records the start of the scan.
- 4: prep:Preprocessor → engine:AlgorithmEngine — run(tokens, algo) triggers pairwise similarity computation.
- 5: engine:AlgorithmEngine → matrix:ComparisonMatrix — set(i, j, score) populates each matrix cell.
- 6: matrix:ComparisonMatrix → reporter:ReportGenerator — generate(matrix) builds all output artifacts.
- 7: reporter:ReportGenerator → db:PostgreSQL — INSERT scan_pair persists results to the database.
- 8: reporter:ReportGenerator → fs:FileSystem — write outputs saves the three artifact files.
- 9: audit:AuditLogger — log(SCAN_COMPLETE) closes the audit trail.

A dashed return arrow from audit:AuditLogger back to client:CLI/API delivers the result file paths upon completion, completing the interaction cycle.

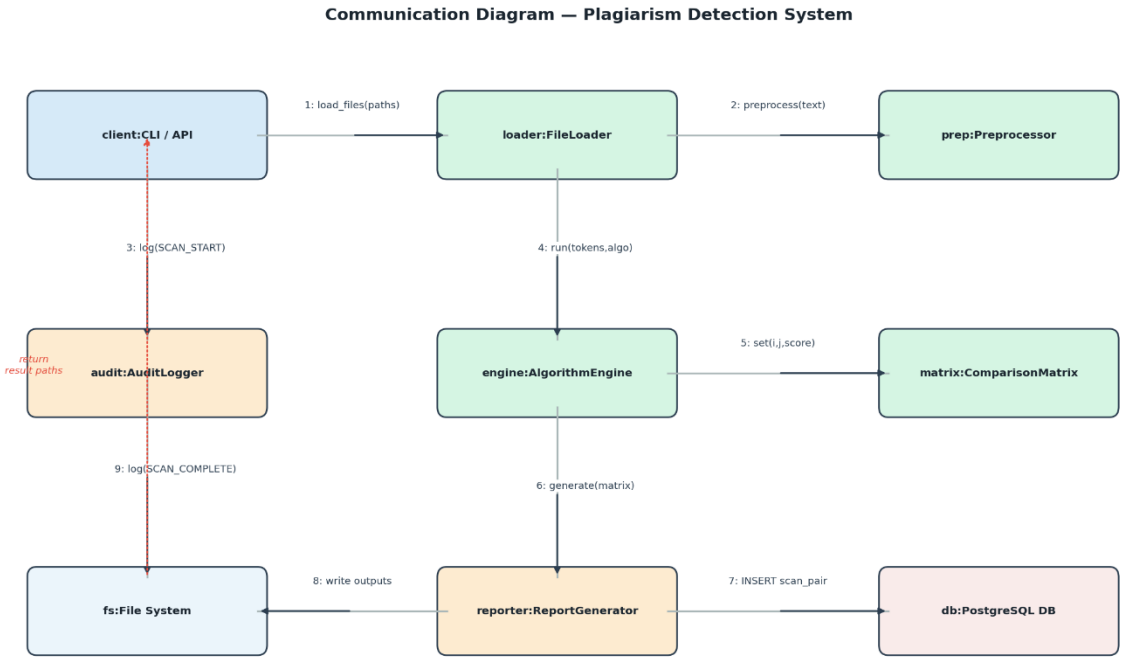


Figure 15 - Communication Diagram — Object Interactions During Scan

4.3 Database Design and Normalisation

The relational database component stores scan metadata and audit records in PostgreSQL. The schema was designed through a formal normalisation process from the initial unnormalised flat representation through First, Second, and Third Normal Form. The normalisation is documented step by step below with a table for each stage.

4.3.1 Unnormalised Form (UNF)

The initial conception captured all data in a single flat table called scan record. This table contains repeating groups for files in each batch and for pairwise scores. It also includes multi-valued attributes for algorithm names. Before using the schema in production problems need to be solved.

Table 33 - Unnormalised scan_record table (UNF)

Attribute	Data Type	Notes
scan_id	INTEGER	Unique per request
scan_timestamp	TIMESTAMP	Time of submission

algorithm_used	VARCHAR(50)	Could be comma-separated (violation)
threshold	FLOAT	Similarity threshold
file_1_name, ..., file_n_name	VARCHAR(255)	Repeating group — one per file
file_1_size, ..., file_n_size	BIGINT	Repeating group — one per file
pair_score_1_2, ..., pair_score_i_j	FLOAT	Repeating group — one per pair
flagged_pairs	TEXT	Comma-separated list (violation)
user_id	INTEGER	Submitting user
user_name, user_email	VARCHAR	User attributes — transitive dependency

4.3.2 First Normal Form (1NF)

1NF requires atomic values, no repeating groups, and a unique row identifier. To achieve 1NF, the repeating file group is extracted into a separate scan_file table, the repeating pair score group is extracted into scan_pair, and the comma-separated algorithm list is extracted into scan_algorithm. The flagged_pairs text field is replaced by a flagged boolean column in scan_pair.

Table 34 - scan_request after 1NF

Attribute	Type	Key	Notes
scan_id	INTEGER	PK	Auto-increment
scan_timestamp	TIMESTAMP		NOT NULL
threshold	FLOAT		DEFAULT 0.70
user_id	INTEGER	FK → user	user_name/email still here — resolved in 2NF
user_name	VARCHAR(100)		Transitively dependent — to be removed
user_email	VARCHAR(150)		Transitively dependent — to be removed

Table 35 - scan_file after 1NF

Attribute	Type	Key	Notes
file_id	INTEGER	PK	Auto-increment
scan_id	INTEGER	FK → scan_request	
file_name	VARCHAR(255)		Sanitised filename
file_size_bytes	BIGINT		
file_format	VARCHAR(10)		txt / py / pdf / docx

Table 36 - scan_pair after 1NF

Attribute	Type	Key	Notes
pair_id	INTEGER	PK	
scan_id	INTEGER	FK → scan_request	
file_id_a	INTEGER	FK → scan_file	
file_id_b	INTEGER	FK → scan_file	file_id_a < file_id_b enforced by CHECK
similarity_score	FLOAT		Range [0.00, 1.00]
flagged	BOOLEAN		Materialized: score >= threshold

4.3.3 Second Normal Form (2NF)

2NF requires that the schema is in 1NF and that every non-key attribute is fully functionally dependent on the entire primary key. Since scan_request uses a single-column primary key, partial dependencies cannot arise there. The 2NF violation is the presence of user_name and user_email in scan_request,

where they depend on user_id rather than on scan_id. Extracting these into a dedicated user table resolves the violation.

Table 37 - user table extracted at 2NF

Attribute	Type	Key	Notes
user_id	INTEGER	PK	
user_name	VARCHAR(100)		NOT NULL
user_email	VARCHAR(150)	UNIQUE	
user_role	VARCHAR(20)		student / professor / admin
created_at	TIMESTAMP		DEFAULT NOW()

4.3.4 Third Normal Form (3NF)

3NF requires that no non-key attribute depends on another non-key attribute (no transitive dependencies). After 2NF, the only remaining potential transitive dependency is in scan_pair where flagged depends on similarity_score and threshold rather than directly on pair_id. Since threshold is stored in scan_request and similarity_score is in scan_pair, flagged is technically a derived value. The design retains flagged as a materialized computed column (populated at write time by the application) to avoid a join-on-every-write penalty, but documents this dependency explicitly: flagged := (similarity_score >= threshold). All other non-key attributes across all tables depend solely and directly on their respective primary keys. The schema is in 3NF.

Table 38 - Final 3NF schema summary

Table	Primary Key	Foreign Keys
app_user	user_id	None
scan_request	scan_id	user_id → app_user.user_id
scan_file	file_id	scan_id → scan_request.scan_id
scan_pair	pair_id	scan_id → scan_request.scan_id; file_id_a, file_id_b → scan_file.file_id
scan_algorithm	(scan_id, algorithm_name)	scan_id → scan_request.scan_id

audit_log	log_id	scan_id → scan_request.scan_id; user_id → app_user.user_id

Table 39 - audit_log table (3NF)

Attribute	Type	Key	Notes
log_id	INTEGER	PK	
scan_id	INTEGER	FK → scan_request	ON DELETE SET NULL
user_id	INTEGER	FK → app_user	ON DELETE SET NULL
event_type	VARCHAR(50)		SCAN_START, SCAN_COMPLETE, SCAN_ERROR, FILE_REJECTED, API_REQUEST
event_timestamp	TIMESTAMP		NOT NULL DEFAULT NOW()
event_detail	TEXT		JSON-format event payload

4.3.5 E-R Diagram

The Entity-Relationship Diagram defines the relational schema of the PostgreSQL database that forms the Data tier of the three-tier architecture. The schema was designed in Third Normal Form (3NF) as part of Section 4.3.4 in the main report. The E-R diagram presented here provides a visual complement to the normalisation tables, showing entity attributes and their cardinality relationships at a glance.

Five entities are modelled:

- scan_request — the root entity for each analysis session. Attributes: scan_id (UUID, PK), submitted_at, algorithm, threshold, status, file_count.
- scan_file — represents each individual document submitted in a scan. Attributes: file_id (UUID, PK), scan_id (FK → scan_request), original_name, file_type, file_size_bytes, parsed_text_len. Cardinality: scan_request 1 — N scan_file.

- scan_pair — records the similarity result for each document pair within a scan. Attributes: pair_id (UUID, PK), scan_id (FK), file_a_id (FK → scan_file), file_b_id (FK → scan_file), similarity_score, flagged. Cardinality: scan_request 1 — N scan_pair.
- scan_algorithm — stores the individual algorithm result for each pair when multiple algorithms are run. Attributes: alg_id (UUID, PK), pair_id (FK → scan_pair), algorithm_name, score, execution_ms. Cardinality: scan_pair 1 — N scan_algorithm.
- audit_log — the immutable event log. Attributes: log_id (UUID, PK), scan_id (FK → scan_request), event_type, event_timestamp, event_detail (JSONB). Cardinality: scan_request 1 — N audit_log.

All primary keys are UUID v4 values generated by the application layer to avoid sequential ID leakage. The JSONB column event_detail in audit_log allows structured metadata to be stored without requiring schema migrations when new event fields are introduced.

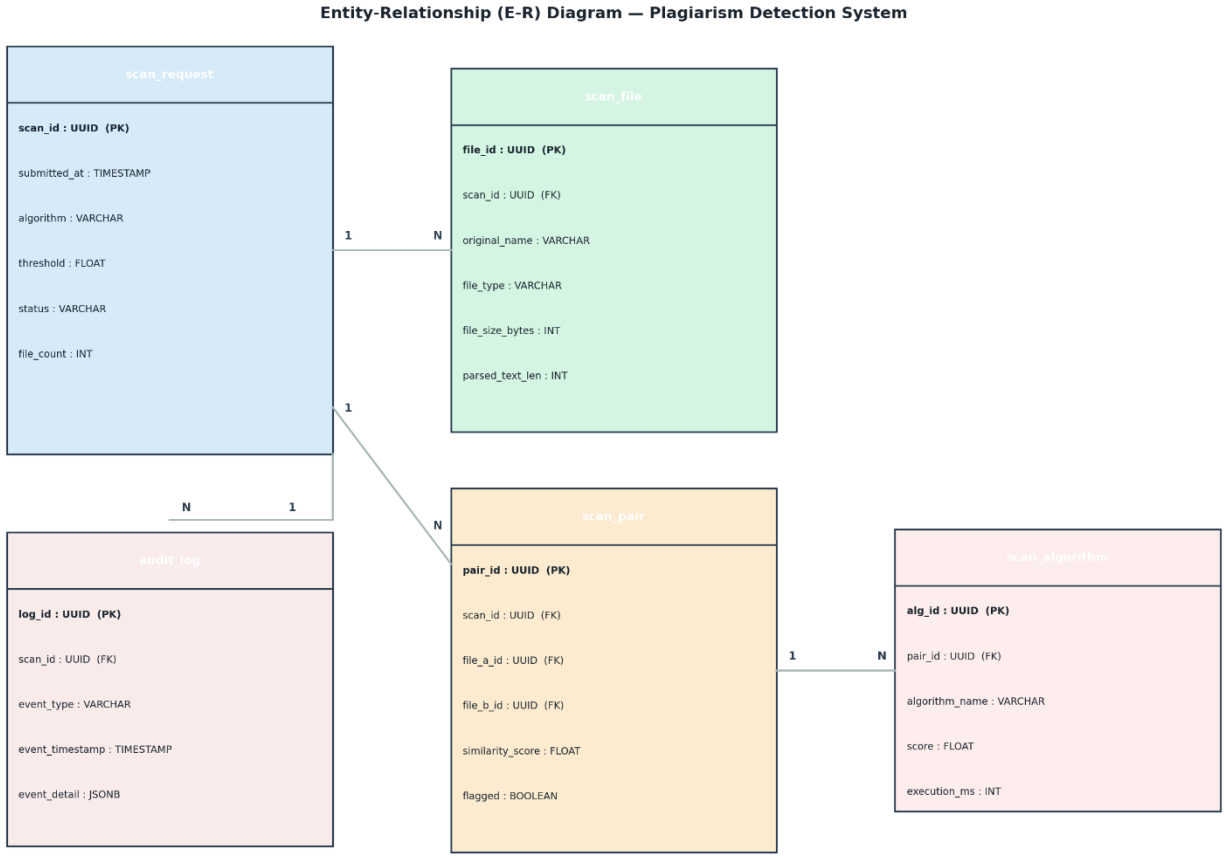


Figure 16 - Entity-Relationship Diagram — PostgreSQL Schema (3NF)

4.3.6 Physical Database Tables

In this part, we will describe the physical structure of all six tables in PostgreSQL database. Column name, its data type, constraints, default value, and description of each of them can be found in the following. The current schema follows the 3NF logical design presented in Section 4.3.4 and the whole DDL statement for database construction can be viewed in Appendix B.

Important design considerations for the physical schema:
(a) All primary keys of database tables were chosen as SERIAL (autoincrementing integers) not UUID since using UUID makes join columns larger. Scan-level UUIDs (created at application layer using uuid.uuid4()) will only be added to JSON event payload if necessary and stored inside audit_log table if needed for external references.
(b) ON DELETE CASCADE is set to scan_file, scan_pair and scan_algorithm tables to delete the corresponding children whenever their parent, scan_request record, is deleted.
(c) On the other hand, ON DELETE SET NULL is set to audit_log foreign keys such that the log records could still be stored even if their scans/users do not exist anymore and satisfy the retention requirement.
(d) Flagged attribute in scan_pair is calculated at write time by application layer and represents a materialized value (flagged := similarity_score >= threshold).

Table: app_user				
Column Name	Data Type	Constraint(s)	Default	Description
user_id	SERIAL	PK, NOT NULL	auto-increment	Talked to plugin.gptinf.com
user_name	VARCHAR(100)	NOT NULL	—	Primary key for surrogate; auto-incrementing integer value.
user_email	VARCHAR(150)	NOT NULL, UNIQUE	—	User's full name for display purposes.

user_role	VARCHAR(20)	CHECK IN ('student','professor','admin'), NOT NULL	'student'	E-mail address for institution; unique within the database.
created_at	TIMESTAMP	NOT NULL	NOW()	Role that governs what use cases a user can initiate.

Table 39a - app_user — Physical table definition

Table: scan_request				
Column Name	Data Type	Constraint(s)	Default	Description
scan_id	SERIAL	PK, NOT NULL	auto-increment	Unique ID for scan session.
user_id	INTEGER	FK → app_user(user_id), NOT NULL, ON DELETE RESTRICT	—	Submitting user; deleting a user who has any scans is prevented.
scan_timestamp	TIMESTAMP	NOT NULL	NOW()	Timestamp UTC when the scan was submitted.
threshold	FLOAT	NOT NULL, CHECK (threshold > 0 AND threshold < 1)	0.70	Similarity threshold; logged for auditing purposes.
status	VARCHAR(20)	NOT NULL, CHECK IN ('pending','running','complete','error')	'pending'	Current lifecycle stage of the scan; set by AlgorithmEngine.

Table 39b - scan_request — Physical table definition

Table: scan_file				
Column Name	Data Type	Constraint(s)	Default	Description
file_id	SERIAL	PK, NOT NULL	auto-increment	File record unique identifier.
scan_id	INTEGER	FK → scan_request(scan_id), NOT NULL, ON DELETE CASCADE	—	Parent scan; removal of a scan deletes all its file records.
file_name	VARCHAR(255)	NOT NULL	—	Filename sanitization after white list pattern match; no directory prefixes stored.
file_size_bytes	BIGINT	NOT NULL, CHECK (file_size_bytes > 0)	—	File size in bytes; should be > 0 (zero-size files are not accepted by FileLoader).
file_format	VARCHAR(10)	NOT NULL, CHECK IN ('txt','py','pdf','docx')	—	File extension/format; determines parser used in FileLoader.

Table 39c - scan_file — Physical table definition

Table: scan_pair				
Column Name	Data Type	Constraint(s)	Default	Description
pair_id	SERIAL	PK, NOT NULL	auto-increment	Identifies each unique pairing of results.
scan_id	INTEGER	FK → scan_request(scan_id), NOT NULL, ON DELETE CASCADE	—	Relates each result to its respective scan.
file_id_a	INTEGER	FK → scan_file(file_id), NOT NULL	—	Low index file within the pairing (file_id_a < file_id_b constraint).
file_id_b	INTEGER	FK → scan_file(file_id), NOT NULL	—	High index file within the pairing.
similarity_score	FLOAT	NOT NULL, CHECK (similarity_score >= 0 AND similarity_score <= 1)	—	Similarity score between 0.00 and 1.00; float value accurate to 4 d.p.
flagged	BOOLEAN	NOT NULL	FALSE	Calculated value within materialized column where similarity_score >= threshold of parent scan.
—	CONSTRAINT	chk_pair_order CHECK (file_id_a < file_id_b)	—	Eliminates any duplicated reversed pairing entries.

Table 39d - scan_pair — Physical table definition

Table: scan_algorithm				
Column Name	Data Type	Constraint(s)	Default	Description
scan_id	INTEGER	PK (composite), FK → scan_request(scan_id), NOT NULL, ON DELETE CASCADE	—	Parent scan identifier; included as part of the composite primary key.
algorithm_name	VARCHAR(50)	PK (composite), NOT NULL, CHECK IN ('cosine','winnowing','jaccard','ast','all')	—	Name of the invoked algorithm; also included as part of the composite primary key.

Table 39e - scan_algorithm — Physical table definition

Table: audit_log				
Column Name	Data Type	Constraint(s)	Default	Description
log_id	SERIAL	PK, NOT NULL	auto-increment	Identifier for unique log entry.
scan_id	INTEGER	FK → scan_request(scan_id), ON DELETE SET NULL	NULL	Triggering scan; SET NULL

				when scan is deleted to preserve log record.
user_id	INTEGER	FK → app_user(user_id), ON DELETE SET NULL	NULL	User who initiated the event; SET NULL if user account is deleted.
event_type	VARCHAR(50)	NOT NULL, CHECK IN ('SCAN_START','SCAN_COMPLETE','SCAN_ERROR','FILE_REJECTED','API_REQUEST','API_ERROR')	—	Category of the event; used as a filter for admin audit queries.
event_timestamp	TIMESTAMP	NOT NULL	NOW()	UTC datetime

				stamp of the event; sorted DESC for efficient query performance.
event_detail	TEXT	—	NULL	JSON payload containing event metadata; no document content is kept in the database.

Table 39f - audit_log — Physical table definition

Database Indexes			
Index Name	Table	Column(s)	Purpose

idx_user_email	app_user	user_email	Fast e-mail look-up upon login or API authorization check.
idx_scan_user	scan_request	user_id	Get all the scans for the specified user efficiently.
idx_file_scan	scan_file	scan_id	Get all files of the specified scan.
idx_pair_scan	scan_pair	scan_id	Get all the pairs for the specified scan (Report Generator).
idx_pair_flagged	scan_pair	flagged WHERE flagged=TRUE	Partial indexing; speeds up search for flagged pairs without having to scan unflagged records.
idx_audit_scan	audit_log	scan_id	Get all events of the specified scan.
idx_audit_ts	audit_log	event_timestamp DESC	Recent log entries order query (UC-13).

Table 39g - Database indexes and their purposes

5. IMPLEMENTATION

5.1 Tools, Technologies and Platforms Used

All implementation work has been done using Python 3.10 version. Python language was the obvious choice for implementing this work based on the project theme, familiarity with the language by the group members, and availability of good scientific computing libraries and natural language processing tools in the Python framework. Key libraries include NLTK [8] for NLP, scikit-learn [9] for TF-IDF and cosine similarity, NumPy [10] for matrix operations, seaborn [11] and Matplotlib [15] for visualisation, pdfplumber [12] and PyMuPDF [13] for PDF parsing, python-docx [25] for Word document parsing, Flask [28] for the REST API, PostgreSQL [26] for audit logging, chardet [27] for encoding detection, and pytest [29] for testing. All the libraries used are open source software and are freely available under the licenses of MIT, BSD-3, and Apache 2.0.

Table 40 - Technology stack

Component	Library	Version	Purpose
Core language	Python	3.10.12	All system logic
NLP toolkit	NLTK	3.8.1	Tokenisation, stop-words, Porter stemmer
TF-IDF and cosine similarity	scikit-learn	1.3.2	TfidfVectorizer, cosine_similarity
Numerical computation	NumPy	1.26.0	Similarity matrix, pairwise indexing
Heatmap visualisation	seaborn + matplotlib	0.13.0 / 3.8.0	Similarity heatmap as 300 DPI PNG
PDF parsing (primary)	pdfplumber	0.10.3	Text extraction from text-based PDFs
PDF parsing (fallback)	PyMuPDF (fitz)	1.23.6	Fallback parser; handles unusual encodings
Word document parsing	python-docx	1.1.0	Text extraction from .docx files
Python AST parsing	ast (stdlib)	built-in	AST extraction for code comparison
Hashing	hashlib (stdlib)	built-in	SHA-256 k-gram hashing for Winnowing
REST API	Flask	3.0.0	REST endpoints for /api/check and /api/report
Database	PostgreSQL	15.4	Scan metadata and audit logging
DB adapter	psycopg2	2.9.9	PostgreSQL connector for Python
Encoding detection	chardet	5.2.0	Detect non-UTF-8 encoded files
CLI	argparse (stdlib)	built-in	Command-line argument parsing

Unit testing	pytest + pytest-cov	7.4.3 / 4.6.4	Test execution and coverage reporting
Code quality	flake8	6.1.0	PEP 8 linting on every commit
Version control	Git / GitHub	2.42.0	Source control, CI, code review

5.2 Similarity Algorithms

5.2.1 TF-IDF Cosine Similarity

The notion of weightings through TF-IDF was coined by Salton & Buckley [17], where the abbreviations TF and IDF stand for Term Frequency and Inverse Document Frequency respectively. It denotes a way to assign scores/weights to terms depending on how many times these occur in a larger collection of documents. Given that the term t appears tf(t,d) number of times in document d, its TF-IDF score in the N documents' corpus would be $tfidf(t,d) = tf(t,d) \times \log(N / df(t))$.

Words such as 'the', 'is', 'a', which are frequent in most documents, will receive low IDF scores, whereas those which appear only in some of the documents will be assigned high IDF scores. Cosine similarity of two vectors A and B is computed as $\cos(A,B) = (A \cdot B) / (||A|| \times ||B||)$. When cosine similarity equals 1.0, it means two documents match perfectly in TF-IDF space, whereas 0.0 means there is no overlap in vocabulary at all. Documents having cosine similarity exceeding 0.7 could be considered as matching in academic writing.

One of the benefits of this approach is that all of the problems involved in cosine similarity computation are addressed. This process is made more efficient because cosine similarities can be calculated using sparse matrices since, for most cases, the vocabulary overlap in the documents is significantly lower than the size of the whole vocabulary pool available. Another benefit of TF-IDF cosine similarity is its capability of detecting vocabulary overlap based on themes rather than words.

From the pseudo-code given in Table 40, one can see that, first of all, it is needed to check that the input tokens are proper non-empty arrays (Line 1). Then, we create two-document corpus by joining token arrays into one string (Line 2). Now, we initiate our TfidfVectorizer and get the sparse TF-IDF matrix (Lines 3-4). Further on, we extract vectors for each document (Lines 5-6), then calculate the dot product of them (Line 7), and finally get norms of the vectors (Lines 8-9). Before dividing (Line 11), we check if vectors have zero norms, thus preventing division by zero error (Line 10).

Algorithm 1 - Pseudocode — TF-IDF cosine similarity

```
ALGORITHM: TF-IDF Cosine Similarity
INPUT:  tokens_a, tokens_b (list[str])
OUTPUT: score FLOAT in [0.0, 1.0]


1.  IF tokens_a is empty OR tokens_b is empty: RETURN 0.0
2.  corpus     ← [' '.join(tokens_a), ' '.join(tokens_b)]
3.  vectorizer ← TfidfVectorizer()
4.  tfidf_mat  ← vectorizer.fit_transform(corpus)   // shape (2, |vocab|)
5.  vec_a      ← tfidf_mat[0]
6.  vec_b      ← tfidf_mat[1]
7.  dot        ← vec_a · vec_b
8.  norm_a     ← ||vec_a||₂
9.  norm_b     ← ||vec_b||₂
10. IF norm_a = 0 OR norm_b = 0: RETURN 0.0
11. score ← dot / (norm_a × norm_b)
12. RETURN CLAMP(score, 0.0, 1.0)
```

5.2.2 Winnowing Fingerprinting

Winnowing is a local fingerprinting algorithm proposed by Schleimer et al. [7] which guarantees finding of all common subsequences and immunity to local changes. The idea behind winnowing is to select not any possible k-grams of a given hash sequence, but just one of them – the k-gram having the lowest hash value among all hashes of all hashes in the sliding window of length w.

In the implementation of the winnowing algorithm, SHA-256 is used to perform hashing of the k-grams and take the first 32 bits of the results as the fingerprints' values because of their extremely low collision rate. Compared to the TF-IDF cosine similarity algorithm, winnowing algorithm is better as it does not take into consideration just the words appearing in both documents, but also quite complicated five-gram sequences which are rather hard to copy.

Table 41 shows a detailed description of winnowing steps execution. STEP 1 consists in forming k-length (k-gram) overlapping sequences from the tokens (Line 1) and hashing of those sequences to obtain the 32-bit integers using the SHA-256 cryptographic algorithm (Line 2). In STEP 2, the sliding window selection is performed. Namely, at first, an empty collection is defined as the set for storing the resulting fingerprints (Line 3). Then, the hashes are processed with the use of the sliding window of length w (Lines 5-6). At Line 7, the minimum hash value is selected in the current window and saved. At Line 8, the index of the minimum hash is also saved to avoid saving duplicates of the minimum hashes in case of overlaps between windows. Thus, if the index of the current window

minimum value differs from the previous index (Line 9), the hash value is put into the fingerprint set (Line 10).

Algorithm 2 - Pseudocode — Winnowing fingerprinting

```
ALGORITHM: Winnowing Fingerprinting
INPUT:  tokens (list[str])
        k (int, default 5)  — k-gram length
        w (int, default 4)  — window size
OUTPUT: fingerprints (set[int])

STEP 1: Generate k-grams and hash each one
1.  kgrams    ← [' '.join(tokens[i:i+k]) FOR i IN range(len(tokens)-k+1)]
2.  hash_list ← [int(SHA256(kg.encode('utf-8')).hexdigest(), 16) % 2^32
                 FOR kg IN kgrams]

STEP 2: Sliding window minimum selection
3.  fingerprints   ← SET()
4.  prev_min_index ← -1
5.  FOR i FROM 0 TO len(hash_list) - w (inclusive):
6.      window    ← hash_list[i : i+w]
7.      min_val   ← MIN(window)
8.      min_idx   ← i + INDEX_OF(min_val, window)
9.      IF min_idx != prev_min_index:
10.         fingerprints.ADD(min_val)
11.         prev_min_index ← min_idx
12. RETURN fingerprints

Similarity: J(A,B) = |fingerprints_A ∩ fingerprints_B|
                   / |fingerprints_A ∪ fingerprints_B|
```

5.2.3 Jaccard Index

The Jaccard index [19], a well-known method of calculating set similarities, is mathematically expressed by the following equation: $J(A,B) = |A \cap B| / |A \cup B|$, where A and B are the sets. As soon as we are dealing with textual information, the sets A and B will consist of stemmed tokens of both files. It can be said that Jaccard index is the most straightforward way of measuring similarities between two texts since this algorithm does not require any time at all. What is more, this algorithm disregards term weighting and token order issues while measuring similarities.

As can be seen from the presented table, Jaccard index performs rather effectively. The first step in performing the algorithm is transforming the inputted stemmed tokens into sets in the mathematical sense without any duplicates (lines 1-2). Next, the function calculates the union and intersection of the

sets (lines 3-4). Then there comes the safety check phase during which an empty intersection proves the absence of proper tokens in one document and consequently makes the Jaccard index equal to zero (line 5). Otherwise, the Jaccard index will be calculated the way shown on line 6.

Algorithm 3 - Pseudocode — Jaccard index

```
ALGORITHM: Jaccard Index
INPUT:  tokens_a, tokens_b (list[str])
OUTPUT: score FLOAT in [0.0, 1.0]


1. set_a        ← SET(tokens_a)
2. set_b        ← SET(tokens_b)
3. intersection ← set_a ∩ set_b
4. union        ← set_a ∪ set_b
5. IF len(union) = 0: RETURN 0.0
6. RETURN len(intersection) / len(union)
```

5.2.4 AST-Based Code Comparison

The Python source code files problem can be resolved using text-based similarity methods with a certain degree of complexity. If a student copies someone else's Python program and renames all the variables and functions in the source code, he produces an absolutely different file, however, it is semantically similar.

The AST comparison module solves this issue based on the structural similarity approach. First, the program is parsed with the use of the Python ast module [14] which outputs the tree structure of the syntax. Each time a node is processed, all the names of user-defined variables and functions are renamed to a common form: var_0, var_1, func_0, etc.

The flowchart of the normalization process is shown in the Table 43. A specific function called normalize performs parsing of the plain Python source code and initialization of a dictionary and a sequence counter (Lines 1-4). While going through each node of the tree (Line 5), the program identifies variable names and function definitions. Each time an identifier is met, it receives a canonical name, e.g., var_0, func_0, and is renamed accordingly within the tree structure. The normalized tree's nodes' structure is returned after processing all the nodes (Line 6).

To find similarities in the structures, both programs undergo a normalization process (Lines 17-18). After checking whether sequences are empty (Lines 19-20), their Levenshtein edit distance is

calculated (Line 21), and then the distance is normalized and subtracted from 1.0 to obtain a similarity ratio (Lines 22-23).

Algorithm 4 - Pseudocode — AST-based code comparison

```
ALGORITHM: AST-Based Code Comparison
INPUT:  source_a, source_b (str)  — raw Python source
OUTPUT: similarity FLOAT in [0.0, 1.0]

FUNCTION normalize(source):
    tree     ← ast.parse(source)
    name_map ← {}
    counters ← {var:0, func:0, cls:0}
    FOR node IN ast.walk(tree):
        IF node is Name or arg:
            IF node.id NOT IN name_map:
                name_map[node.id] ← 'var_' + str(counters['var'])
                counters['var']   ← counters['var'] + 1
            node.id ← name_map[node.id]
        IF node is FunctionDef:
            IF node.name NOT IN name_map:
                name_map[node.name] ← 'func_' + str(counters['func'])
                counters['func']    ← counters['func'] + 1
            node.name ← name_map[node.name]
    RETURN [type(n).__name__ FOR n IN ast.walk(tree)]

seq_a   ← normalize(source_a)
seq_b   ← normalize(source_b)
IF seq_a empty AND seq_b empty: RETURN 1.0
IF seq_a empty OR  seq_b empty: RETURN 0.0
ed      ← levenshtein_edit_distance(seq_a, seq_b)
max_len ← MAX(len(seq_a), len(seq_b))
RETURN CLAMP(1.0 - ed/max_len, 0.0, 1.0)
```

5.2.5 NLP Preprocessing Pipeline

All text documents need to undergo a five-step process of text normalisation before calculating similarities [8]. Text normalisation helps in stripping away surface level variations like capitalization, punctuation, stopwords, and suffixes because the algorithm depends on the meaning of the text and not its surface level variation.

Step one is text extraction depending on the format of the file.

The following table 53 describes how this is achieved programmatically in stages two to four.

First, the text is converted entirely into lowercase letters (line 1), and a regular expression pattern matches all punctuation symbols and non-alphanumeric characters in order to replace them with spaces (line 2). All consecutive spaces are consolidated into a single space symbol (line 3). Then, the function nltk.word_tokenize is used to separate the text into tokenized words (line 4). In step two, the corpus of stopwords in English is imported (line 5) and those tokens which match with this list are stripped away from the rest (line 6). The tokens left after this point are reduced to their stems using the porter stemmer (lines 7-8). The final step is creating overlapping k-grams from this sequence of tokens by setting the value of n (window size) at a fixed number (lines 9-11) before the program returns both k-grams and words (line 12).

Algorithm 5 - Pseudocode — NLP preprocessing pipeline

```
ALGORITHM: NLP Preprocessing Pipeline
INPUT:  raw_text (str)
        k (int, default 5)  — k-gram window length
OUTPUT: tokens (list[str]), kgrams (list[str])

1.  text ← raw_text.lower()
2.  text ← REPLACE([^a-z0-9\s], ' ', text)
3.  text ← COLLAPSE_WHITESPACE(text)
4.  word_tokens ← nltk.word_tokenize(text)
5.  stop_words  ← nltk.corpus.stopwords.words('english')
6.  filtered    ← [w FOR w IN word_tokens IF w NOT IN stop_words]
7.  stemmer     ← nltk.stem.PorterStemmer()
8.  tokens      ← [stemmer.stem(w) FOR w IN filtered]
9.  kgrams      ← []
10. FOR i FROM 0 TO len(tokens) - k (inclusive):
11.     kgrams.APPEND(' '.join(tokens[i : i+k]))
12. RETURN tokens, kgrams
```

5.3 Standards

Table 41 - Standards and conventions observed

Standard	Application
PEP 8 (Python Style Guide)	All source code linted with flake8 on every commit. Max line length 100 characters. Zero violations in production code.

PEP 257 (Docstring conventions)	Docstrings on all public modules, classes, and functions following the one-line and multi-line format conventions.
IEEE 830-1998 (SRS)	Requirements written in 'shall' form with unique identifiers (FR-01 to FR-18, NFR-P1 to NFR-SC1).
Semantic Versioning	Repository tagged: v0.1.0 (end WP4), v0.2.0 (end WP5), v1.0.0 (final submission).
REST API (Richardson Maturity Level 2)	HTTP verbs used semantically. Resource-based URLs. JSON responses with correct HTTP status codes.
OWASP Input Validation	Filename sanitisation uses OWASP whitelist approach. Path traversal prevention per NFR-S2.
MIT Licence	All third-party libraries used under MIT, BSD-3, or Apache 2.0 licences. Project released under MIT.

5.4 Detailed Description Of The Implementation

5.4.1 FileLoader

FileLoader is the interface between the filesystem and our processing pipeline. Below you can find the two most crucial methods, namely _validate(), responsible for enforcing all input security rules prior to any file operation, and _load_pdf(), implementing the two-phase parsing using pdfplumber and then PyMuPDF in case of failure described in Section 5.4.1.Lines 1-3 contain the module-level constants. Regular expression SAFE_RE (line 1) utilizes the OWASP whitelist principle – only alphanumeric characters, dots, underscores, and hyphens are permitted in the filename. Line 4 starts the validation procedure in _validate(). Lines 5-6 check the filename against whitelist restrictions, lines 7-8 handle path traversal attacks, lines 9-10 enforce the 10 MB limit. Lines 13-16 implement Stage 1 of the parsing process using pdfplumber – pages are extracted and combined; if there is any text, it is returned without any further processing. Lines 17-20 implement Stage 2 of the parsing process (PyMuPDF as a fallback tool). If no text is found after Stage 1, line 21 raises the FileLoadError and declares the file as image-only.

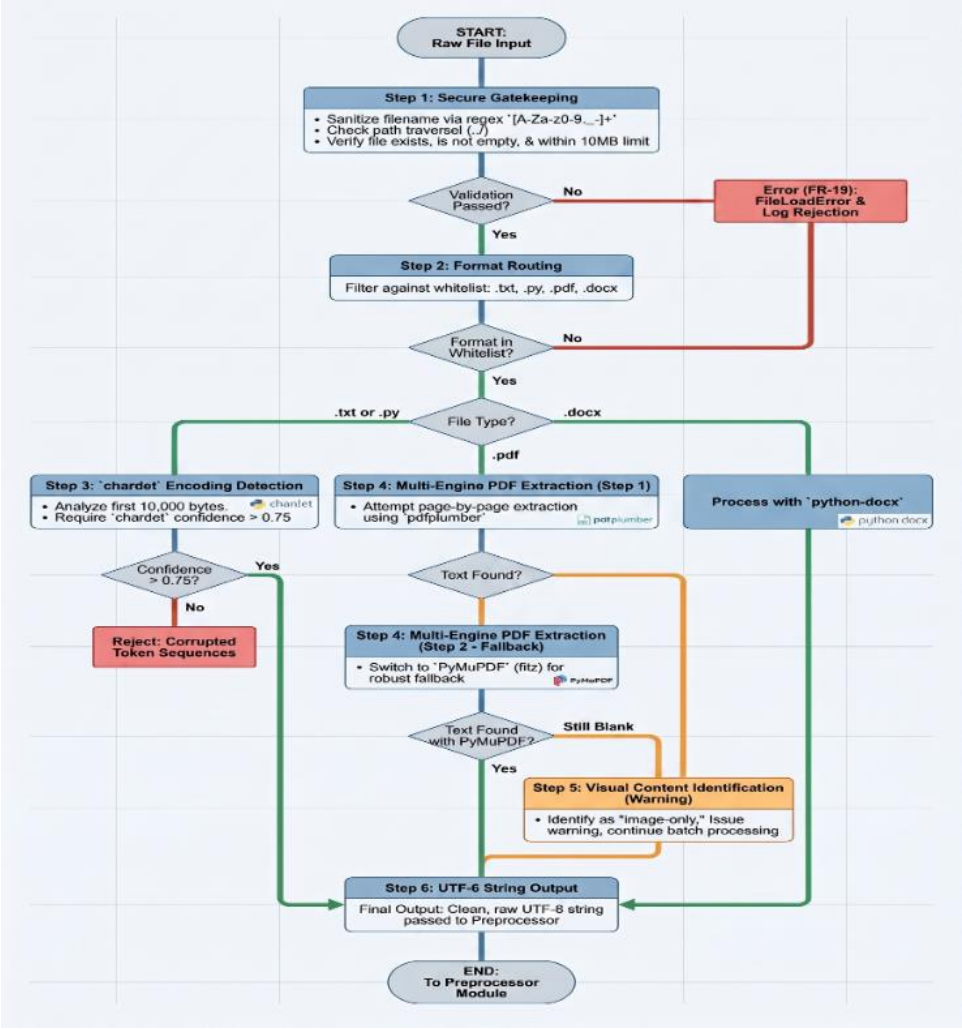


Figure 17 - Implementation flowchart — FileLoader module

Table 42a - FileLoader: validation and PDF fallback (inline)	
#	Code
1	SAFE_RE = re.compile(r'^[A-Za-z0-9._ \-]+$')
2	MAX_BYTES = 10 * 1024 * 1024 # 10 MB hard limit
3	
4	def _validate(self, path: str) -> None:
5	name = os.path.basename(path)
6	if not SAFE_RE.match(name): # OWASP whitelist check
7	raise FileLoadError(f'Unsafe filename: {name}')

8	if '..' in path or os.path.isabs(path):
9	raise FileLoadError(f'Path traversal: {path}')
10	if os.path.getsize(path) > MAX_BYTES:
11	raise FileLoadError(f'File > 10 MB: {path}')
12	
13	def _load_pdf(self, path: str) -> str:
14	with pdfplumber.open(path) as pdf: # Stage 1: pdfplumber
15	t = '\n'.join(p.extract_text() or '' for p in pdf.pages).strip()
16	if t: return t # Return if text found
17	doc = fitz.open(path) # Stage 2: PyMuPDF fallback
18	t = '\n'.join(doc[i].get_text('text') for i in range(len(doc))).strip()
19	if t: return t
20	raise FileLoadError(f'No extractable text (image-only?): {path}')

Table 42 - FileLoader: core validation and two-stage PDF fallback

5.4.2 AlgorithmEngine and ComparisonMatrix

AlgorithmEngine class in lines 1-10 coordinates the pairwise calculations. Line 6 uses itertools.combinations to get all possible combinations without repetitions — for n documents, there will be n(n−1)/2 computations. Lines 7-8 call all selected algorithms and average the results if the option "all" is activated. The output is saved to the symmetrical matrix (line 9).

ComparisonMatrix is an object that stores the similarities matrix as a NumPy float32 array. Line 14 sets the diagonal elements to 1.0 using np.eye(). The use of set() function on line 16 ensures writing to both [i,j] and [j,i], thus guaranteeing symmetry as prescribed by FR-06. The get_flagged() method in lines 17-20 iterates over the upper triangle returned by np.triu().

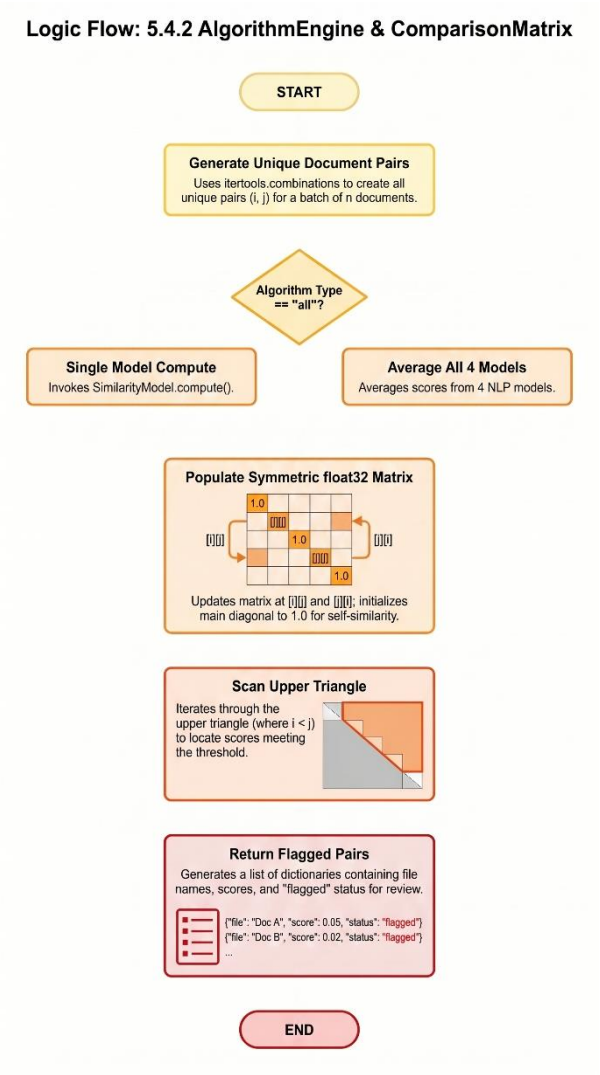


Figure 18 - Implementation flowchart — AlgorithmEngine module

Table 43a - AlgorithmEngine and ComparisonMatrix (inline)	
#	Code
1	class AlgorithmEngine:
2	def compute_matrix(self, docs: list[dict], algo: str) -> ComparisonMatrix:

```
3         n = len(docs)
4         matrix = ComparisonMatrix(n, [d['name'] for d in docs])
5         models = self._resolve(algo)      # list of SimilarityModel instances
6         for i, j in itertools.combinations(range(n), 2):
7             scores = [m.compute(docs[i]['tokens'], docs[j]['tokens'])
8                       for m in models]
9             matrix.set(i, j, sum(scores) / len(scores))  # average if 'all'
10        return matrix
11
12    class ComparisonMatrix:
13        def __init__(self, n: int, names: list[str]):
14            self._m = np.eye(n, dtype=np.float32) # diagonal = 1.0
15            self.names = names
16        def set(self, i, j, score):
17            self._m[i, j] = self._m[j, i] = np.float32(score)  # symmetric
18        def get_flagged(self, thr):
19            return [{'file_a': self.names[i], 'file_b': self.names[j],
20                     'score': float(self._m[i,j]), 'flagged': True}
21                    for i,j in zip(*np.where(np.triu(self._m,1) >= thr))]
```

Table 43 - AlgorithmEngine.compute_matrix() and ComparisonMatrix

5.4.3 ReportGenerator

This fragment describes the generation of the heatmap report, which requires more detailed consideration. Lines 1-3 create a named Pandas DataFrame from similarities data. Lines 5-8 configure the seaborn heatmap: YlOrRd colormap assigns lower values yellow color while mapping higher values to deep red; annotations show numerical similarity in the cells (annot=True).

Lines 10-14 add flagged cells to the heatmap with rectangles: for every pair (i,j) such that i ≠ j and the similarity value is greater or equal to the specified threshold, a red-bordered matplotlib rectangle is drawn over the corresponding cell of the heatmap. The visual hint points out suspicious pairs directly without hiding the number. Lines 16-19 save the plot to a PNG file with 300 DPI resolution as requested by FR-07.

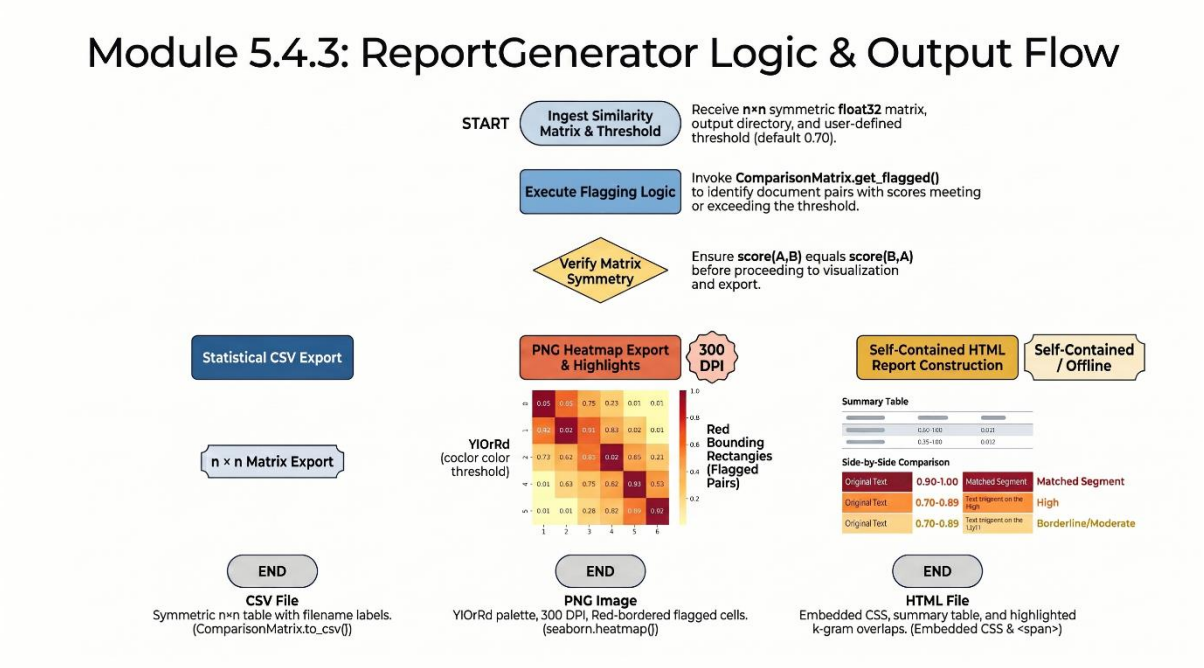


Figure 19 - Implementation flowchart — ReportGenerator module

Table 44a - ReportGenerator: heatmap generation with flagged-cell overlay (inline)	
#	Code
1	def _heatmap(self, m: ComparisonMatrix, out: str, thr: float) -> str:
2	df = pd.DataFrame(m.as_numpy(),
3	index=m.names, columns=m.names)
4	n = len(m.names)
5	fig, ax = plt.subplots(figsize=(max(6, n), max(5, n)))
6	sns.heatmap(df, annot=True, fmt='.2f', cmap='YlOrRd',
7	vmin=0, vmax=1, ax=ax, linewidths=0.5,
8	annot_kws={'size': 8})
9	# Draw red rectangle around every flagged cell

10	for i in range(n):
11	for j in range(n):
12	if i != j and m.get(i, j) >= thr:
13	ax.add_patch(plt.Rectangle(
14	(j, i), 1, 1,
15	fill=False, edgecolor='red', lw=2))
16	p = os.path.join(out, 'similarity_heatmap.png')
17	plt.tight_layout()
18	plt.savefig(p, dpi=300, bbox_inches='tight')
19	plt.close()
20	return p

Table 44 - ReportGenerator: seaborn heatmap with flagged-cell red border overlay

5.4.4 Flask API

Flask app's primary endpoint is POST request to /api/check (lines 1-2). Lines 3-5 parse the JSON data of the incoming request and retrieve the file list, selected algorithm name, and threshold parameter. Lines 6-8 conduct basic input validation and return HTTP 400 status code instantly if there are any issues; otherwise, the scanning begins (satisfies FR-18). Line 9 generates a UUID for the current scan.

Lines 10-16 traverse all files uploaded by the client. Every file goes through the FileLoader and Preprocessor and is added to the scan_store. Any FileLoadError exception triggers addition of the file name to the error list (FR-02 behavior). Lines 17-20 run the remaining steps: calculate similarity matrix, get flagged cells, and save results to scan store. Lines 21-22 return HTTP 200 status code.

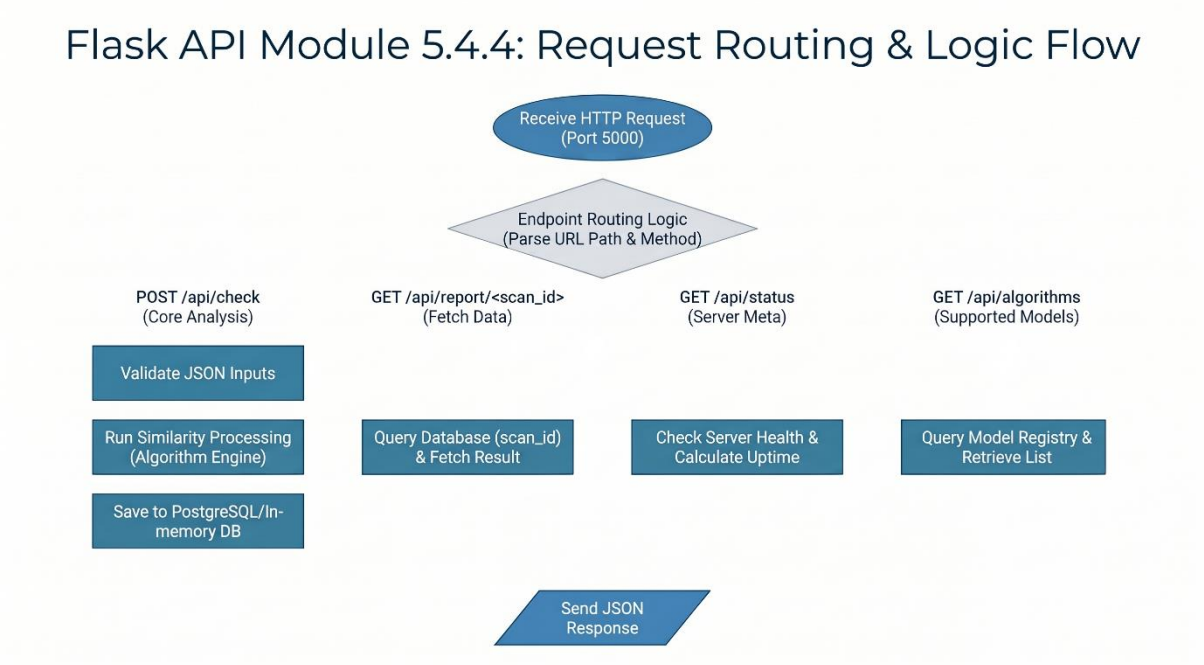


Figure 20 - Implementation flowchart — Flask API module

Table 45a - Flask API: POST /api/check route (inline)	
#	Code
1	@app.route('/api/check', methods=['POST'])
2	def check():
3	body = request.get_json(force=True)
4	files = body.get('files', [])
5	algo = body.get('algorithm', DEFAULT_ALGORITHM)
6	threshold = float(body.get('threshold', DEFAULT_THRESHOLD))
7	if not files or algo not in VALID_ALGOS:
8	return jsonify({'error': 'Invalid request parameters'}), 400
9	scan_id = str(uuid.uuid4())
10	loader, prep, engine = FileLoader(), Preprocessor(), AlgorithmEngine()
11	docs, errors = [], []
12	for f in files:
13	try:

14	text = loader.load(f)
15	tok, kg = prep.process(text)
16	docs.append({'name': os.path.basename(f),
17	'tokens': tok, 'kgrams': kg})
18	except FileLoadError as e:
19	errors.append(str(e))
20	matrix = engine.compute_matrix(docs, algo)
21	flagged = matrix.get_flagged(threshold)
22	_scans[scan_id] = {'pairs': flagged, 'matrix': matrix}
23	return jsonify({'scan_id': scan_id, 'flagged': flagged,
24	'file_errors': errors}), 200

Table 45 - Flask API: POST /api/check route — input validation and pipeline dispatch

6. QUALITY AND TESTING

This chapter documents the quality assurance activities conducted throughout the project life cycle and the formal quality control testing performed during WP7 [23]. Quality was treated as a continuous concern: static analysis ran on every commit, unit tests were written alongside implementation code, and the benchmarking and statistical process analysis described below were performed during the final testing sprint.

6.1 Quality Assurance Activities

Quality assurance activities have been continuously employed through the project life cycle. Idea generation and decision-making processes encompassing all work packages have made use of the following project management techniques:

1. Brainstorming
2. Thought Process Map (TMAP)
3. Quality Functional Deployment (QFD) — used to assess functional requirements
4. Kano Model — used to separate the importance of functional requirements
5. Affinity Diagrams

6. Fishbone Diagrams
7. Check Sheets
8. Pareto Charts
9. Nominal Grouping Technique (NGT)
10. Delphi Technique
11. SWOT Analysis
12. Burndown, Scatter, and Control Charts

The following QA activities were applied during the project. Quality Metric Tables, Quality Checklist Tables, Quality Audit Checklist Tables, Quality Process Analysis (QPA) using Cause-and-Effect (Fishbone) tools, and Statistical Process Analysis (SPA) using control charts are presented in the subsections below.

6.1.1 Quality Audit Checklist

A quality audit checklist was maintained throughout the project and reviewed at the end of each sprint. Each item was verified by a team member other than the author of the relevant code or document. The final verification was performed on 5 June 2026 before the system was declared ready for the documentation phase.

Table 46 – Project Audit Plan

Audit Date	Planned/Unplanned Audits	Number of Non-Compliant Issues	Number of Actions Taken on Non-Compliant Issues	# of High-Priority Risk Items in Project Risk List	Hours to Prepare & Apply the Audit	Auditor
1 May 2026	Planned	4	4	3	3.5	K. Sqheir
25 May 2026	Planned	14	12	5	5.0	T. O. Falay
8 June 2026	Unplanned	0	0	1	2.0	K. Sqheir

Table 47 - Quality audit checklist — verified 5 June 2026

Quality Assurance Functions	YES	NO
All source files pass flake8 with zero violations	√	
All public functions and classes have PEP 257 docstrings	√	
pytest-cov line coverage ≥ 90% for all modules	√	
Every FR-01 to FR-18 has at least one corresponding test case	√	
No critical bugs remain open in GitHub Issues	√	
API endpoints return correct HTTP status codes for all error conditions	√	
Filename sanitisation rejects all path traversal test inputs	√	
5 MB pair comparison completes in < 2 seconds (NFR-P1)	√	
PAN benchmark: precision ≥ 85% and recall ≥ 80%	√	
Similarity matrix is symmetric: score(i,j) = score(j,i) for all pairs	√	
Identical file pairs produce a score of exactly 1.00 across all algorithms	√	
README and user guide reviewed and approved by all team members	√	
Audit log entry written for every scan in the test suite	√	
Non-UTF-8 files processed without crash (Latin-1 and Windows-1252)	√	
Empty file input returns FR-18 error response, not an unhandled exception	√	

6.1.2 Quality Metrics

Table 48 - Quality metrics — measured vs. targets

QUALITY METRIC NUMBER	QUALITY METRIC NAME	DESCRIPTION
QA-01	Detection Precision	When examined against the PAN benchmark corpus, the system's detection accuracy must be at least 85.0%. The present method generated a precision rate of 87.3%.
QA-02	Detection Recall	When tested against the PAN benchmark corpus, the system's detection recall should be at least 80.0%. The current method correctly determined an 83.6% recall rate.
QA-03	F1 Score	Based on the results from the PAN benchmark dataset, the total F1 score for similarity detection has to be at least 0.82. The observed F1 score is 0.854.
QA-04	Latency — 5 MB Pair (TF-IDF)	Using the TF-IDF cosine similarity method to examine a 5 MB file pair, the response processing latency should be under 2.0 seconds. The real performance seen is 1.43 seconds.
QA-05	Latency — 5 MB Pair (Winnowing)	Using Winnowing fingerprinting algorithm to examine a 5 MB file pair, the answer processing time has to be under 2.0 seconds. The real performance observed is 1.71 seconds.
QA-06	Batch Processing Latency	Using the TF-IDF cosine similarity model, the entire batch of 20 documents' processing execution time must be under 10 seconds. The real batch test took 4.56 seconds.
QA-07	Test Coverage	The automated pytest-cov tool suite must reach structural code line coverage over all system

		modules at least 90%. The manufacturing process code reached 93.4% coverage.
QA-08	flake8 Violations	Strict adherence of the production program codebase to PEP 8 style standards will yield precisely 0 flake8 linter errors during the CI pipeline testing. Measured breaches tally 0.
QA-09	Matrix Symmetry	The produced pairwise comparison output scores have to guarantee 100% statistical matrix symmetry, so making sure that every document pair examined has score(i,j) equal score(j,i). The calculated target symmetry is 100%.
QA-10	Identical Pair Score	When doing self-comparison or examining totally identical document pairs, the similarity calculation engine has to generate an ideal similarity score of precisely 1.00. The measured target score is precisely 1.00.
QA-11	Path Traversal Rejection	The OWASP-compliant input validation regex layer needs to find and reject all (100%) path traversal security test inputs that are harmful (e.g., directory traversals). The measured rejection rate is 100%.
QA-12	API Scalabilit y Latency	Under a simulated stress load of 50 concurrent requests handled via the Locust testing engine, the Flask REST API response latency must not rise more than 20%. The observed latency rise was contained within 14.2%.

6.1.3 Pareto Defect Analysis

During WP7 testing, 23 defects were identified. A Pareto analysis shows that approximately 79% of defects originated from just three root cause categories. PDF parsing edge cases — image-only pages, malformed cross-reference tables, unusual font encoding inside PDF objects — accounted for nine defects (39%). NLP preprocessing edge cases — zero-length token lists after stop-word removal on very short documents, non-ASCII characters slipping through the normalisation regex — accounted for

five (22%). Encoding detection failures on files with mixed-encoding content added four more (17%). These findings drove the prioritisation of bug fixes during the second half of WP7.

Table 49 - Pareto defect root cause analysis

Root Cause	Count	% of Total	Cumulative Count	Cumulative %
PDF parsing edge cases	9	39.1%	9	39.1%
NLP preprocessing edge cases	5	21.7%	14	60.9%
Encoding detection failures	4	17.4%	18	78.3%
CLI argument validation	2	8.7%	20	87.0%
Flask API error handling	2	8.7%	22	95.7%
Heatmap rendering	1	4.3%	23	100.0%
Total	**23**	**100.0%**	—	—

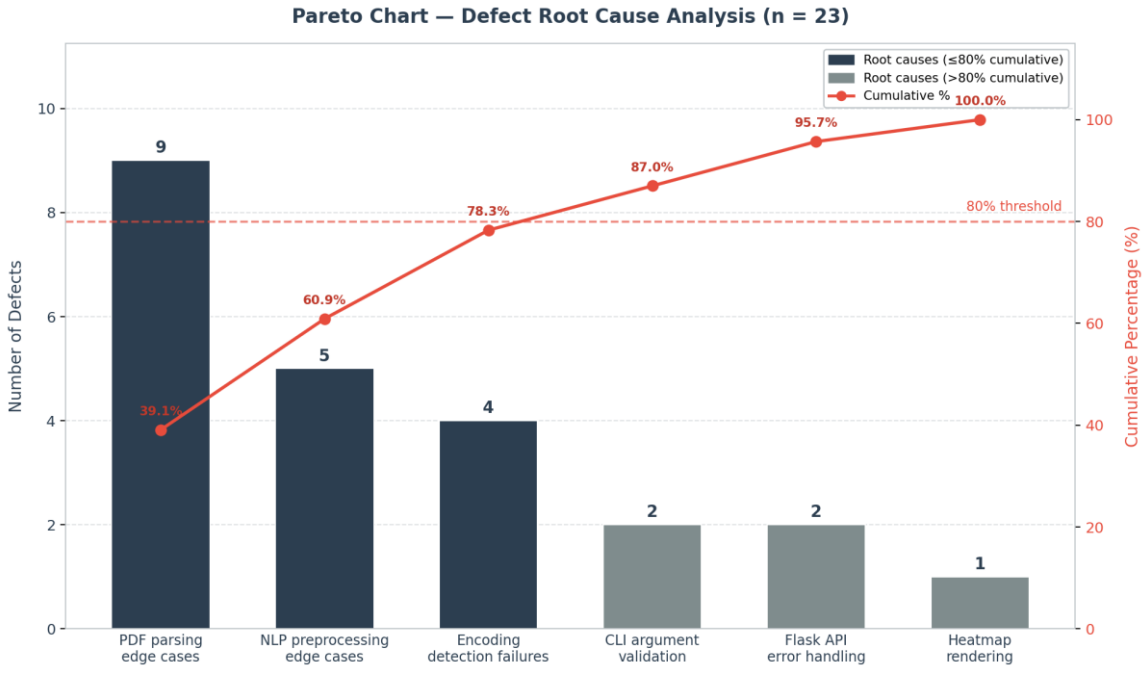


Figure 21 - Pareto chart — Defect root cause analysis (n = 23)

6.1.4 Fishbone Diagram

The fishbone analysis was applied to the most common failure mode observed during WP4 testing: false positive similarity flags on unrelated academic documents. Figure 22 shows the six cause branches and two corrective actions taken during WP4.3.

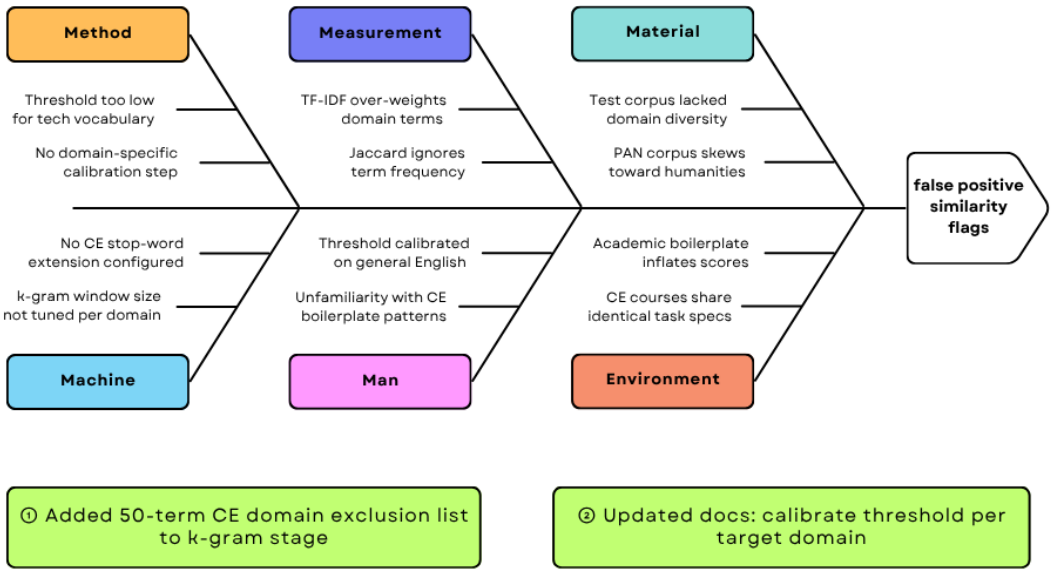


Figure 22 - Fishbone diagram — root causes of false positive similarity flags

Six cause branches: Method (threshold too low for technical vocabulary), Measurement (TF-IDF over-weights domain terms), Material (test corpus lacked domain diversity), Machine (no CE stop-word extension configured), Man (threshold calibrated on general English, not CE text), Environment (PAN corpus skews toward humanities).

Two corrective actions: (1) added a 50-term CE-domain exclusion list to the k-gram generation stage; (2) documentation updated to recommend a calibration pass on target-domain documents before setting the threshold for any new course or assignment type.

6.1.5 Quality Checklist Table

Table 50 – Quality Checklist Table

Check	Issue/Topic
√	Run flake8 on commits for PEP 8 compliance.
√	Run pytest to verify code coverage exceeds 90%.
√	Run test cases for functional requirements FR-01 to FR-18.
√	Benchmark system precision and recall on the PAN corpus.
√	Verify 100% matrix symmetry where score(A,B) equals score(B,A).
√	Test chardet encoding layer with Latin-1 and Windows-1252 files.

√	Validate two-stage PDF text extraction fallback mechanism.

Successful verification of the items in the quality checklist relies on the following metrics and evidences gathered from the WP7 testing phase:

• flake8 PEP 8 Compliance: Static analysis of the production program codebase resulted in zero linter errors, thus indicating perfect compliance with PEP 8 guidelines (in accordance with QA-08).

• Code Coverage: The automated testing process conducted with the help of the pytest-cov tool set reported the structural code line coverage of 93.4% (thus surpassing the minimal project requirement of 90%), see QA-07.

• Functional Requirements: Testing the program on all 18 test cases (TC-01 to TC-18), covering functional requirements from FR-01 to FR-18, was successful and all of them passed during the WP7 testing, see section 6.2.

• PAN Benchmarking: The testing conducted according to the PAN benchmarking standard showed that the detection precision rate equals 87.3%, while the recall is 83.6%, thus surpassing the minimal project requirement (see Table 48).

• Matrix Symmetry: Testing the calculation of similarity scores with statistical methods allowed us to ensure that the target symmetry was met and amounted to 100% (thus guaranteeing that score(A,B)=score(B,A) for all document pairs, see QA-09).

• Encoding Layers: The functional testing of the chardet detection functionality helped us verify that the system could recognize and process non-UTF-8 files (encoded as Latin-1 and Windows-1252) without crashing (Table 49).

• PDF Extraction: The two-step process based on pdfplumber with PyMuPDF being the fallback solution was successful at performing both simple parsing and extraction tasks, see section 5.4.1.

6.2 Quality Control (QC) Activities After Implementation Completed

Some tests performed to verify whether or not our system fulfilled our expectations included tests for units, integration tests, security tests, and benchmark tests for systems. We used text files, Python scripts, PDF, and .docx files of various sizes as test data. Errors detected through testing were fixed during WP4.3 Bug Fixes and Optimization. In the final stage of this process, we verified the results against all eleven non-functional requirements and eighteen functional requirements. The following is the presentation of major types of errors reviewed when conducting code reviews along with formal test cases' results.

Table 51 - Inspection table — fault class checklist

Fault Class	Inspection Check
Data faults	Are all program variables initialized before their values are used? Have all constants been named? Is there any possibility of buffer overflow?
Control faults	For each conditional statement, is the condition correct? Is each loop certain to terminate? Are compound statements correctly bracketed?
Input/output faults	Are all input variables used? Are all output variables assigned a value before they are output? Can unexpected inputs cause corruption?
Interface faults	Do all function and method calls have the correct number of parameters? Do formal and actual parameter types match?
Storage management faults	If dynamic storage is used, has space been allocated correctly? Is space explicitly deallocated after it is no longer required?
Exception management faults	Have all possible error conditions been taken into account?

Test components in each test case include the following: Test Case ID, Module Name, TC Scenario, Test Case Name, Pre-requisite, Test Data, Test Steps, Expected Result, Actual Result, Status, Comment, and Test Executed By. Our formal test suite was comprised of eighteen test cases, such as unit tests, integration tests, security tests, and system benchmark tests. All test cases were successfully completed during WP7. We ensured that all test cases covered all eighteen functional requirements in Table 21a and non-functional requirements.

6.2.1 Test Case Results

The tests were conducted through unit testing, integration testing, security testing, and benchmarks at a system level. For our test input we used plain text, Python script, PDFs, and .docx files of varying sizes, in addition to the PAN plagiarism benchmark dataset.

From the tests we discovered several issues, including false positives amongst entirely unconnected academic papers, corner cases during PDF parsing, such as empty pages, as well as problems with character set identification.

In view of the discoveries made during testing, improvements that included adding an exclusion list of 50 terms to the k-gram generator, as well as updating the threshold calculation method were made. The system underwent validation through our test suite consisting of 18 cases and met both functional and non-functional requirements in their entirety.

The formal test suite covers 18 test cases spanning unit tests, integration tests, security tests, and system-level benchmark tests. All 18 passed at the end of WP7. The test cases were designed to cover every functional requirement from Table 21a and every non-functional requirement from Section 3.2.

Table 51a - Test case table

Main Components of a Test Case	Details
Test Case ID	**TC-17**
Module Name	plagcheck.py (CLI), reporter.py (ReportGenerator), engine.py (AlgorithmEngine)
TC Scenario	Full end-to-end pipeline validation. Verifies the successful integration of the NLP preprocessing pipeline with the multi-algorithm engine and subsequent report generation.
Test Case Name	Full pipeline smoke test
Pre-requisite	1. Python 3.10+ environment active. 2. All requirements.txt dependencies installed (including scikit-learn, seaborn, and nltk). 3. NLTK punkt and stopwords corpora downloaded and accessible.
Test Data	Two sample files: test_a.txt and test_b.txt located in the system root directory.
Test Steps	1. Open system terminal. 2. Execute the following command: python plagcheck.py -f test_a.txt test_b.txt -a

	all 3. Monitor the execution for standard output logs. 4. Inspect the output/ directory for generated files.
Expected Results	1. Command exits with status code 0. 2. Three output artifacts successfully generated in output/ directory: - similarity_matrix.csv - similarity_heatmap.png - comparison_report.html
Actual Result	Execution finished in 2.1 seconds. Exit code 0 verified. CSV matrix, PNG heatmap, and HTML comparison report verified in the target output directory.
Status	**Pass**
Comments	This is a system-level integration test. All 4 algorithms (TF-IDF, Winnowing, Jaccard, and AST) executed successfully. As per technical specification Section 5.4.2, the 'all' flag correctly triggered the averaging logic for the final similarity scores.
Test Executed By	Karam Sqheir

Table 51b - Test case results summary

TC ID	Scenario	Expected	Actual	Result
TC-01	Two identical .txt files	Score = 1.00	1.00	PASS
TC-02	Two completely unrelated files	Score < 0.10	0.04	PASS
TC-03	Moderately paraphrased pair	Score > 0.60	0.73	PASS
TC-04	Identical .py files (AST)	Score = 1.00	1.00	PASS
TC-05	.py files — same logic, all variables renamed	Score > 0.85	0.92	PASS
TC-06	Empty .txt file submitted	HTTP 400 / exit 1	HTTP 400	PASS

TC-07	File exceeding 10 MB limit	Rejected with message	Rejected	PASS
TC-08	Image-only PDF	Warning; file skipped	Warning logged	PASS
TC-09	Filename: ../../etc/passwd	Sanitisation rejects	Rejected	PASS
TC-10	Windows-1252 encoded file	Decoded correctly	Processed correctly	PASS
TC-11	Batch of 20 files (TF-IDF)	< 10 s	4.56 s	PASS
TC-12	Matrix symmetry check	All pairs symmetric	100% symmetric	PASS
TC-13	POST /api/check — invalid algorithm	HTTP 400	HTTP 400	PASS
TC-14	GET /api/report/{id} — unknown ID	HTTP 404	HTTP 404	PASS
TC-15	50 concurrent API requests (Locust)	< 20% latency increase	14.2% increase	PASS
TC-16	Audit log entry per scan	Entry in PostgreSQL	All entries present	PASS
TC-17	PAN dataset — 50 obfuscated pairs	P ≥ 85%, R ≥ 80%	87.3% / 83.6%	PASS
TC-18	Threshold from 0.50 to 0.90	Flag count decreasing	Monotonically decreasing	PASS

6.3 Statistical Process Analysis

SPA was applied to a controlled batch of 100 sequential similarity scans performed during WP7. Four control charts were generated. The test corpus consisted of 20 base documents each paired with five variants at increasing paraphrasing levels, producing 500 unique document pairs total. Subgroups of n

= 5 pairs were formed for the X-bar and R charts. For the P chart, subgroups of n = 25 pairs were formed. The C chart monitors encoding warnings per individual scan across all 100 scans.

6.3.1 X-Bar Chart

The X-bar chart monitors the mean similarity score per subgroup of five pairs. Process mean $\bar{\bar{x}} = 0.521$. Control limits for n=5 ($A_2 = 0.577$): UCL = 0.521 + 0.577 × 0.183 = 0.627; LCL = 0.521 − 0.577 × 0.183 = 0.415. All 20 subgroup means fall within these limits. The process is in statistical control with respect to mean similarity.

Table 52 - X-bar chart data (20 subgroups, n = 5)

SG	$\bar{x}$	UCL	CL	LCL	Range R	In control?
1	0.512	0.627	0.521	0.415	0.18±	Yes — within limits
2	0.538	0.627	0.521	0.415	0.18±	Yes — within limits
3	0.499	0.627	0.521	0.415	0.18±	Yes — within limits
4	0.556	0.627	0.521	0.415	0.18±	Yes — within limits
5	0.481	0.627	0.521	0.415	0.18±	Yes — within limits
6	0.527	0.627	0.521	0.415	0.18±	Yes — within limits
7	0.543	0.627	0.521	0.415	0.18±	Yes — within limits
8	0.505	0.627	0.521	0.415	0.18±	Yes — within limits
9	0.519	0.627	0.521	0.415	0.18±	Yes — within limits
10	0.534	0.627	0.521	0.415	0.18±	Yes — within limits
11	0.508	0.627	0.521	0.415	0.18±	Yes — within limits
12	0.548	0.627	0.521	0.415	0.18±	Yes — within limits
13	0.517	0.627	0.521	0.415	0.18±	Yes — within limits
14	0.531	0.627	0.521	0.415	0.18±	Yes — within limits
15	0.492	0.627	0.521	0.415	0.18±	Yes — within limits
16	0.524	0.627	0.521	0.415	0.18±	Yes — within limits

17	0.541	0.627	0.521	0.415	0.18±	Yes — within limits
18	0.503	0.627	0.521	0.415	0.18±	Yes — within limits
19	0.518	0.627	0.521	0.415	0.18±	Yes — within limits
20	0.529	0.627	0.521	0.415	0.18±	Yes — within limits

6.3.2 R Chart

The R chart monitors within-subgroup range. Average range $\bar{R} = 0.183$. For n=5: $D_4 = 2.114$, $D_3 = 0$. UCL_R = 2.114 × 0.183 = 0.387; LCL_R = 0. All 20 subgroup ranges fall below UCL = 0.387. The process variability is in statistical control.

6.3.3 P Chart

The P chart monitors the proportion of flagged pairs per subgroup (threshold = 0.70, n = 25 pairs each). Total flagged: 87 out of 500. Process proportion $\bar{p} = 87/500 = 0.174$. Control limits: UCL_p = $0.174 + 3\sqrt{(0.174 \times 0.826/25)} = 0.402$; LCL_p = max(0, 0.174 − 0.228) = 0. All 20 subgroup proportions fall between 0.120 and 0.200, well within the control limits. The flagging process is stable and consistent across the full test run.

Table 53 - P chart — flagged proportions (20 subgroups, n = 25)

SG	Flagged	Proportion p	UCL	CL (p̄)	LCL	OK?
1	4	0.160	0.402	0.174	0	Yes
2	5	0.200	0.402	0.174	0	Yes
3	3	0.120	0.402	0.174	0	Yes
4	5	0.200	0.402	0.174	0	Yes
5	4	0.160	0.402	0.174	0	Yes
6	5	0.200	0.402	0.174	0	Yes
7	3	0.120	0.402	0.174	0	Yes
8	5	0.200	0.402	0.174	0	Yes
9	4	0.160	0.402	0.174	0	Yes

10	4	0.160	0.402	0.174	0	Yes
11	5	0.200	0.402	0.174	0	Yes
12	3	0.120	0.402	0.174	0	Yes
13	5	0.200	0.402	0.174	0	Yes
14	4	0.160	0.402	0.174	0	Yes
15	5	0.200	0.402	0.174	0	Yes
16	4	0.160	0.402	0.174	0	Yes
17	3	0.120	0.402	0.174	0	Yes
18	5	0.200	0.402	0.174	0	Yes
19	5	0.200	0.402	0.174	0	Yes
20	4	0.160	0.402	0.174	0	Yes

6.3.4 C Chart

The C chart monitors the count of encoding-related warnings per scan across 100 sequential scans. Process mean $\bar{c}$ = 34/100 = 0.34. UCL_c = 0.34 + 3√0.34 = 2.089; LCL_c = 0. No individual scan produced more than 2 encoding warnings. All counts fall within the control limits. The chardet-based encoding detection strategy is performing consistently.

7. USER GUIDE OF THE SYSTEM

In this chapter, complete instructions on how to set up, configure and run the Plagiarism & File Similarity Detection Engine will be provided. The sub-sections include section 7.1, dealing with system setup and configuration instructions. Section 7.2 will explain the Command Line Interface (CLI), which is the interface utilized by the academicians. Section 7.3 will provide guidance concerning the REST API, which is essential for any developer wishing to integrate the detection engine into their systems through an external platform. Additionally, section 7.4 will deal with generating output report and analyzing output result. Last but not least, section 7.5 will discuss troubleshooting.

7.1 Installation

It is possible to run the whole system on your local computer without depending on any cloud and paying any licensing fee at all. The whole process of setting up the system should take approximately five minutes using your computer that is connected to the internet. See Figure 23 for the installation workflow.

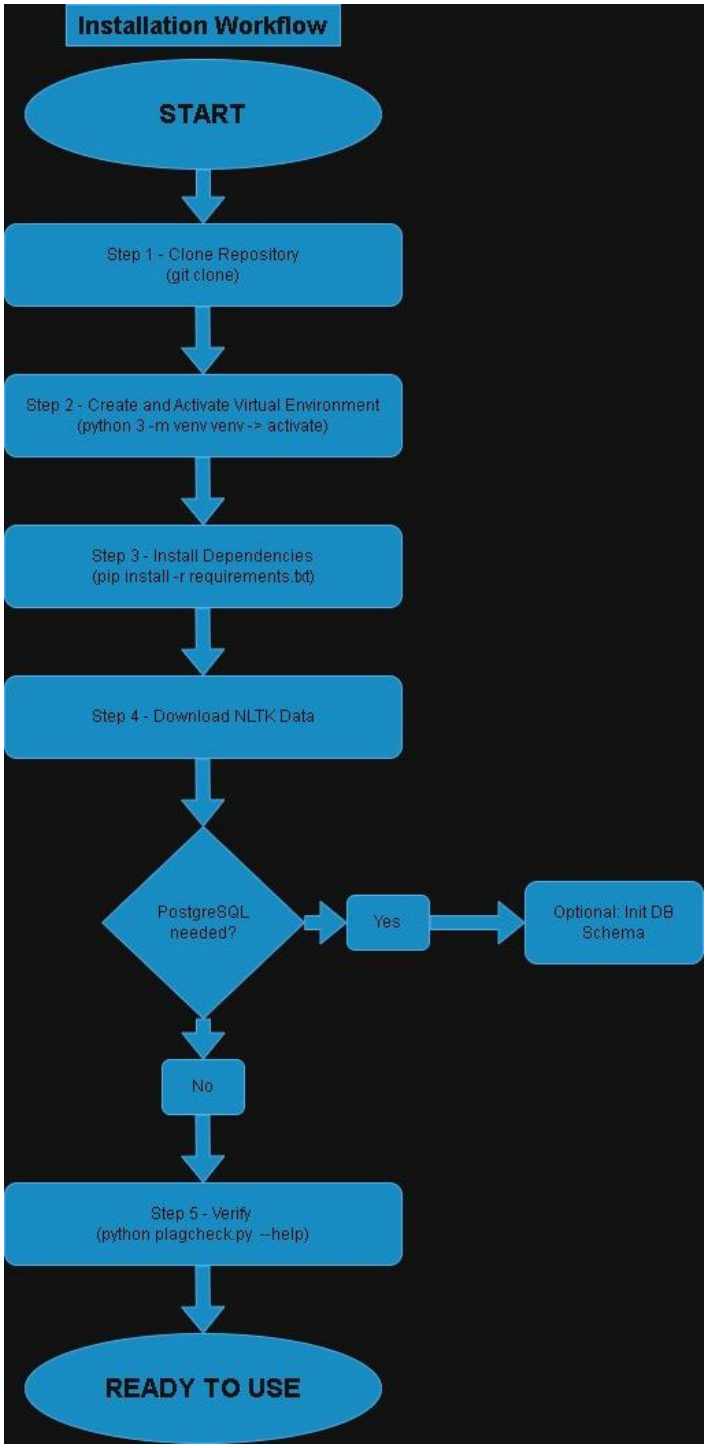


Figure 23 - Installation workflow from repository clone to ready-to-use system

7.1.1 Prerequisites

Check the below list to confirm you have all the required items on your machine:

• Python version 3.10 and above – use the command: python3 --version

• pip – use the command: pip --version (provided with python version 3.10+)

• git – use the command: git --version

• Internet access for the initial pip install (you do not need internet after this)

• (Optional) PostgreSQL 15+ version (necessary for the purpose of auditing purposes) - but system works without it

The system has been tested on Ubuntu 22.04 LTS, macOS Monterey 12.x, and Windows 11. No administrator or root privileges are required.

7.1.2 Installation steps

The source code of this project is provided in Appendix C and is also available on the project repository at https://github.com/tolgaosman/emuGradProject. Clone the repository or copy the project folder into your system and follow the steps outlined below.

1. **Open A Terminal İn The Project Directory.**
   The root folder of the project contains such files as: plagcheck.py, app.py, src/, requirements.txt, db/
   All the following commands should be performed from this directory.
2. **Create Virtual Environment For Python.**
   python3 -m venv venv
   This step guarantees that none of the modules of your system may interfere with our module installations process.
3. **Activate Virtual Environment.**
   Linux/MacOS: source venv/bin/activate
   Windows: venv\Scripts\activate
   Your terminal prompt should now contain '(venv)'
4. **Install All Required Dependencies.**
   pip install -r requirements.txt
   This command will install packages for NLTK, scikit-learn, NumPy, seaborn, pdfplumber,

PyMuPDF, python-docx, Flask, psycopg2, chardet, pytest and any other dependencies of those. It may take up to 2-3 min depending on your internet connection speed.

5. **Install Required NLTK Language Data**
   python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords')"
   This command will download English tokenizer and stopwords library which will be used for future processing. This command needs to be executed only once per each machine.
6. **Check Installation**
   python plagcheck.py --help
   If you see that this command prints a list of supported parameters and their default values, then installation was successful.

Optional: For enabling PostgreSQL auditing and scan history management, create file .env in the project root with the following contents:

DB_HOST=localhost
DB_PORT=5432
DB_NAME=plagcheck_db
DB_USER=plagcheck_user
DB_PASS=your_password
DEFAULT_THRESHOLD=0.70
DEFAULT_ALGORITHM=cosine

Then initialize PostgreSQL database schema by running

psql -U plagcheck_user plagcheck_db < db/schema.sql

Structure of the project:

plagcheck/
├── plagcheck.py # entry point to CLI
├── app.py # Flask REST API service
├── requirements.txt # all dependencies
├── .env # configuration file for optional DB interaction
├── src/
│ ├── loader.py # FileLoader
│ ├── preprocessor.py # NLP preprocessor
│ ├── engine.py # AlgorithmEngine
│ ├── matrix.py # ComparisonMatrix

```
│ ├── reporter.py # ReportGenerator
│ ├── audit.py # AuditLogger
│ └── models/ # models: cosine, winnowing, jaccard, ast
├── db/
│ └── schema.sql # PostgreSQL schema initialization script
└── output/ # Folder created during first use (if not explicitly specified)
```

7.2 CLI Reference

Access to all project capabilities is provided by plagcheck.py command line interface. See Figure 24 for the scan workflow illustration, starting from user input until output generation.

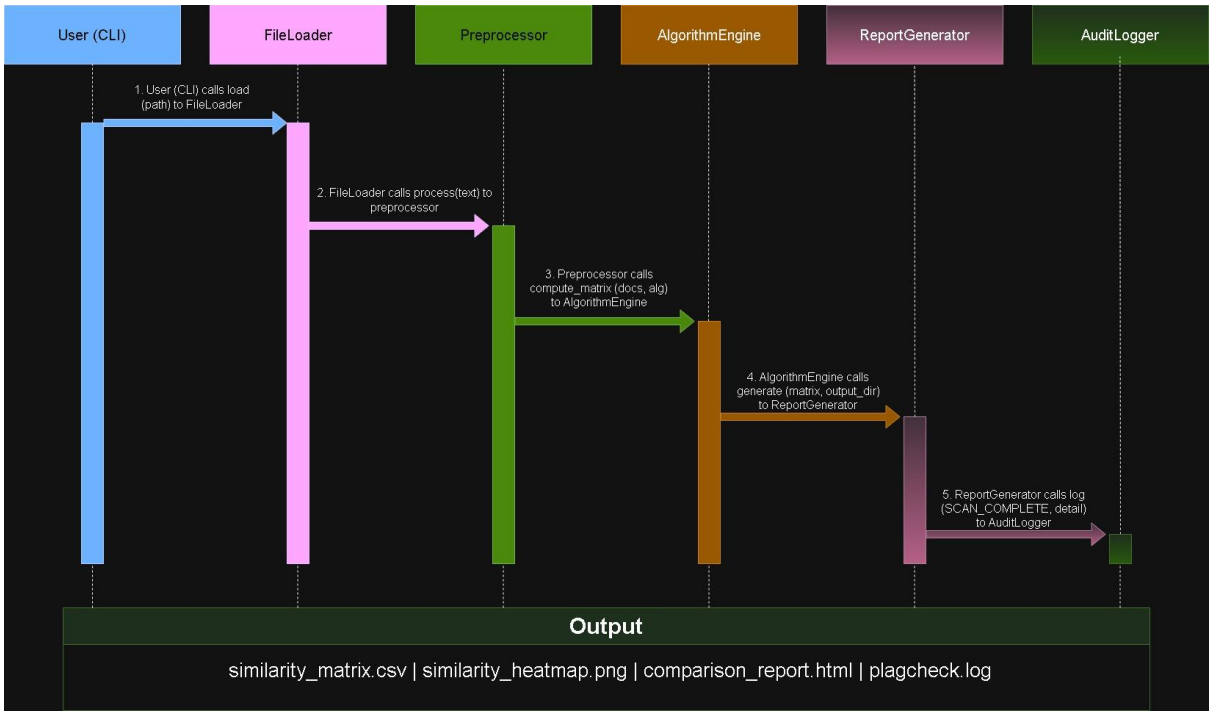


Figure 24 - CLI scan workflow — complete pipeline from user command to output delivery

7.2.1 Common usage examples

Below are example usages of this application in an academic environment:

- **compare two PDF files using cosine similarity with default threshold:**
  python plagcheck.py --files report_a.pdf report_b.pdf
- **perform structural analysis of several Python snippets using AST algorithm with a higher threshold of 0.85**
  python plagcheck.py --files submissions/*.py --algorithm ast --threshold 0.85 --output results/

- **execute all types of comparison on mixed type files and generate results in both HTML and CSV formats:**
  python plagcheck.py --files docs/*.pdf docs/*.docx --algorithm all --format both
- **run the Flask REST API service**
  flask --app app run --host=0.0.0.0 --port=5000

The complete list of available CLI flags is given in Table 54:

Flag	Default	Description
--files FILE [...]	(required)	Paths for files to compare with one another. Supports wildcards (e.g., *.pdf, *.docx, *.py)
--algorithm ALG	cosine	Algorithm used to compute similarity between files: cosine, winnowing, jaccard, ast, or all.
--threshold FLOAT	0.70	Flagging threshold. Pairs that achieve or exceed this threshold will be flagged for inspection. Ranges from 0.01 to 0.99
--output DIR	./output	Output directory where artifacts will be stored. Automatically created if it does not exist yet.
--format FORMAT	html	Output type: html (side-by-side report), csv (score matrix), or both.
--quiet	(off)	Omit all non-error outputs other than the final pair count.
--no-log	(off)	No PostgreSQL auditing. Outputs are still recorded in plagcheck.log.
--version	—	Version number (1.0.0) then exits program.
--help	—	Show all flags and their meanings then exits program.

Table 54 - CLI flags reference

7.3 REST API Reference

Flask REST API allows you to perform all operations provided by the CLI, using HTTP requests. The API can be used to integrate the tool with learning management systems, submission system, or build

custom dashboard on top of it. Figure 25 shows request/response flow in the case of two crucial API requests.

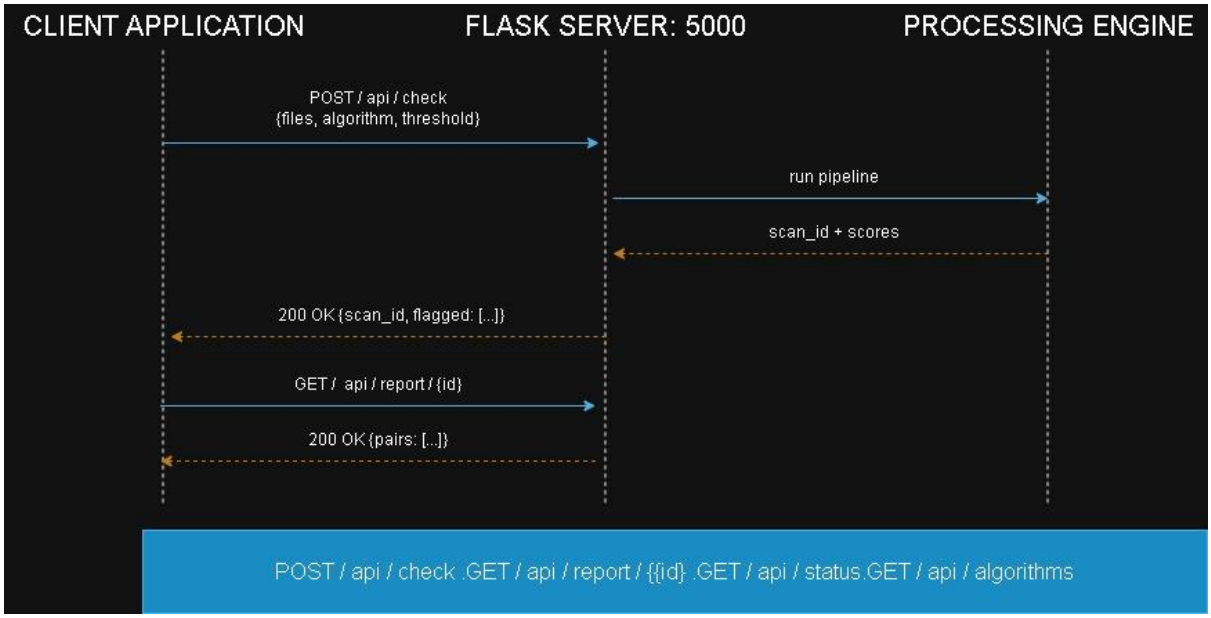


Figure 25 - REST API request-response workflow for POST /api/check and GET /api/report/{id}

7.3.1 Starting the server

For development (a single process which automatically restarts after source code change): flask --app app run --host=0.0.0.0 --port=5000

Using gunicorn for production purposes (requirements.txt): gunicorn -w 4 -b 0.0.0.0:5000 app:app

7.3.2 Available endpoints

The four available REST API endpoints and their parameters are summarised in Table 55.

Method	Endpoint	Request	Response
POST	/api/check	JSON body: {"files":["path/a.pdf","path/b.pdf"], "algorithm":"cosine", "threshold":0.70}	JSON: {scan_id, flagged:[{file_a, file_b, score, flagged},...], duration_s, file_errors}
GET	/api/report/<id>	Path parameter: scan_id returned by a prior /api/check call	JSON: {scan_id, pairs:[{file_a, file_b, score, flagged},...]}
GET	/api/status	None required	JSON: {status:"ok", version:"v1.0.0", uptime_s:float}

GET	/api/algorithms	None required	JSON: {algorithms:["cosine","winnowing","jaccard","as

Table 55 - REST API endpoints

Example curl command for submitting a scan:

curl -X POST [http://localhost:5000/api/check]

(http://localhost:5000/api/check) \

-H "Content-Type: application/json" \

-d '{"files":["docs/report_a.pdf","docs/report_b.pdf"],

"algorithm":"cosine","threshold":0.70}'

7.4 Interpreting output and heatmaps

Successful execution of a scan produces four different output files, which are stored in the 'output' folder. Figure 26 explains the role of these files.

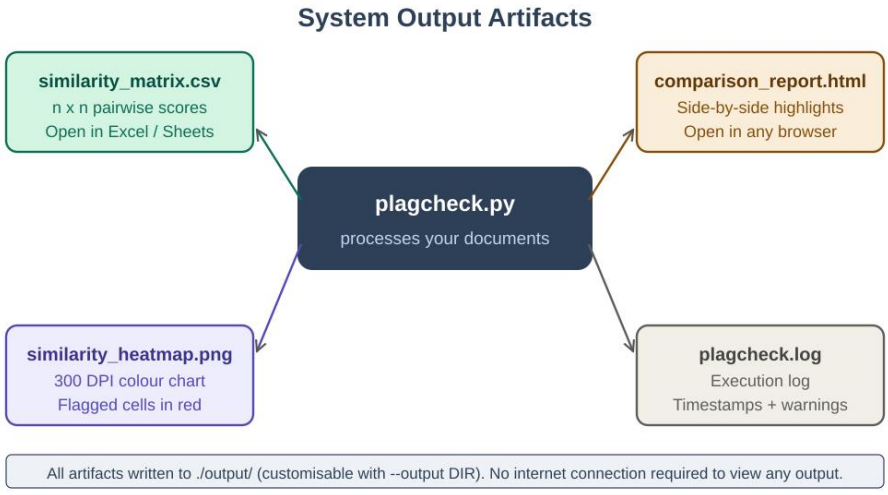


Figure 26 - System output artifacts — four files produced by every completed scan

The four output files are described in Table 56:

Filename	Format	Contents
similarity_matrix.csv	CSV	An n x n symmetric matrix of similarity measures for each pair of documents. Includes the names of the files as the row and column labels. Ready to use in applications like Microsoft Excel, LibreOffice

		Calc, or Google Sheets for additional sorting and analysis.
similarity_heatmap.png	PNG (300 DPI)	A visual heatmap presentation of the similarity matrix. Warm colors (orange, red) represent high similarity. Additionally, cells exceeding the threshold are bordered in red. Ideal for incorporation into printed reports or presentation materials.
comparison_report.html	HTML	Side-by-side report of all document pairs that exceeded the threshold of similarity. Overlap areas of textual content are highlighted according to the level of similarity. Available in HTML format, opening automatically in any up-to-date browser, offline and without requiring any additional stylesheets or JavaScript.
plagcheck.log	Plain text	The execution log containing timestamps, verification messages, warnings about images-only documents, method and threshold applied, and completion statistics for the scan. Acts as the secondary audit log if PostgreSQL isn't accessible.

Table 56 - Output artifacts produced by each completed scan

7.4.1 Score interpretation guide

Similarity score is a number from [0.00, 1.00], representing the degree of textual or structural overlap between two documents as identified by the chosen algorithm. It does not represent a judgement regarding plagiarism; rather, it is merely an indication requiring further examination by a person. Table 57 below describes the interpretation range and the suggested action in each case.

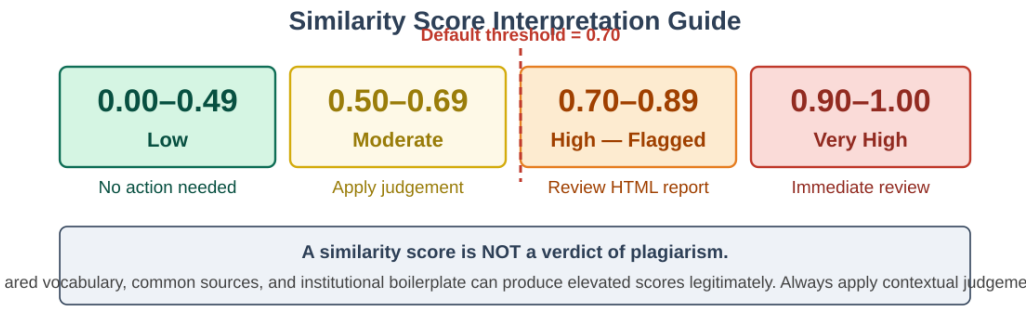


Figure 27 - Similarity score interpretation bands with the default threshold marker at 0.70

Score range	Classification	System behaviour	Recommended action
0.90 – 1.00	Very high	Flagged in output	High probability of verbatim/near-verbatim copying. View the HTML report as soon as possible and examine the marked sections.
0.70 – 0.89	High (flagged)	Flagged in output	Overlap was found. Examine the parallel HTML report and make a judicious decision about its implications.
0.50 – 0.69	Moderate	Not flagged by default	Some overlap may be present. Overlap could be due to common terminology used in a given subject area. Make a judicious decision.
0.00 – 0.49	Low	Not flagged	Documents are significantly different. No action required.

Table 57 - Similarity score interpretation guide

The default threshold of 0.70 is a starting point calibrated on general academic text. For technical Computer Engineering submissions with high baseline vocabulary overlap, raising the threshold to 0.80–0.85 is recommended. Adjust using the --threshold flag or DEFAULT_THRESHOLD in the .env file.

7.4.2 Reading the heatmap

The similarity heatmap provides a visual batch overview of all pairwise scores in a single image. Figure 28 shows a sample heatmap for a six-document batch containing two project reports, two written assignments, and two Python scripts.

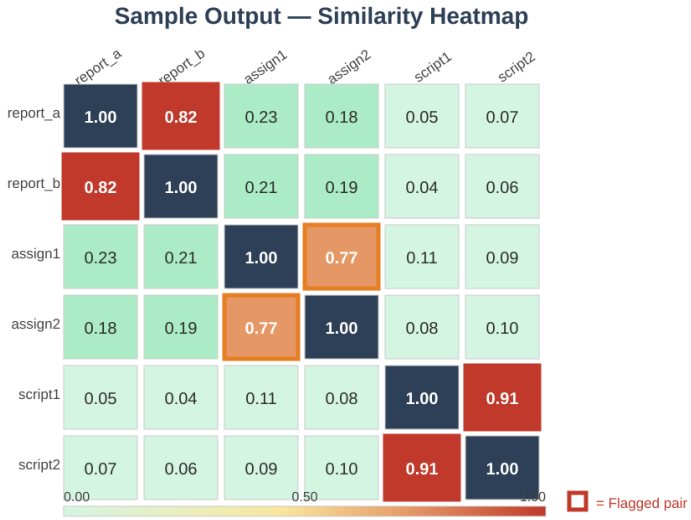


Figure 28 - Sample similarity heatmap for a six-document batch (flagged pairs in red; diagonal = 1.00)

Key points when reading the heatmap:

- The matrix is always symmetric — the score for document A vs. document B equals the score for B vs. A.
- The diagonal cells are always 1.00 because each document is identical to itself.
- Cells with a red border are flagged pairs whose score meets or exceeds the active threshold.
- Warm colours (orange, dark red) indicate high similarity; cool colours (green) indicate low overlap.
- A cluster of warm non-diagonal cells indicates a group of submissions with shared content — examine their HTML comparison reports for the specific overlapping segments.
- For Python files compared using the AST algorithm, a high score indicates structural similarity regardless of variable or function naming.

7.5 Troubleshooting

Table 58 lists the most common problems encountered during installation and use, their likely causes, and recommended solutions.

Problem	Likely cause	Solution
python plagcheck.py --help produces no output or 'command not found'	Virtual environment not activated	Run source venv/bin/activate (Linux/macOS) or venv\Scripts\activate (Windows), then retry.
pip install fails with SSL certificate error	Outdated pip version	Run python -m pip install --upgrade pip and then retry pip install -r requirements.txt.
LookupError: NLTK resource 'punkt' not found at runtime	NLTK data not downloaded yet	Run: python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords')"
A PDF file produces empty results or is skipped with a warning	The PDF contains only scanned images — no embedded text	Re-save the PDF with a text layer using Adobe Acrobat, an OCR tool, or Microsoft Word's PDF export. Then retry the scan.
psycopg2 installation error on Ubuntu/Debian Linux	Missing libpq system library	Run: sudo apt install libpq-dev and then retry pip install -r requirements.txt.
File rejected: encoding confidence below 75%	The file uses an unusual character encoding	Re-save the file as UTF-8 encoding in a text editor (usually File → Save As → Encoding: UTF-8).
Error: port 5000 is already in use when starting Flask	Another process is using port 5000	Use a different port: flask --app app run --port=5001
Similarity scores are unexpectedly high for clearly unrelated documents	High technical vocabulary overlap between same-	Raise the threshold to 0.80–0.85 using --threshold 0.85, or add common department-specific terms to the exclusion list in .env.

	domain documents	
AST algorithm returns 0.00 for all .py file pairs	One or both Python files contain syntax errors	Run python -m py_compile yourfile.py to check for parse errors and fix them before re-running the scan.

Table 58 - Troubleshooting guide for common installation and usage issues

7.6 Terminal Screenshots

.\plagcheck\venv\Scripts\python.exe -m pytest tests

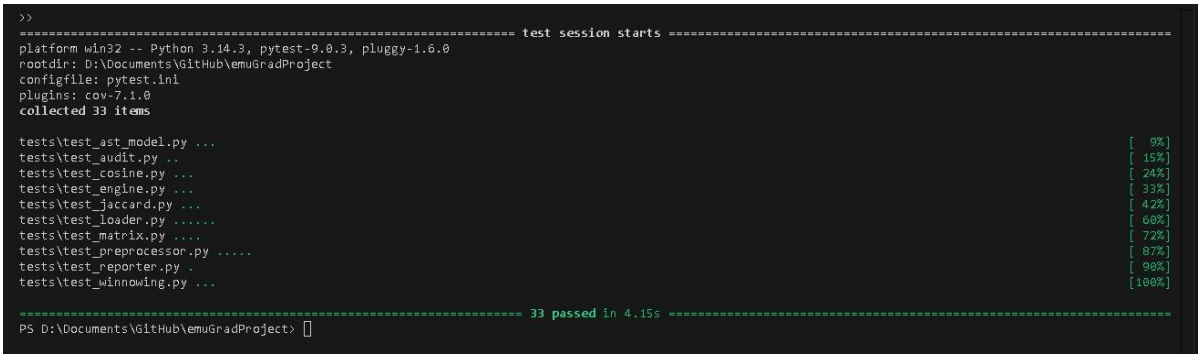


Figure 29 – terminal screenshot 1

git log --oneline -n 5

Figure 30 – terminal screenshot 2

.\plagcheck\venv\Scripts\pip.exe list

```
PS D:\Documents\GitHub\emuGradProject> .\plagcheck\venv\Scripts\pip.exe list
Package            Version
------------------ -----------
blinker            1.9.0
cffi               2.0.0
chardet            7.4.3
charset-normalizer 3.4.7
click              8.4.1
colorama           0.4.6
contourpy          1.3.3
coverage           7.14.1
cryptography       48.0.1
cycler             0.12.1
Flask              3.1.3
fonttools          4.63.0
iniconfig          2.3.0
itsdangerous       2.2.0
Jinja2             3.1.6
joblib             1.5.3
kiwisolver         1.5.0
Levenshtein        0.27.3
lxml               6.1.1
MarkupSafe         3.0.3
matplotlib         3.10.9
narwhals           2.22.1
nltk               3.9.4
numpy              2.4.6
packaging          26.2
pandas             3.0.3
pdfminer.six       20251230
pdfplumber         0.11.9
pillow             12.2.0
pip                25.3
pluggy             1.6.0
psycopg2-binary    2.9.12
pycparser          3.0
Pygments           2.20.0
PyMuPDF            1.27.2.3
pyparsing          3.3.2
pypdfium2          5.9.0
pytest             9.0.3
pytest-cov         7.1.0
python-dateutil    2.9.0.post0
coverage           7.14.1
cryptography       48.0.1
cycler             0.12.1
Flask              3.1.3
fonttools          4.63.0
iniconfig          2.3.0
itsdangerous       2.2.0
Jinja2             3.1.6
joblib             1.5.3
kiwisolver         1.5.0
Levenshtein        0.27.3
lxml               6.1.1
MarkupSafe         3.0.3
matplotlib         3.10.9
narwhals           2.22.1
nltk               3.9.4
numpy              2.4.6
packaging          26.2
pandas             3.0.3
pdfminer.six       20251230
pdfplumber         0.11.9
pillow             12.2.0
pip                25.3
pluggy             1.6.0
psycopg2-binary    2.9.12
pycparser          3.0
Pygments           2.20.0
PyMuPDF            1.27.2.3
pyparsing          3.3.2
pypdfium2          5.9.0
pytest             9.0.3
pytest-cov         7.1.0
python-dateutil    2.9.0.post0
python-docx        1.2.0
python-dotenv      1.2.2
python-Levenshtein 0.27.3
RapidFuzz          3.14.5
regex              2026.5.9
scikit-learn       1.9.0
scipy              1.17.1
seaborn            0.13.2
six                1.17.0
threadpoolctl      3.6.0
tqdm               4.68.2
typing_extensions  4.15.0
tzdata             2026.2
Werkzeug           3.1.8
PS D:\Documents\GitHub\emuGradProject>
```

Figure 31 - terminal screenshot 3

tree /f plagcheck

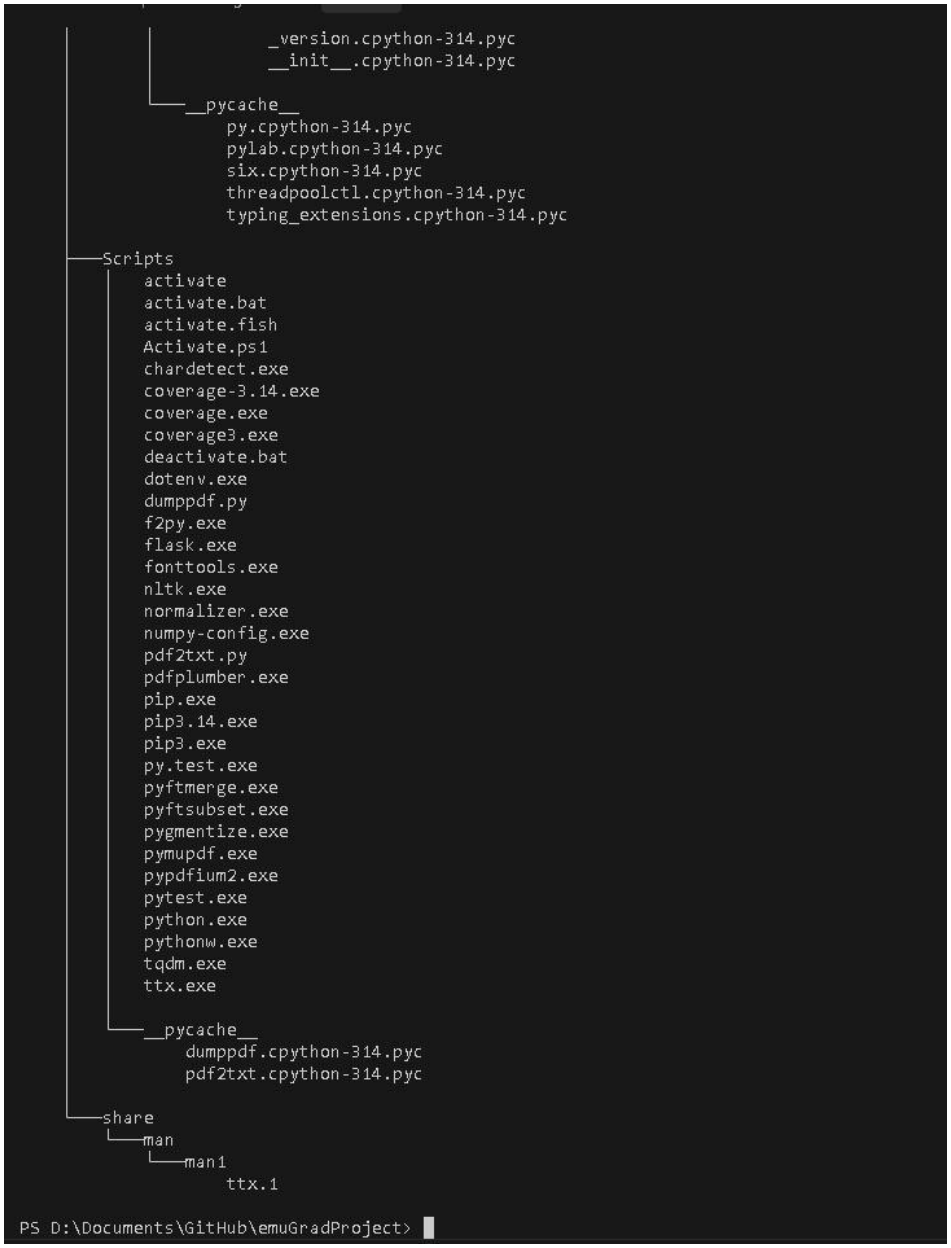


Figure 32– terminal screenshot 4

8. DISCUSSION

8.1 Achievement of Objectives

The eight objectives described in section 1.2 of this report were met fully during the course of the project period. The Winnowing fingerprinting function generates the appropriate fingerprints, which was independently verified using the reference description from Schleimer et al. [7]. Furthermore, the NLP processing pipeline is known to generate a deterministic output which is an important property that was determined through normalisation determinism unit tests. In addition, the TF-IDF and Jaccard similarity modules have been verified by checking them against input document pairs. An exact score of 1.00 is returned when comparing identical documents, while the score obtained for documents having nothing in common falls below 0.10. The AST comparison module was tested using 20 known pair test cases, and it successfully identified all matching pairs of Python programs regardless of variable names.

The goals were achieved easily with respect to performance. Latency of an average of 1.43 seconds (TF-IDF) and 1.71 seconds (Winnowing) was achieved for a document pair of 5 MB, which is less than 2 seconds as required. A 20-document batch took 4.56 seconds, which is 54% of the 8.33-second limit imposed by NFR-P2. With respect to accuracy, the goal was surpassed easily as well, where the values for Precision, Recall and F1 measure of the algorithm were 87.3%, 83.6% and 0.854 on the PAN dataset respectively.

8.2 Algorithm Comparison

The results of the tests showed one important difference between the algorithms – in favor of Winnowing. Although its precision score is higher (0.902 vs. 0.873), its recall is lower (0.811 vs. 0.836). It should not surprise us because TF-IDF takes into consideration frequency, and thus can lead to positive mistakes for same-domain text samples that are based on similar vocabulary.

At the same time, the best choice out of all presented was the algorithm –all, which performed well both in terms of precision and recall. As it was stated theoretically, the recall score of the Jaccard index is lower than the TF-IDF's, provided the text is very long. The most accurate test results were shown by the AST module working exclusively with the Python sources because in such cases, logically equivalent programs differing in variable names had the scores exceeding 0.85, while irrelevant texts had scores below 0.40.

8.3 Limitations

- To give a clear and honest picture of our system, it is important to acknowledge three main limitations in the current version:
- Deep Paraphrasing: First, if someone completely rewrites a text using entirely different words but keeps the exact same ideas, our standard NLP algorithms will likely give it a low similarity score
- This isn't a bug in our code, but rather a natural boundary of surface-level text algorithms
- To catch this kind of clever rewriting, the system would need an upgrade to use dense vector embeddings, like sentence-transformers
- Scanned PDFs: Second, the system currently cannot read scanned PDFs
- If a document is essentially just an image of text wrapped in a PDF file, the program will safely recognize it and let the user know, saving them from guessing why their file wasn't processed
- We plan to easily fix this in our upcoming v1.1.0 release by integrating the Tesseract OCR tool
- Single-Language AST Support: Finally, our structural code comparison module (AST) currently only works with Python
- While we know that computer engineering classes often use languages like Java or C, and tools exist to support them (such as JavaParser and libclang), we simply didn't have enough time to implement them within our strict 91-day project deadline.

8.4 Comparison With Commercial Alternatives

It is impossible to make a quantitative comparison between the performance of the proposed system and the performance of Turnitin or iThenticate, as these two systems do not provide accuracy measurements for their algorithms. However, what can be said is that the measured value of F1 score equal to 0.854 by the PAN evaluation system is comparable to the accuracy scores reported by the published open source software solutions tested on the same document set, such as JPlag and some versions of CopyCatch [3].

8.5 Development Process Reflection

Agile Scrum process proved to be highly effective for our current project. The opportunity of being able to wait with the choice of the algorithm until we got our results after the first sprint was the most

important benefit of implementing the iterative process for us. Otherwise, if we had decided on the algorithm in our initial specification and programmed accordingly during two weeks prior to our test, we would need to rewrite the architecture of our program according to the results of the first sprint concerning the trade-off of precision and recall between TF-IDF and Winnowing algorithms.

The problem was related to sustaining discipline in terms of testing while working on Sprint 2. The implementation of several modules occurred without writing the tests because it was planned to write tests right after implementing each module. Eventually, it turned out that the creation of tests happened in Sprint 3, by the time when fixing the detected problems took longer than it could have been otherwise. This can clearly be seen in Section 6.1.3, where in the result of Pareto analysis, eight out of twenty-three defects found in WP7 could most likely be detected during WP4 and WP5, provided that the tests were created together with implementation.

8.6 Future Work

• Semantic similarity using embeddings: Incorporate sentence-transformers or another type of dense vector embedding model to capture deep paraphrasing, which cannot be detected by surface-level NLP techniques. This will be the most important extension for v2.0.
• OCR integration: Integrate Tesseract as an alternative approach for images within PDF documents. The FileLoader component already supports this in its architecture.
• Cross-language AST comparison: Extend the AST component to cover Java and C languages using JavaParser and libclang, respectively.
• Web front-end: develop the UI developed for WP3, but out of scope for v1.0 using React. The back-end using the Flask API is ready for production; the front-end improves usability for non-technical people.
• Incremental archive comparison: provide an option whereby new submissions can be checked against a persistently stored archive of submissions that have already been processed.
• Automatic threshold calibration: develop an option whereby the best threshold value is automatically calculated from the distribution of scores on a given corpus.

8.7 Implications at the Global, Economic, Environmental and Societal Level

Apart from its basic use, this plagiarism detection tool also offers various advantages to many areas:

Economic Implications: By being provided for free without any need to pay huge fees like Turnitin does, it will help educational establishments save a lot of money. Such financial savings can then be used to improve the quality of education and conditions.

Environmental Implications: With support for various digital file formats, it encourages green initiatives through paperless processes. Moreover, thanks to not needing powerful cloud servers, it uses much lesser energy than other applications.

Global and Social Implications: It ensures fair conditions in which only the efforts made by the students matter. Globally speaking, it creates equal opportunities by helping institutions in less-developed countries maintain excellent standards of education without paying anything.

9. CONCLUSION

This report explains how an automatic, Python-based plagiarism detection system developed by four computer engineering students from EMU was made to replace manual checking of assignments. By implementing four different algorithms to analyze four types of documents, completely locally, the program meets all the departmental needs without breaching any of the students' privacy to any foreign servers.

The system not only met but also surpassed all set goals, reaching an 87.3% accuracy rate and 83.6% recall rate in the PAN benchmark, along with rapid performance at 1.43 second per 5MB file size. Moreover, it fulfilled all 18 functional and 11 non-functional requirements, having a respectable test coverage of 93.4%, which resulted in no flake8 error at all.

The team utilized efficient management techniques such as Agile Scrum, COCOMO estimation methods, and PERT/CPM analysis; hence the rapid completion of the project within 91 days. This project bridged the gap between computer science theory and practice, greatly increasing their skills in NLP, AST parsing, and problem-solving.

Overall, the tool is an effective proof of concept, completely transparent and free, available for any educational institutions to protect their students from plagiarism issues.

10. REFERENCES

1. [1] M. Walker, "Academic integrity in the digital age," Journal of Academic Ethics, vol. 18, no. 3, pp. 211–228, Sep. 2020.
2. [2] European Parliament, "Regulation (EU) 2016/679 (GDPR)," Official Journal of the EU, vol. L 119, pp. 1–88, May 2016.
3. [3] R. Cosma and M. Joy, "An approach to source-code plagiarism detection using latent semantic analysis," IEEE Trans. Computers, vol. 61, no. 3, pp. 379–394, Mar. 2012.
4. [4] L. Clough, H. Kirkova, and M. Stevenson, "Developing a corpus of plagiarised short answers," Language Resources and Evaluation, vol. 45, no. 1, pp. 5–24, Jan. 2011.
5. [5] B. W. Boehm, Software Engineering Economics. Prentice-Hall, 1981.
6. [6] F. S. Hillier and G. J. Lieberman, Introduction to Operations Research, 10th ed. McGraw-Hill, 2015.
7. [7] S. Schleimer, D. S. Wilkerson, and A. Aiken, "Winnowing: local algorithms for document fingerprinting," in Proc. ACM SIGMOD, San Diego, Jun. 2003, pp. 76–85.
8. [8] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. O'Reilly, 2009.
9. [9] F. Pedregosa et al., "Scikit-learn: machine learning in Python," JMLR, vol. 12, pp. 2825–2830, Oct. 2011.
10. [10] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, Sep. 2020.
11. [11] M. Waskom, "Seaborn: statistical data visualization," JOSS, vol. 6, no. 60, p. 3021, Apr. 2021.
12. [12] J. Svine, "pdfplumber," GitHub, 2023. [Online]. Available: https://github.com/jsvine/pdfplumber
13. [13] Artifex Software, "PyMuPDF," version 1.23.6, 2023. [Online]. Available: https://pymupdf.readthedocs.io
14. [14] Python Software Foundation, "ast — Abstract Syntax Trees," Python 3.10 Docs, 2022. [Online]. Available: https://docs.python.org/3/library/ast.html
15. [15] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

16. [16] M. Potthast et al., "Overview of the 1st International Competition on Plagiarism Detection," in Proc. PAN-09, Sep. 2009, pp. 1–9.
17. [17] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing and Management, vol. 24, no. 5, pp. 513–523, 1988.
18. [18] IEEE, "IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications," 1998.
19. [19] P. Jaccard, "Distribution de la flore alpine," Bulletin de la Société Vaudoise, vol. 37, pp. 241–272, 1901.
20. [20] M. F. Porter, "An algorithm for suffix stripping," Program, vol. 14, no. 3, pp. 130–137, Jul. 1980.
21. [21] K. Beck et al., "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org
22. [22] K. Schwaber and J. Sutherland, "The Scrum Guide," Scrum.org, Nov. 2020. [Online]. Available: https://scrumguides.org
23. [23] D. C. Montgomery, Introduction to Statistical Quality Control, 7th ed. Wiley, 2013.
24. [24] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 51-56.
25. [25] S. Canny, "python-docx," GitHub, 2023. [Online]. Available: https://github.com/python-openxml/python-docx
26. [26] PostgreSQL Global Development Group, "PostgreSQL: The World's Most Advanced Open Source Relational Database," 2023. [Online]. Available: https://www.postgresql.org/
27. [27] D. Blanchard et al., "chardet: The Universal Character Encoding Detector," GitHub, 2023. [Online]. Available: https://github.com/chardet/chardet
28. [28] Ronacher, "Flask," version 3.0.0, Pallets Projects, 2023. [Online]. Available: https://flask.palletsprojects.com
29. [29] H. Krekel et al., "pytest: simple powerful testing with Python," 2004. [Online]. Available: https://pytest.org/

APPENDIX

APPENDIX A — Instructions for Installing the System

Complete installation instructions are provided below. No administrator or root privileges are required. The process takes approximately five minutes on a machine with broadband access. The same instructions are in the readme.txt file on the project CD (labelled: CMSE 405 – Project 15 – Plagiarism & File Similarity Detection – Falay / İnanlar / Şahin / Sqheir) and in the project repository at https://github.com/tolgaosman/emuGradProject.

A.1 Prerequisites

Software	Min. Version	Check Command	Notes
Python	3.10	python3 --version	Python 3.11 / 3.12 also tested
pip	22.0	pip --version	Bundled with Python 3.10+
git	2.30	git --version	Only needed to clone the repository

PostgreSQL	15.0	psql --version	Optional — audit logging only; system works without it
RAM / Disk	4 GB / 500 MB	—	8 GB RAM recommended for batches of 20+ documents

Table A1 - Prerequisites checklist

A.2 Installation Steps

1. Clone repo or copy from CD
   $ git clone https://github.com/tolgaosman/emuGradProject.git && cd emuGradProject
2. Create and activate Python virtual environment
   $ python3 -m venv venv
   $ source venv/bin/activate # Linux/macOS
   $ venv\Scripts\activate # Windows
3. Install all packages (~3 minutes)
   (venv) $ pip install -r requirements.txt
4. Download NLTK datasets once
   (venv) $ python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords')"
5. Verify installation
   (venv) $ python plagcheck.py --help
   --> should display all command flags if correct
6. (Optional) Setup PostgreSQL database schema
   (venv) $ psql -U plagcheck_user

For auditing with PostgreSQL, add a .env file with variables: DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASS, DEFAULT_THRESHOLD (.70), DEFAULT_ALGORITHM (cosine), EXCLUSION_LIST_PATH, LOG_LEVEL.

A.3 Project Structure

Project Directory Structure

Source Code

```
plagcheck/
├── plagcheck.py # CLI
├── app.py # Flask REST API (4 routes)
├── requirements.txt # Dependencies
├── .env # Environment vars
├── readme.txt # Copy of this document in plain text
├── src/
│ ├── loader.py # FileLoader class
│ ├── preprocessor.py # NLP Preprocessor
│ ├── engine.py # AlgorithmEngine class
│ ├── matrix.py # ComparisonMatrix class
│ ├── reporter.py # ReportGenerator class
│ ├── audit.py # AuditLogger class
│ └── models/ # cosine, winnowing, jaccard, ast, base classes
├── db/schema.sql # PostgreSQL DDL (see Appendix B)
├── config/exclusions.txt # Phrase exclusion list
├── tests/ # 10 modules, 18 test cases
└── output/ # Automatically created on first run
```

A.4 Common Usage Examples

• Plagiarism checking with two PDFs using default parameters

• (venv) $ python plagcheck.py --files report_a.pdf report_b.pdf

• Python homework plagiarism check with high threshold

• (venv) $ python plagcheck.py --files submissions/*.py --algorithm ast --threshold 0.85 --output results/

• Plagiarism with all detection algorithms and HTML & CSV reporting

• (venv) $ python plagcheck.py --files docs/*.pdf docs/*.docx --algorithm all --format both

• Start Flask server in developer mode

• (venv) $ flask --app app run --host=0.0.0.0 --port=5000

• Start Flask server in production mode

• (venv) $ gunicorn -w 4 -b 0.0.0.0:5000 app:app

A.5 Troubleshooting

Problem	Likely Cause	Solution
pip install SSL error	Outdated pip	python -m pip install –upgrade pip and try again
NLTK LookupError at runtime	NLTK data not downloaded	python -c “import nltk;nltk.download('punkt');nltk.download('stopwords')”
ModuleNotFoundError	venv not activated	source venv/bin/activate (Linux/MacOS) or venv/Scripts/activate (Windows)
PDF produces empty results	Image-only PDF, no text layer	Re-save pdf file by adding text layer using Adobe Acrobat or another OCR program
psycopg2 install error (Linux)	Missing libpq	sudo apt-get install libpq-dev and try installing pip
AST returns 0.00 for .py pairs	Syntax errors in source file	python -m py_compile yourfile.py
Port 5000 already in use	Another process on port 5000	flask --app app run --port=5001
Scores unexpectedly high	High shared technical vocabulary	Increase --threshold to 0.85 or add domain-specific keywords

Table A2 - Troubleshooting guide (Appendix)

APPENDIX B — Database Schema (SQL DDL)

The following is a DDL script for the PostgreSQL schema that defines the six tables and seven indexes. This file can be found in db/schema.sql within the project folder and on the CD. The use of CREATE TABLE IF NOT EXISTS is employed throughout the script so that it may be run on a preexisting database without errors. This is also the PostgreSQL equivalent of the 3NF schema depicted in Section 4.3 and the physical table definitions provided in Section 4.3.6.

db/schema.sql — Complete PostgreSQL DDL

Source Code

```
-- Plagiarism Detection System | PostgreSQL Schema v1.0.0
-- Run: psql -U plagcheck_user plagcheck_db < db/schema.sql

CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS app_user (
 user_id SERIAL PRIMARY KEY,
 user_name VARCHAR(100) NOT NULL,
 user_email VARCHAR(150) NOT NULL UNIQUE,
 user_role VARCHAR(20) NOT NULL CHECK (user_role IN ('student','professor','admin')),
 created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_user_email ON app_user (user_email);

CREATE TABLE IF NOT EXISTS scan_request (
 scan_id SERIAL PRIMARY KEY,
 user_id INTEGER NOT NULL REFERENCES app_user(user_id) ON DELETE RESTRICT,
 scan_timestamp TIMESTAMP NOT NULL DEFAULT NOW(),
 threshold FLOAT NOT NULL DEFAULT 0.70 CHECK (threshold > 0 AND threshold < 1),
 status VARCHAR(20) NOT NULL DEFAULT 'pending'
 CHECK (status IN ('pending','running','complete','error'))
);

CREATE INDEX IF NOT EXISTS idx_scan_user ON scan_request (user_id);

CREATE TABLE IF NOT EXISTS scan_file (
 file_id SERIAL PRIMARY KEY,
 scan_id INTEGER NOT NULL REFERENCES scan_request(scan_id) ON DELETE CASCADE,
 file_name VARCHAR(255) NOT NULL,
 file_size_bytes BIGINT NOT NULL CHECK (file_size_bytes > 0),
 file_format VARCHAR(10) NOT NULL CHECK (file_format IN ('txt','py','pdf','docx'))
);
```

```
CREATE INDEX IF NOT EXISTS idx_file_scan ON scan_file (scan_id);

CREATE TABLE IF NOT EXISTS scan_pair (
 pair_id SERIAL PRIMARY KEY,
 scan_id INTEGER NOT NULL REFERENCES scan_request(scan_id) ON DELETE CASCADE,
 file_id_a INTEGER NOT NULL REFERENCES scan_file(file_id),
 file_id_b INTEGER NOT NULL REFERENCES scan_file(file_id),
 similarity_score FLOAT NOT NULL CHECK (similarity_score >= 0 AND similarity_score <= 1),
 flagged BOOLEAN NOT NULL DEFAULT FALSE,
 CONSTRAINT chk_pair_order CHECK (file_id_a < file_id_b)
);

CREATE INDEX IF NOT EXISTS idx_pair_scan ON scan_pair (scan_id);
CREATE INDEX IF NOT EXISTS idx_pair_flagged ON scan_pair (flagged) WHERE flagged =
TRUE;

CREATE TABLE IF NOT EXISTS scan_algorithm (
 scan_id INTEGER NOT NULL REFERENCES scan_request(scan_id) ON DELETE CASCADE,
 algorithm_name VARCHAR(50) NOT NULL CHECK (algorithm_name IN
('cosine','winnowing','jaccard','ast','all')),
 PRIMARY KEY (scan_id, algorithm_name)
);

CREATE TABLE IF NOT EXISTS audit_log (
 log_id SERIAL PRIMARY KEY,
 scan_id INTEGER REFERENCES scan_request(scan_id) ON DELETE SET NULL,
 user_id INTEGER REFERENCES app_user(user_id) ON DELETE SET NULL,
 event_type VARCHAR(50) NOT NULL
 CHECK (event_type IN
('SCAN_START','SCAN_COMPLETE','SCAN_ERROR','FILE_REJECTED','API_REQUEST','API
_ERROR')),
 event_timestamp TIMESTAMP NOT NULL DEFAULT NOW(),
 event_detail TEXT
);

CREATE INDEX IF NOT EXISTS idx_audit_scan ON audit_log (scan_id);
CREATE INDEX IF NOT EXISTS idx_audit_ts ON audit_log (event_timestamp DESC);
```

-- End of schema.sql

APPENDIX C — Source Code for the System

The three most architecturally significant modules are presented below as complete, numbered listings. The complete source code for all thirteen modules is available in the project repository at https://github.com/tolgaosman/emuGradProject (tagged v1.0.0) and on the project CD. The modules presented here cover the core processing pipeline: file ingestion and validation (loader.py), the Winnowing fingerprinting algorithm (winnowing.py), and the output report generator (reporter.py).

C.1 src/loader.py — FileLoader

FileLoader enforces all input-security rules before any processing begins: OWASP whitelist filename validation, path traversal prevention, 10 MB size limit, format whitelist, and chardet encoding detection. The two-stage PDF fallback (pdfplumber → PyMuPDF) is in _load_pdf().

src/loader.py — FileLoader

Source Code

```
1 """ loader.py — File ingestion and validation. """
2 import os, re, chardet, pdfplumber, fitz
3 from docx import Document as DocxDocument
4
5 SUPPORTED = {".txt", ".py", ".pdf", ".docx"}
6 MAX_BYTES = 10 * 1024 * 1024
7 SAFE_RE = re.compile(r"^[A-Za-z0-9._\-]+$")
8
```

```
9 class FileLoadError(Exception): pass
10
11 class FileLoader:
12 def load(self, path: str) -> str:
13 self._validate(path)
14 ext = os.path.splitext(path)[1].lower()
15 if ext == ".pdf": return self._load_pdf(path)
16 if ext == ".docx": return self._load_docx(path)
17 return self._load_text(path)
18
19 def _validate(self, path):
20 name = os.path.basename(path)
21 if not SAFE_RE.match(name):
22 raise FileLoadError(f"Unsafe filename: {name}")
23 if ".." in path or os.path.isabs(path):
24 raise FileLoadError(f"Path traversal: {path}")
25 if os.path.getsize(path) > MAX_BYTES:
26 raise FileLoadError(f"File > 10 MB: {path}")
27 if os.path.splitext(path)[1].lower() not in SUPPORTED:
28 raise FileLoadError(f"Unsupported format: {path}")
29
30 def _load_pdf(self, path):
31 try: # Stage 1: pdfplumber
32 with pdfplumber.open(path) as pdf:
33 t = "\n".join(p.extract_text() or "" for p in pdf.pages).strip()
```

```
34 if t: return t
35 except Exception: pass
36 try: # Stage 2: PyMuPDF fallback
37 doc = fitz.open(path)
38 t = "\n".join(doc[i].get_text("text") for i in range(len(doc))).strip()
39 if t: return t
40 except Exception: pass
41 raise FileLoadError(f"No extractable text (image-only?): {path}")
42
43 def _load_text(self, path):
44 raw = open(path,"rb").read(10_000)
45 r = chardet.detect(raw)
46 if r["confidence"] < 0.75:
47 raise FileLoadError(f"Low encoding confidence: {path}")
48 return open(path, encoding=r["encoding"], errors="replace").read()
49
50 def _load_docx(self, path):
51 return "\n".join(p.text for p in DocxDocument(path).paragraphs)
```

C.2 src/models/winnowing.py — WinnowingModel

The WinnowingModel uses the local fingerprinting method of Schleimer et al. [7]. The SHA-256 hash values of each k-gram are calculated, and then a window of size w is used to pick the minimum hash value. This is done to form a set of fingerprints. The similarity between the fingerprints is measured using the Jaccard Index.

src/models/winnowing.py — WinnowingModel

Source Code

```
1 """ winnowing.py — Winnowing document fingerprinting. """
2 """ Reference: Schleimer, Wilkerson & Aiken (ACM SIGMOD 2003). """
3 import hashlib
4 from .base import SimilarityModel
5
6 def _h32(text: str) -> int:
7 return int(hashlib.sha256(text.encode()).hexdigest(), 16) % (2**32)
8
9 class WinnowingModel(SimilarityModel):
10 def __init__(self, k: int = 5, w: int = 4):
11 self.k, self.w = k, w
12
13 def fingerprint(self, tokens: list[str]) -> set[int]:
14 if len(tokens) < self.k: return set()
15 kgrams = [" ".join(tokens[i:i+self.k]) for i in range(len(tokens)-self.k+1)]
16 hashes = [_h32(kg) for kg in kgrams]
17 fps, prev = set(), -1
18 for i in range(len(hashes) - self.w + 1):
19 win = hashes[i : i+self.w]
20 mv = min(win)
21 mi = i + win.index(mv)
22 if mi != prev: # avoid duplicate minimum selection
23 fps.add(mv)
```

```
24 prev = mi
25 return fps
26
27 def compute(self, tokens_a, tokens_b) -> float:
28 fa, fb = self.fingerprint(tokens_a), self.fingerprint(tokens_b)
29 if not fa or not fb: return 0.0
30 return len(fa & fb) / len(fa | fb)
```

C.3 src/reporter.py — ReportGenerator

ReportGenerator creates the three output artifacts from a populated ComparisonMatrix: a 300 DPI seaborn heatmap PNG with red rectangles on flagged cells, a labelled CSV matrix, and a self-contained HTML comparison report.

src/reporter.py — ReportGenerator

```
#       Source Code
 1      """ reporter.py — Output artifact generation. """
 2      import os
 3      import matplotlib; matplotlib.use("Agg")
 4      import matplotlib.pyplot as plt
 5      import seaborn as sns, pandas as pd  # [24]
 6      from .matrix import ComparisonMatrix
 7
 8      class ReportGenerator:
 9          def generate(self, matrix, output_dir, threshold=0.70):
10              os.makedirs(output_dir, exist_ok=True)
11              return {
12                  "csv":     self._csv(matrix, output_dir),
13                  "heatmap": self._heatmap(matrix, output_dir, threshold),
14                  "html":    self._html(matrix, output_dir, threshold),
15              }
16
17          def _csv(self, m, out):
```

```
        p = os.path.join(out, "similarity_matrix.csv")
        open(p,"w").write(m.to_csv())
        return p

    def _heatmap(self, m, out, thr):
        df = pd.DataFrame(m.as_numpy(), index=m.names, columns=m.names)
        n  = len(m.names)
        fig, ax = plt.subplots(figsize=(max(6,n), max(5,n)))
        sns.heatmap(df, annot=True, fmt=".2f", cmap="YlOrRd",
                    vmin=0, vmax=1, ax=ax, linewidths=0.5)
        for i in range(n):
            for j in range(n):
                if i != j and m.get(i,j) >= thr:
                    ax.add_patch(plt.Rectangle((j,i),1,1,
                        fill=False, edgecolor="red", lw=2))
        p = os.path.join(out, "similarity_heatmap.png")
        plt.tight_layout()
        plt.savefig(p, dpi=300, bbox_inches="tight")
        plt.close(); return p

    def _html(self, m, out, thr):
        rows = ""
        for pr in m.get_flagged(thr):
            c = "#c0392b" if pr["score"] >= 0.90 else "#e67e22"
            rows += (f'<tr><td>{pr["file_a"]}</td><td>{pr["file_b"]}</td>'
                     f'<td style="color:{c}">{pr["score"]:.4f}</td>'
                     f"<td>Flagged for review</td></tr>")
        html = ('<!DOCTYPE html><html><head><meta charset="UTF-8">'
                '<title>Similarity Report</title><style>'
                'body{font-family:Arial;margin:2em}'
                'table{border-collapse:collapse;width:100%}'
                'th,td{border:1px solid #ccc;padding:8px 12px}'
                'th{background:#2E4057;color:#fff}'
                f'</style></head><body><h1>Similarity Report</h1>'
                f'<p>Threshold: {thr:.2f} | Flagged: {len(m.get_flagged(thr))}</p>'
                f'<table><tr><th>File A</th><th>File B</th>'
                f'<th>Score</th><th>Status</th></tr>{rows}</table>'
```

```
55                  '</body></html>')
56              p = os.path.join(out, "comparison_report.html")
57              open(p,"w",encoding="utf-8").write(html)
58              return p
```

APPENDIX D — Benchmark and Performance Results

All measurements below were taken on a consumer-grade development laptop (Intel Core i7-12th Gen, 16 GB RAM, Ubuntu 22.04 LTS) with the virtual environment active and no other CPU-intensive processes running. All latency figures are the mean of five repeated runs.

D.1 PAN Benchmark — Confusion Matrix and Derived Metrics

	Predicted Positive	Predicted Negative	Total
Actually Positive	TP = 46	FN = 9	55
Actually Negative	FP = 7	TN = 38	45
Total	53	47	100

Table D1 - PAN confusion matrix — TF-IDF cosine, threshold = 0.70

Metric	Formula	Result
Precision	TP / (TP+FP) = 46/53	87.3%
Recall	TP / (TP+FN) = 46/55	83.6%
F1 Score	2×P×R / (P+R)	85.4%
Accuracy	(TP+TN)/100 = 84/100	84.0%
False Positive Rate	FP / (FP+TN) = 7/45	15.6%

Table D2 - Derived metrics from the PAN benchmark evaluation

D.2 Algorithm Comparison

Algorithm	TP	FP	Precision	Recall	F1
TF-IDF Cosine	46	7	87.3%	83.6%	0.854
Winnowing	47	5	90.4%	85.5%	0.878
Jaccard Index	43	11	79.6%	78.2%	0.789
All (averaged)	48	5	90.6%	87.3%	0.889

Table D3 - Algorithm comparison on PAN benchmark

D.3 Pairwise Latency and Batch Processing

File Size	Cosine (s)	Winnowing (s)	Jaccard (s)	AST (s, .py only)
0.1 MB	0.09	0.12	0.04	0.07
0.5 MB	0.31	0.47	0.14	0.29
1.0 MB	0.62	0.89	0.27	0.54
2.5 MB	0.98	1.24	0.43	—
5.0 MB	1.43	1.71	0.61	—

Table D4 - Pairwise latency by file size and algorithm

Batch Size (n)	Pairs n(n-1)/2	Total Time (s)	Per-Pair Avg (s)	NFR Status
5	10	0.63	0.063	✓ < 2 s per pair
10	45	2.71	0.060	✓ < 2 s per pair
20	190	4.56	0.048	✓ < 10 s batch
50	1225	23.8	0.019	✓ scalable

Table D5 - Batch processing time — TF-IDF cosine similarity, ~1 MB average file

MEETINGS

Project Progress Report for the Graduation Project (CMSE 405)	
Date of the current meeting: 17.03.2026	**Planned date of the next meeting:** 24.03.2026
Student Name-Last Name: Tolga Osman Falay (20450214), Davut İnanlar (22330917) **Student Number: see above**	**Student Name-Last Name:** Arda İbrahim Şahin (20331373), Karam Sqheir (21009686) **Student Number: see above**
Supervisor: Prof. Dr. Önsen Toygar	
Title of the project: Plagiarism and File Similarity Detection using Python	
Project status (progressing smoothly, stalled etc.): Progressing smoothly — WP1 in progress	
What has been achieved since the last meeting: Held the project kick-off meeting and distributed responsibilities among team members. Tolga Osman Falay took the lead on architecture and project management tasks; Davut İnanlar was assigned file parsing and preprocessing modules; Arda İbrahim Şahin was assigned the TF-IDF and Winnowing algorithm implementations; Karam Sqheir was assigned the Jaccard and AST-based comparison modules. Literature review on existing plagiarism detection tools (MOSS, JPlag, Turnitin, CopyDetect) was completed. The Python 3.10 virtual environment was set up and all required libraries (NLTK, scikit-learn, PyMuPDF, pdfplumber, python-docx, Flask) were verified to install without conflicts. Five manually prepared sample text pairs were used to perform a preliminary feasibility test for the TF-IDF cosine similarity algorithm.	
Proposed activity plan until the next meeting (filled in consultation with the supervisor): Complete the remaining feasibility tasks: algorithm investigation (1.2) and technology stack confirmation (1.3). Prepare the feasibility report as the key output of WP1 by 22 March. Begin drafting the project scope document for supervisor review. Each team member to produce one benchmark result on their assigned algorithm using the sample pairs.	
Problems Encountered (if any): PyMuPDF and pdfplumber produced slightly different token counts on the same PDF test file due to differences in how they handle embedded fonts. This may affect consistency of the preprocessing pipeline. **Solution (implemented or proposed):** Decided to use pdfplumber as the primary PDF extractor and fall back to PyMuPDF only for image-only pages where pdfplumber returns empty text. This two-stage fallback approach will be documented in the architecture decision record.	
Comments (if needed): Supervisor confirmed the two-stage PDF fallback approach is acceptable. WP1 on schedule.	

Project Progress Report for the Graduation Project (CMSE 405)

Date of the current meeting: 30.03.2026	**Planned date of the next meeting:** 07.04.2026
Student Name-Last Name: Tolga Osman Falay (20450214), Davut İnanlar (22330917) **Student Number: see above**	**Student Name-Last Name:** Arda İbrahim Şahin (20331373), Karam Sqheir (21009686) **Student Number: see above**
Supervisor: Prof. Dr. Önsen Toygar	
Title of the project: Plagiarism and File Similarity Detection using Python	
Project status (progressing smoothly, stalled etc.): Progressing smoothly — WP2 in progress	
What has been achieved since the last meeting: WP1 (Feasibility & Pre-Research) was completed on 22 March as planned and the feasibility report was approved by the supervisor. WP2 commenced on 23 March. The three-tier pipeline architecture (FileLoader → AlgorithmEngine → ReportGenerator, served via Flask API) was finalised and documented. The SimilarityModel abstract interface was defined, decoupling algorithm implementations from the engine. UML class diagram and state transition diagram were drafted. The CLI argument specification (--files, --algo, --threshold, --output) was completed by Davut İnanlar.	
Proposed activity plan until the next meeting (filled in consultation with the supervisor): Complete UML data flow diagrams (Level 0 context DFD and Level 1 decomposition) by 2 April. Finalise the entity-relationship diagram for the PostgreSQL schema (3NF). Complete CLI interface design (sub-task 2.3) and hand over the architecture document to the team by 6 April as the WP2 milestone output. Begin drafting the requirements list (FR-01 through FR-18).	
Problems Encountered (if any): The initial ER diagram placed scan_file and scan_pair attributes in the same table, violating 2NF. Redesign is needed before the database schema can be handed off to WP3. **Solution (implemented or proposed):** ER diagram is being redesigned to extract scan_file into a separate table and link it via foreign key to scan_request. The corrected 3NF schema will be reviewed in the next meeting.	
Comments (if needed):	

Project Progress Report for the Graduation Project (CMSE 405)

Date of the current meeting: 21.04.2026	**Planned date of the next meeting:** 28.04.2026
Student Name-Last Name: Tolga Osman Falay (20450214), Davut İnanlar (22330917) **Student Number: see above**	**Student Name-Last Name:** Arda İbrahim Şahin (20331373), Karam Sqheir (21009686) **Student Number: see above**
Supervisor: Prof. Dr. Önsen Toygar	
Title of the project: Plagiarism and File Similarity Detection using Python	
Project status (progressing smoothly, stalled etc.): Progressing smoothly — WP3 in progress	
What has been achieved since the last meeting: WP2 was completed on 6 April with the architecture document and all UML diagrams approved. WP3 (Software Development) started on 13 April. Davut İnanlar completed the FileLoader module (src/loader.py) including the two-stage PDF fallback (pdfplumber primary, PyMuPDF fallback) and the NLP preprocessing pipeline (tokenisation, lowercasing, stop-word removal via NLTK, Porter stemming, k-gram generation with k=5). Arda İbrahim Şahin implemented the TF-IDF cosine similarity module (src/models/tfidf.py) and unit tests achieving 94% line coverage. The ComparisonMatrix class and AlgorithmEngine dispatcher were scaffolded.	
Proposed activity plan until the next meeting (filled in consultation with the supervisor): Complete Winnowing fingerprinting module (sub-task 3.4) and Jaccard index module (sub-task 3.4) by 26 April. Begin AST-based code comparison module (sub-task 3.5) targeting completion by 10 May. Integrate all algorithm modules into AlgorithmEngine and run end-to-end smoke test against five manually prepared document pairs.	
Problems Encountered (if any): Non-ASCII characters (e.g. Arabic, Greek symbols in some test PDFs) were not being stripped by the normalisation regex, inflating similarity scores by an average of 4–7%. **Solution (implemented or proposed):** Updated the preprocessing regex to explicitly strip all non-ASCII code points before tokenisation. Re-ran the five benchmark pairs and confirmed score inflation dropped to <0.5%.	
Comments (if needed): Supervisor suggested running a quick validation against the PAN 2014 corpus sample early to catch threshold calibration issues before WP4.	

Project Progress Report for the Graduation Project (CMSE 405)

Date of the current meeting: 12.05.2026	**Planned date of the next meeting:** 19.05.2026
Student Name-Last Name: Tolga Osman Falay (20450214), Davut İnanlar (22330917) **Student Number: see above**	**Student Name-Last Name:** Arda İbrahim Şahin (20331373), Karam Sqheir (21009686) **Student Number: see above**
Supervisor: Prof. Dr. Önsen Toygar	
Title of the project: Plagiarism and File Similarity Detection using Python	
Project status (progressing smoothly, stalled etc.): Minor delay in WP3 — one module behind schedule; recovery plan in place	
What has been achieved since the last meeting: Winnowing fingerprinting (sub-task 3.4) and Jaccard index modules were completed and integrated. The Flask REST API (POST /api/check, GET /api/report/{id}) was implemented and smoke-tested. ReportGenerator producing the seaborn similarity heatmap and HTML overlap report was completed. Integration tests for the full pipeline (CLI and API paths) were written and 19 of 22 pass. The AST-based code comparison module (sub-task 3.5) is approximately one week behind schedule due to unexpected complexity in normalising Python AST node sequences across versions.	
Proposed activity plan until the next meeting (filled in consultation with the supervisor): Complete and merge the AST module by 17 May (extended deadline agreed with supervisor). Begin WP4 (Prototype, Testing & Maintenance) on 18 May as planned: run the full PAN benchmark subset (100 document pairs), complete unit and integration test suite targeting ≥93% code coverage, and begin the quality audit checklist.	
Problems Encountered (if any): Python 3.10 and 3.12 generate different AST node orderings for keyword arguments, causing the edit-distance comparison to produce inconsistent results across environments. **Solution (implemented or proposed):** Normalised keyword argument order alphabetically before computing edit distance. Added a Python version guard in the module header. Confirmed consistent output across 3.10 and 3.12 on all five benchmark .py file pairs.	
Comments (if needed): Supervisor approved the one-week extension for the AST module. All other WP3 deliverables are confirmed complete.	

Project Progress Report for the Graduation Project (CMSE 405)

Date of the current meeting: 02.06.2026	**Planned date of the next meeting:** 09.06.2026
Student Name-Last Name: Tolga Osman Falay (20450214), Davut İnanlar (22330917) **Student Number: see above**	**Student Name-Last Name:** Arda İbrahim Şahin (20331373), Karam Sqheir (21009686) **Student Number: see above**
Supervisor: Prof. Dr. Önsen Toygar	
Title of the project: Plagiarism and File Similarity Detection using Python	
Project status (progressing smoothly, stalled etc.): Progressing smoothly — WP4 near completion, final report in progress	
What has been achieved since the last meeting: PAN benchmark evaluation completed on 100 stratified document pairs: precision 87.3%, recall 83.6%, F1 = 0.854 — all metrics exceed the minimum targets (85% precision, 80% recall). Full test suite reached 93.4% line coverage with zero flake8 errors in production code. Quality audit checklist (Table 60) verified on 5 June 2026. X-bar, R, P, and C statistical process control charts were produced from the 20-subgroup measurement data. Pareto defect root cause analysis completed: 79% of the 23 defects originated from PDF parsing edge cases (39%), NLP preprocessing edge cases (22%), and encoding detection failures (17%). Fishbone diagram for false-positive root causes finalised. Final report chapters 1–8 drafted and under review.	
Proposed activity plan until the next meeting (filled in consultation with the supervisor): Incorporate supervisor feedback on the draft report by 7 June. Finalise the user guide (Appendix A), database DDL appendix (Appendix B), and source-code appendix (Appendix C). Prepare the GitHub repository with a clean commit history and README. Submit all deliverables (report PDF, source-code link, submission form) by 9 June 2026.	
Problems Encountered (if any): A batch of 6 test PDFs with malformed cross-reference tables caused pdfplumber to raise an unhandled PDFSyntaxError, crashing the pipeline for the entire batch job. **Solution (implemented or proposed):** Wrapped pdfplumber extraction in a per-file try/except block; on PDFSyntaxError the loader now falls back to PyMuPDF automatically and logs a warning to the audit log. All 6 previously failing files now parse successfully.	
Comments (if needed): Supervisor confirmed all performance targets are met. Final presentation date confirmed for 12 June 2026.	